IMPERIAL

MEng Individual Project

Imperial College London

Department of Computing

Beyond Before-and-After: Process-Quality Evaluation of Refactoring Sessions

Supervisor:
Dr Robert Chatley

Author:
Ethan Hosier

Second Marker:
Dr Cristian Cadar

May 26, 2026

Abstract

Your abstract goes here

Contents

Chapter 1

Introduction

1.1 Background

Refactoring is a routine activity in software development in which developers restructure code to improve readability, maintainability, and evolvability while preserving externally observable behaviour; traditional definitions describe it as a sequence of small, semantics-preserving transformations [1, 2, 3], and subsequent surveys categorise these refactoring operations and the tools that support them [4]. In practice, refactoring is widely encouraged because poorly structured code becomes an obstacle when enhancing and maintaining code, and long-lived systems need continual adjustment (as argued in work building on Lehman's laws and empirical studies of software maintenance). At the same time, refactoring can be costly: it consumes developer time, introduces risk, and is frequently intertwined with other work such as feature development and bug fixing rather than occurring as isolated "refactoring-only" tasks [5, 6].

1.2 Motivation

Most research and tooling that evaluates refactoring focuses on *outcomes* rather than the *process*. A common approach is to measure refactoring success as a change in static quality indicators between a "before" and "after" state: reductions in code smells, improvements in cohesion/coupling, decreases in complexity, or related metrics to do with maintainability. Typical strands of work include metric-driven detection or prioritisation of candidate refactorings (e.g., using metric changes to separate refactoring-like edits from functional ones, or to choose between different refactoring options), as well as refactoring recommendation systems that rank alternatives by their impact on design metrics. More recent work expands the signals used for guidance by incorporating traceability alongside code metrics when ranking refactoring solutions, and exploring how product/process metrics relate to developers' motivations to refactor (e.g., studies building on mined refactorings and repository history, and proposals that use LLMs to determine developers' motivations from commit artefacts). Alongside this, there is a substantial line of work that has improved the ability to *detect* refactorings from code changes, notably by using AST-based differencing and structured matching to infer refactoring operations between revisions (e.g., RefactoringMiner and related approaches), which has enabled large-scale empirical studies of how often refactoring happens, what kinds occur, and in what contexts [7, 8].

However, these perspectives evaluate refactoring as a mapping from an initial program state to a final program state, leaving the *trajectory* between those states largely outside the picture. This matters because developers rarely refactor in a single atomic operation: they work through intermediate states, sometimes take detours, redo work, and often accept temporary degradation (e.g., in readability, modular structure, or test/build stability) before arriving at a final outcome. Two developers can arrive at the same end state via very different paths: one might take lots of disruptive steps, bounce between designs, or create avoidable churn, while another might get there via a steadier, more stable sequence. Existing endpoint-only evaluation cannot distinguish these different cases. Meanwhile, research on process realism suggests that refactoring is often "flossed" into ongoing development instead of being a separate dedicated task, which increases the chance of mixed-intent steps and noisy traces [5]. This makes the quality of the process (how a developer navigates intermediate states) potentially important for both developer productivity and the health of the codebase as it evolves.

Recent work has started to look more closely at refactoring trajectories themselves. RefactorBench, for example, uses a state/action/observation view to study how language-model agents fail on multi-file refactoring tasks [9], while Bouzenia et al. characterise recurring action patterns in software-engineering agent traces [10]. A recent systematic literature review also surveys how LLM-based refactoring work is evaluated and highlights several open problems in this area [11]. However, this still leaves a gap for evaluating an observed developer trace directly. The closest related systems either synthesise a path to a desired end state, without judging the path the developer actually took, or recommend refactorings towards an architectural target, without reference to an observed process. This project addresses that gap by scoring recorded refactoring traces with an explicit process-quality metric, generating higher-scoring counterfactual reference trajectories, and identifying measurable points where the observed trace diverges from those references.

1.3 Research Objectives

This thesis asks whether, given the start and end state of a refactoring session, we can judge the quality of the path the developer took and point to places where a better path was available. The core idea is to model refactoring as a *process* over states, rather than judging it only by comparing endpoints. To do this, we adopt a state/action/trace framing: snapshots of the codebase are treated as *states*, and developer edits-including IDE refactorings and manual transformations-are treated as *actions* that produce a trace from the start state to the end state.

A core component will be a **process-quality metric** that scores traces according to criteria that will be defined and justified using existing research on code quality metrics, smell detection validity, and process-level signals. The metric is not assumed to be purely structural: it may incorporate multiple terms capturing endpoint improvement, intermediate-state degradation, churn/disruption, and "safety" proxies such as preserving build and test behaviour, reflecting the known limitations of relying on any one signal alone.

To go beyond scoring a single observed refactoring trace we also construct *counterfactual* alternatives: **reference trajectories** between the same start and end states that score better under the chosen process-quality metric. These references give the observed trace something concrete to be compared against. The analysis can then identify *divergence points*, where the developer's path moves away from a higher-scoring alternative, and estimate how much each divergence contributes to the final quality gap. This is related to systems that generate or recommend refactoring sequences: ReSynth synthesises a path to a target end state from a developer's partial edits [12], Refactoring Navigator recommends steps towards an architectural target [13], and search-based refactoring treats sequences as an optimisation problem under explicit quality functions [14, 15]. The difference is that these systems are mainly concerned with producing a useful sequence. Here, the generated trajectory is used as a reference for evaluating the process the developer actually followed.

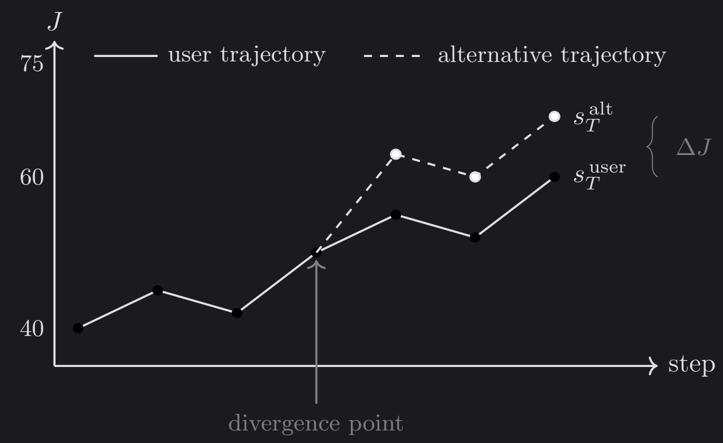

Figure 1.1: Illustration of two refactoring trajectories scored by the process-quality metric J. The solid line shows the developer's observed trajectory, while the dashed line shows a higher-scoring reference trajectory that diverges at an intermediate state. The gap ΔJ is the estimated contribution of that divergence to the final score difference.

1.4 Contributions

The main contributions of the project are:

- A process-quality metric for refactoring trajectories. The metric combines terms from prior work on code quality and developer process signals – including both endpoint cleanliness gain and an intermediate-cleanliness lag penalty so that two trajectories which finish at the same code can still score differently if one front-loaded the cleanup – and we test how much the results change under sensitivity and ablation experiments.

- A divergence-point detector that compares an observed trace with a higher-scoring reference trajectory. The detector groups divergences into four kinds: *Ordering*, *Manual-Refactor*, *Rework*, and *Hygiene*.

- A 45-session corpus of recorded refactoring sessions on a purpose-built Java codebase. We provide multi-label ground-truth annotations and the experimental scripts used for the sensitivity, ablation, and divergence analyses.

- An empirical evaluation of the detector, reporting precision and recall for each divergence kind. The evaluation also shows which parts of the score formula are useful in practice, and which kinds of process improvement it does not capture well.

1.5 Approach and Scope

The approach is intended for realistic refactoring sessions, where IDE-supported refactorings and manual edits are often mixed together. In this project we focus on Java, since it gives us mature parsing tools, established quality metrics, and refactoring mining support such as RefactoringMiner. We represent a refactoring session as a sequence of code snapshots. Each step may correspond to a burst of manual edits or the use of an automated IDE refactoring tool, and the same model could later be applied to traces at different levels of granularity where that data is available.

The work does not attempt to enumerate every possible refactoring sequence or prove that a generated trajectory is globally optimal. Instead, it uses the analysis component to find a plausible higher-scoring reference path between the same start and end states.

This keeps the task focused on explaining the developer's observed process, rather than replacing the developer with an automated refactoring planner.

1.6 Thesis Roadmap

The remainder of the thesis is organised as follows. Chapter 2 reviews related work on refactoring evaluation, refactoring mining, and refactoring recommendation. Chapter 3 defines the process-quality metric and divergence-point detector. Chapter 4 describes the implementation of the tool. Chapter 5 presents the evaluation and discusses threats to validity. Chapter 6 concludes the thesis and outlines possible future work.

Chapter 2

Background

This chapter reviews the background needed to treat *refactoring as a process*, rather than only as a change between two endpoints. It begins with definitions of refactoring and refactoring sequences (§2.1), then discusses outcome-based evaluation and the code-quality signals commonly used to judge whether code has improved (§2.2–§2.3). It then reviews refactoring mining, recommendation and sequence synthesis, and IDE-based process tracing (§2.4–§2.6). The chapter ends by positioning this project against that work (§2.7).

2.1 Definitions and framing

2.1.1 Refactoring as behaviour-preserving transformation

Refactoring is usually defined as changing a program's internal structure to improve qualities such as readability or maintainability, while preserving externally observable behaviour. Classic definitions stress that refactoring is built from small, semantics-preserving steps, with safety supported by preconditions and validation such as testing [1, 2, 3]. Opdyke's work frames refactorings as program transformations with applicability conditions intended to preserve behaviour under a static approximation [1]. Fowler's catalogue helped popularise refactoring as an incremental developer activity made up of many small operations [2, 3]. Mens and Tourwé later survey the wider research landscape, bringing together definitions, taxonomies, and tool support, and emphasising the role of precondition checking and program analysis in automated refactoring [4].

In practice, this clean definition is harder to apply. Developers often refactor while also adding features or fixing bugs, and some refactorings are only partly completed. As a result, it is often ambiguous whether a change seen in version history is a "pure refactoring". In this thesis, behaviour preservation is therefore treated as an *intended property* of refactoring steps, while recognising that real traces may include edits with more than one purpose.

2.1.2 Atomic refactorings, smells, and sequences

Research on refactoring often describes refactorings as a set of *operators*, or atomic refactorings, such as Extract Method, Move Method, and Rename [2, 4]. Larger changes can then be understood as sequences of these smaller operations. Code smells are closely related: they are heuristic indicators of possible design problems, and therefore of possible refactoring opportunities [2, 3]. Although smells are not formal correctness criteria, they provide a useful bridge between developer judgement and measurable signals, discussed further in §2.3.

A key distinction in this thesis is between judging only the start and end states of a refactoring, and judging the path taken between them. Two developers may reach the same final code, but one path may involve more temporary degradation, churn, or disruption than the other. These differences can matter for productivity and risk, even when the endpoint looks the same.

2.1.3 Representing refactoring as states and actions

To represent this refactoring path explicitly, we use a state/action/trace model. A codebase snapshot is treated as a *state* $s \in \mathcal{S}$, such as the repository at a particular point in time. An *action* $a \in \mathcal{A}$ is a developer edit step, which may be an IDE-supported refactoring, a manual change, or a mixture of both. A refactoring process is then a *trace*

$$\tau = (s_0, a_0, s_1, a_1, \ldots, a_{T-1}, s_T),$$

where each action a_t transforms state s_t into s_{t+1}. This makes it possible to talk about "process quality" as a property of the whole trace τ, rather than only as a comparison between the first and final states (s_0, s_T).

This framing is also close to recent software-engineering work that models agent and developer behaviour as a partially-observable Markov decision process, where a trajectory is a sequence of action and observation pairs [9]. We keep the explicit state notation because this thesis scores the intermediate code states themselves, not only the actions or observations produced along the way.

2.2 Outcome-based refactoring evaluation

2.2.1 Judging refactoring by the final state

Refactoring is often evaluated by comparing quality indicators before and after the change. Under this view, a refactoring is successful if the final snapshot improves on the starting snapshot according to selected measures, such as lower coupling, higher cohesion, lower complexity, or fewer code smells. This endpoint-focused view appears both in empirical studies of refactoring impact and in recommendation systems that rank candidate refactorings by their predicted improvement.

A clear example is *search-based refactoring*, where refactoring is treated as an optimisation problem over possible refactoring operations or sequences. Harman and Tratt apply multi-objective search at the design level, treating coupling and cohesion as objectives that may need to be traded off against each other [14]. Later work extends this idea to many-objective settings and adds constraints or preferences for practical concerns such as test coverage or prioritisation [15]. This line of work is useful here because it shows that "better" code is not a single fixed property: different quality goals can pull in different directions.

The same endpoint-based idea is also used in refactoring recommendation systems. These systems compare possible refactorings and rank them by the improvement they are expected to produce. Some go beyond code-structure metrics by using information such as requirements traceability, since a design may be easier to maintain when it also supports tasks like impact analysis [16]. More recent work has also looked at how to choose among large sets of candidate refactorings, and which code characteristics are associated with positive or negative effects in practice [17].

2.2.2 What endpoint evaluation captures-and what it misses

Endpoint-based evaluation is useful when the main concern is whether the final code is better than the starting code under some metric Q. It gives recommendation systems a simple strategy: simulate each candidate refactoring and choose the one that gives the largest improvement in Q. It is also useful for empirical studies, where refactoring events can be compared with changes in measured code quality. This fits naturally with many existing metrics, since they are defined over a single codebase snapshot.

The weakness of this approach is that it says little about the refactoring process itself. Two developers may reach the same endpoint through very different paths, but endpoint metrics would treat those paths as equivalent. In practice, the intermediate states can still matter: code may become harder to understand for a period of time, the modular structure may be disrupted, or the build and tests may break before being repaired. The path can also be costly in other ways, for example by creating extra churn, making review harder, or increasing integration risk. Empirical work on modularity improvement shows this trade-off clearly: improvements in design metrics can come with substantial structural disruption [18]. This motivates evaluating not only the final code, but also the stability and cost of the path taken to reach it.

This is the point at which the process view becomes useful. Final-code metrics still matter, but they need to be considered alongside what happened on the way there: whether the work introduced temporary degradation, required rework, created unnecessary churn, or broke build/test stability.

2.3 Code smell detection and quality metrics

2.3.1 Using code smells as design signals

Code smells provide a way to connect qualitative design concerns with measurable indicators [2]. Smell detectors usually identify these cases either through hand-written rules or learned models. They often rely on structural properties such as size, coupling, and cohesion, with some tools also using lexical or semantic features. One prominent rule-based example is DECOR, which defines smells and design anti-patterns through measurable symptoms and detection rules [19]. DECOR is widely used in empirical studies because its definitions are explicit and it can be run efficiently at scale.

Smell detectors are useful, but they should not be treated as ground truth. Comparative studies show that tools can disagree in both precision and recall, and that performance varies across systems and smell types [20]. Much of this comes from the practical choices each tool makes, such as thresholds, feature sets, and implementation details. Detectors can also produce false positives, marking code as "smelly" even when there is no clear negative impact or when the design reflects an acceptable trade-off [21]. For this reason, smell counts are used in this thesis as one signal among several. They are combined with structural metrics, readability-oriented measures, and process-level safety signals, rather than being allowed to determine the process-quality score on their own.

2.3.2 Structural and readability metrics

Static metrics provide another way to measure code quality at the level of classes, methods, and packages. Common metric families cover coupling, cohesion, complexity, and size, and are often used as indicators of maintainability. These structural measures are useful, but they do not capture everything developers care about when refactoring. Readability is one example: models that combine structural features, such as line structure, with textual features, such as identifier patterns, tend to align better with human judgements of readability [22]. This matters for process evaluation because readability and understandability are common reasons for refactoring, and they can change noticeably in intermediate states even if the final code improves.

2.3.3 What this means for the process-quality metric

The process-quality metric $J(\tau)$ uses these signals, but it also has to account for their limitations. The literature suggests three design choices:

1. **Use multiple signals.** No single metric or smell detector gives reliable ground truth across all contexts [20, 21]. Combining several signals reduces dependence on the quirks of one tool.

2. **Separate endpoint improvement from process cost.** Improving the final snapshot is important, but disruption and temporary degradation can still matter even when the endpoint improves [18].

3. **Keep the components interpretable.** A process critique is more useful when it can point to a specific issue, such as a readability dip, churn spike, or smell increase, rather than only reporting a single opaque score.

A process metric should also include some notion of developer **effort**, such as the number of steps in the trajectory or the size of the edits between successive states. Fewer steps are not always better, since small steps can reduce risk, but step count and edit size are still useful for identifying unnecessary detours and rework. Similar cost terms are already used alongside structural quality goals in search-based refactoring and recommendation work, for example when trying to improve quality while minimising change [14, 15].

Table 2.1 summarises the candidate signals, their main benefits and limitations, and examples of Java tooling. The table does not define the final formula for $J(\tau)$, including weights or normalisation. Its purpose is to motivate the signals used later; the precise definition of $J(\tau)$ is given in the methodology chapter and used consistently in the evaluation.

2.4 Refactoring detection and mining from code changes

2.4.1 From diffs to refactoring actions

To evaluate a process trace in practice, developer actions either need to be captured directly, for example through IDE event logs, or inferred from the differences between snapshots such as bursts of manual user edits. Much of this work uses the second route: given two versions of a program, infer whether a refactoring happened and identify which refactoring types occurred.

Structured code differencing is important here because many refactorings are not easy to recognise from line-based diffs alone. AST-based differencing can represent moves, renames, and other non-local edits more directly. GumTree, for example, maps fine-grained AST nodes between program versions and produces edit scripts that better reflect the structural changes developers intended [8]. This kind of differencing is often used as the base layer for refactoring detection.

Modern refactoring miners use these structural mappings together with heuristics for particular refactoring types. RefactoringMiner detects a wide range of refactorings between two versions of the code by matching related classes, methods, and fields [7]. RefDiff takes a similar approach, using similarity heuristics and static analysis to detect common refactoring types in version histories [23]. For process analysis, tools like these are useful because they can turn a transition $s_t \to s_{t+1}$ into a set of detected refactoring actions, giving the trace a concrete *action vocabulary*.

2.4.2 What refactoring detectors miss

Refactoring detection tools usually report what kind of refactoring occurred, such as Move Method or Extract Class, and which classes, methods, or fields were involved. This

Signal	Pros	Cons / threats	Typical tooling
Structural metrics (coupling/cohesion/complexity/size)	Widely used; efficient; interpretable at class/method granularity	Validity depends on context; may incentivise disruptive restructuring; metric interactions	Static analysis metric extractors; IDE/CI integrations
Smell counts / severity (rule-based)	Directly targets design issues; aligns with developer's motivations	Detector disagreement and false positives; threshold sensitivity; context dependence	DECOR-style detectors [19]; other smell tools [20]
Readability models	Closer to human comprehension than structural-only metrics; captures lexical/style clues	Model generalisation and dataset bias; may be language/style dependent	Readability predictors [22]
Churn / disruption (LOC changed, file moves/renames)	Captures cost and instability of change; naturally trajectory-oriented	Churn may be necessary for large refactors; can penalise required restructuring	VCS mining; diff statistics
Process length / effort (number of steps; edit magnitude)	Trajectory-native cost signal; discourages detours/rework; easy to compute	Sensitive to step granularity (commits vs. checkpoints vs. IDE events); may penalise safe micro-steps; requires calibration	VCS checkpoints; IDE logs; diff/AST edit size
Build/test preservation signals	Direct safety signal; aligns with "behaviour-preserving intent"	Requires runnable builds/tests; noisy due to flaky tests; environment dependence	CI logs; local build/test runs
Traceability-augmented signals	Reflects maintenance tasks beyond code structure; complements metric-only ranking	Requires trace links (often unavailable); domain-specific assumptions	Traceability-based ranking methods [16]

Table 2.1: Candidate signals for constructing a process-quality objective. We combine endpoint quality change with trajectory-oriented cost and stability terms, rather than relying on any single indicator.

fits the operator-based view of refactoring used in refactoring guides and surveys [2, 4]. However, the literature also reports several limitations that matter for process evaluation.

Mixed and tangled changes. Version control commits often mix refactoring with feature work or bug fixes, and they may bundle several unrelated activities together. A single commit-level transition can therefore include both behaviour-preserving and behaviour-changing edits. In these cases, refactoring detection may return only partial or ambiguous results. This makes snapshot-derived actions harder to interpret and motivates careful trace construction, such as using finer checkpoints and methods that can tolerate imperfect refactoring detections.

Limited coverage and combined refactorings. Detectors only cover a finite set of operators, and refactorings can be composed, such as a move followed by a rename, in ways that make matching harder. Some refactorings are also performed manually without tool support and may not fit the patterns that detectors expect. For process evaluation, this means that "actions" should be treated as partially observed: detected refactorings are useful features, but they are not a complete description of developer intent.

Behaviour preservation is not verified. Detection is usually structural. It infers that a change *looks like* a refactoring, but it does not prove that behaviour was preserved. Because of this, downstream evaluation should not treat a detected refactoring as automatically safe. Where possible, it should also use independent safety signals, such as whether the build and tests still pass.

2.4.3 Tie-in to our work

Refactoring mining is important infrastructure for building traces and describing actions at scale [7, 23]. This thesis does not try to improve refactoring detection accuracy. Instead, mined refactorings are used as part of the observational layer: they provide labels and features for transitions between states, which can then support process scoring and reference-trajectory generation. The next section builds on this by discussing work that generates and navigates refactoring sequences.

2.5 Refactoring recommendation and search/synthesis

The previous sections point to two needs for process-level evaluation: a way to represent and score a developer trace, and a way to construct alternative trajectories for comparison. There is already a large body of work on generating refactoring suggestions and sequences, but most of it is not designed to evaluate a developer's process. This section reviews three related families of work: synthesis approaches that reconstruct or complete refactoring sequences, search-based approaches that optimise sequences under quality objectives, and interactive approaches that adapt recommendations based on developer feedback.

2.5.1 Synthesis and path completion from partial edits

One way to generate alternatives is to treat refactoring as a program transformation problem. Given a start program and a target, or a partially edited program, the task is to synthesise a sequence of refactoring operators that carries out the transformation. ReSynth, for example, synthesises a sequence of IDE-supported refactorings that is consistent with a developer's partial edits [12]. It uses the edited classes, methods, or fields, together with the operators' preconditions, to constrain the search. Its goal is reachability and consistency: can some operator sequence reproduce the developer's edits? This is different from evaluating process quality, but it shows that refactoring can be treated as a structured search problem and that the search can be guided by evidence from the developer trace.

A related line of work learns transformations from examples. REFAZER learns program transformations from input-output examples of code edits [24]. Rather than assuming a fixed list of defined refactoring types, it synthesises AST-level rewrite rules from a small number of examples and can apply them to automate repetitive edits. Although REFAZER is not a refactoring miner in the narrow Fowler sense, it is relevant here because it shows that plausible transformations can be inferred from the edits themselves, including edits that do not fit neatly into IDE-supported refactoring operations.

Relevance and limitations for process evaluation. Synthesis systems are mainly concerned with whether a target program can be reached, or whether a sequence can be found that is consistent with partial edits, while still satisfying refactoring preconditions [12]. They do not judge the intermediate choices the developer actually made: for example, whether those choices introduced avoidable churn, temporary degradation, or extra risk. They are still useful for this thesis because they show how candidate alternative trajectories can be generated in a structured way, rather than invented by hand.

2.5.2 Search-based optimisation of refactoring sequences

A second family treats refactoring recommendation as an optimisation problem. Search-based methods explore possible changes or sequences of changes, then choose those that perform best against one or more quality objectives [14]. Harman and Tratt, for example, optimise for multiple objectives at once, treating coupling and cohesion as competing concerns and producing Pareto fronts rather than reducing design quality to a single score [14]. This is useful because it makes trade-offs between different notions of "better code" explicit.

Later work brings in concerns that are closer to everyday development. Many-objective approaches, for example, consider not only quality improvement, but also prioritisation and how refactoring effort is spread across the codebase [15]. This matters for process evaluation because it recognises that refactoring has costs while it is being carried out, not only benefits in the final version of the code.

Search-based techniques also show that improving metrics can come with disruption. Empirical studies of modularity optimisation show that maximising cohesion and coupling improvements can substantially disturb the original modular structure [18]. More recent work makes this trade-off explicit by treating *architectural distance*, a measure of disruption, as an objective alongside structural quality [25]. Other work adds socio-technical costs, such as reviewer availability, to the recommendation process [26]. This matters here because a final design can look better under structural metrics while still being costly or disruptive to reach.

Relevance and limitations for process evaluation. Search-based refactoring is useful here because it provides two ideas this thesis builds on: explicit quality objectives and generated candidate sequences. The difference is in how those sequences are used. In most recommendation work, the generated sequence is the proposed solution, and success is measured by the improvement it produces or by whether developers prefer it. In this thesis, the developer's observed trajectory is what we evaluate. Generated alternatives are used only as references for comparison.

2.5.3 Goal-directed navigation and architectural refactoring

A third family focuses on guiding refactoring toward a target design. Refactoring Navigator, for example, takes a current system and a desired architecture or design, then recommends a sequence of refactoring steps using a search-based strategy [13]. The important point is that it guides developers step by step, rather than only ranking isolated refactorings. Controlled evaluations report large reductions in task time and manual edits compared with unguided refactoring [13], suggesting that sequence-level guidance can change how developers refactor in practice.

Goal-directed refactoring is relevant because it shows how an overall design goal can be translated into a sequence of concrete refactoring actions. This supports the comparison used in this thesis: an observed developer trajectory can be compared with a higher-scoring reference trajectory that works towards the same end state.

Relevance and limitations for process evaluation. Navigation systems usually focus on reaching a target design, using design metrics to guide the suggested steps [13]. Their purpose is not to judge the quality of a developer's actual path, or to explain where that path moved away from a better alternative. Even so, they are important for this thesis because they show that refactoring can be treated as a sequence of decisions, and that sequence-level guidance can be computed and shown to developers in a usable form.

2.5.4 Interactive and developer-in-the-loop recommendation

Fully automatic optimisation can miss developer intent, constraints, or local context. Interactive recommenders try to reduce this problem by taking developer feedback into account during the search. Alizadeh and Kessentini, for example, propose a dynamic search-based approach that presents recommendations over several rounds and updates the search based on the developer's choices [27]. This shows that refactoring recommendation is not just a matter of finding a high-scoring sequence under fixed metrics. The suggested steps also need to stay aligned with what the developer is trying to achieve.

Relevance and limitations for process evaluation. Interactive recommenders are not trying to evaluate a completed refactoring process. However, they are still relevant because they show how refactoring feedback can be presented inside a developer's workflow. If process evaluation is delivered through an IDE, the feedback needs to be understandable, not too disruptive, and sensitive to the context of the task. Work on interactive recommendation therefore provides useful ideas for presenting sequence-level feedback and for letting developers respond to it [27]. The difference in this thesis is that the goal is comparison and evaluation: the observed trajectory is compared with counterfactual alternatives, rather than simply recommending the next step to optimise.

2.5.5 Beyond structural metrics: alternative objectives for ranking refactorings

A recurring theme in recommendation research is that structural metrics do not capture everything that makes a refactoring useful. For example, some work ranks refactoring candidates using requirements traceability as well as code metrics. In these approaches, quality depends partly on how clearly requirements map to code, especially for maintenance tasks such as impact analysis and program comprehension [16]. This supports the view taken in this thesis: a process-quality objective should not be assumed to be purely structural, because the costs and benefits of refactoring can also depend on non-structural information and task context.

Recent work also studies why developers refactor and how recommendations can reflect those motivations. Some studies mine refactoring operations and repository history to find product and process signals linked to refactoring activity and developer intent. Others propose using LLMs to infer motivations from commit artefacts [28, 29]. These works mostly ask *when* and *why* refactoring happens, rather than *how well* the refactoring process is carried out. Even so, they support the idea that developer-aligned objectives may need process signals and contextual information, not just static code metrics.

2.5.6 Synthesis: what sequence-generation work enables, and what remains missing

Taken together, sequence generation and recommendation systems show that refactoring can be treated as more than a set of isolated edits. Existing systems can generate sequences that realise a transformation or move towards a target design [12, 13]. Search-based approaches can optimise those sequences under explicit metric objectives [14, 15]. Interactive recommenders show that developer feedback can be incorporated during the process [27], and traceability-based approaches show that "quality" can include information beyond structural code metrics where suitable artefacts exist [16].

What these systems do not directly answer is whether the developer's observed trajectory was a good way to carry out the refactoring, according to a defined process score. They also do not explain where that trajectory diverged from a better alternative. This is the key distinction for this thesis. Recommendation and synthesis work usually treats the generated sequence as the main object of interest, and evaluates whether that sequence is useful or reaches a good outcome. Process evaluation treats the *developer trace* as the main object, and compares it with counterfactual trajectories that score better according to the chosen process score.

2.5.7 Recent trajectory-formalism work in software engineering

Recent work on software-engineering agents has also started to incorporate the concept of a trajectory. RefactorBench, for example, models agent behaviour as a sequence of actions and observations in a POMDP-shaped environment [9]; related work studies recurring agent action patterns [10] and training environments [30]. Alomar et al. also note, in their review of LLM-based refactoring research, that evaluation remains an open issue in this area [11]. This shows that trajectory-based analysis is becoming more common, but most of this work is still aimed at LM-agent behaviour and benchmark task success rather than the quality of a human developer's refactoring process. Observational studies of real-world refactoring make the human side of this point more directly: refactoring usually unfolds as an ordered sequence of operations, often interleaved with other change activities [31]. This supports the trajectory-level framing used in this thesis.

2.6 Process-aware work and IDE trace capture

Evaluating a refactoring process requires access to traces: sequences of intermediate states and actions. There are two broad sources of traces: (i) version control histories (commits/checkpoints), and (ii) IDE-level interaction logs that capture finer-grained behaviour. This section focuses on IDE logging and process-aware studies, since they show that refactoring traces can be captured at a useful granularity, while also making clear some of the practical limits of working with this kind of data.

2.6.1 Logging developer interactions in IDEs

Several studies have used logs of developer interactions to recover workflows and common behaviour patterns. Damevski et al., for example, mine sequences of Visual Studio interactions from telemetry-like event streams, aiming to find frequent usage patterns without recording the full source code [32]. For trace collection, this shows that commands, navigation actions, and edits can be recorded in a structured way while still protecting developer privacy. The trade-off is that these logs do not preserve enough project state to replay the session later, which limits the kinds of process-quality analysis that can be done.

Poncin et al. take a broader repository-level view, combining logs from version control, bug trackers, and mail archives to infer developer workflows with process mining techniques [33]. This shows how low-level records can be turned into higher-level activity models, such as sequences of commits, issue updates, and discussions. It also highlights a recurring problem with this kind of data: the event streams are noisy, developers may be working across several tasks at once, and raw events have to be segmented before they can be interpreted as meaningful steps. Since these logs are collected at repository level, they also miss IDE-level details such as which command was run or whether a refactoring was invoked through the IDE.

More recent logging infrastructure has mostly focused on developer–LLM interactions rather than refactoring-specific traces. *CodeWatcher*, for example, is a lightweight VS Code extension that records fine-grained IDE events such as LLM-tool insertions, deletions, copy-paste actions, and focus changes, with timestamps that support later reconstruction of a coding session [34]. The same kind of event capture could be useful for refactoring traces, but the available work does not map directly onto this thesis: it is centred on VS Code and LLM-assisted coding, not Java refactoring in IntelliJ. As a result, the most relevant IDE-side work for refactoring trajectories remains older, while recent process-mining work has largely moved towards repository, CI/CD, or LLM-coding logs.

Taken together, the available datasets either capture rich IDE events for the wrong activity, capture refactoring activity at repository rather than IDE granularity, or do not retain enough code content to replay the trajectory. For this thesis, then, the dataset has to be built for the task rather than reused directly from prior work. Chapter 3 returns to this in §3.6.

2.6.2 Granularity, segmentation, and semantic interpretation

IDE logging also raises a basic modelling question: what counts as one "step" in a refactoring process? Commits are usually too coarse, since they often mix refactoring with feature work, fixes, or cleanup. Raw IDE events go too far in the other direction: navigation, edits, refactoring-tool calls, and test runs all appear as separate events, even when they belong to one conceptual refactoring. The process mining and interaction analysis literature shows that splitting this kind of activity into meaningful tasks or episodes is difficult [33, 32]. For refactoring, this is especially important because one refactoring may involve many small interactions, while a short burst of editing may contain several distinct actions.

A process-evaluation system therefore needs a step definition that can be extracted reliably, but is still meaningful enough to interpret.

2.7 Summary and positioning: toward process-quality evaluation

The literature discussed in §2.1–§2.6 gives a strong starting point for this thesis, but it also shows where existing work stops short of process-level evaluation.

Refactoring itself is well established as behaviour-preserving change, supported by catalogues of operators and by tool-based precondition checking [1, 2, 4]. Most evaluation work, however, still judges refactoring by comparing start and end snapshots, using structural metrics, code-smell counts, or related quality indicators [14, 19]. These signals are useful, but they are imperfect and can disagree across tools, so combining several signals is often more reliable than relying on one measure alone [20, 21, 22]. Refactoring mining and AST differencing make it possible to infer actions from code changes and reconstruct traces from repository histories [7, 8]. Recommendation and synthesis systems show that refactoring sequences can be generated and planned towards final goal states [12, 13, 27], and that quality may include contextual objectives beyond structural metrics [16, 28, 29]. IDE logging work adds that fine-grained traces can be captured, but also shows how difficult they can be to segment and interpret [33, 32, 35].

There has therefore been substantial work on *detecting* refactorings, *recommending* refactorings, and *synthesising* refactoring sequences. What is still missing is a system that scores the path a developer actually took against an explicit process-quality metric, then compares that path with higher-scoring alternatives. The closest existing work usually treats the generated sequence as the answer: if it reaches a good target state, the system has done its job. In this thesis, the observed developer trace is the object being evaluated. The aim is to judge whether the process itself was good, including whether it introduced unnecessary intermediate degradation, disruption, or instability, and to explain where it diverged from better-scoring reference trajectories.

This thesis takes that missing comparison as its starting point. For a recorded refactoring session, it scores the developer's trace and builds a higher-scoring reference path between the same start and end states, so the two can be compared directly. This leads to the overall thesis question: *given the start and end state of a refactoring session, can we judge the quality of the path the developer took and point to places where a better path was available?*

Chapter 3

Methodology

This chapter explains how the proposed process-evaluation approach is defined and implemented. It begins by setting out the state/action/trace notation used throughout the chapter (§3.1). It then defines the process-quality score (§3.2.1–§3.2.3) and describes the event-capture, shadow-repository, and metrics infrastructure that supplies the data for that score (§3.2.4–§3.2.6). Next, it introduces the divergence-point detector (§3.3), the Eclipse JDT substrate used to construct alternatives (§3.3.1), the per-kind alternative trajectory synthesisers (§3.3.2–§3.3.5), and the validator used to check that each alternative still reaches the same end state (§3.3.6). The chapter then describes the overall system architecture and offline/replay pipeline (§3.4–§3.4.1; §3.4.2), followed by the participant dashboard (§3.5) and the session corpus and labelling procedure used for the evaluation in Chapter 4 (§3.6; §4.2.2).

3.1 Formal definitions and notation

The formal definitions in this section build on the state/action/trace framing from §2.1. A refactoring trajectory is a sequence

$$\tau = (s_0, a_0, s_1, a_1, \ldots, a_{T-1}, s_T),$$

where each s_t is a code snapshot and each a_t is a developer action that turns s_t into s_{t+1}. Actions are either IDE-supported refactoring operations or manual edits; in practice a single action can carry both a structured refactoring spec (recovered by RefactoringMiner [7]) and some leftover unstructured edits. We treat this as one action with two parts rather than two separate actions.

The tool evaluates a trajectory by producing an *analysis report*. For a trajectory τ, this report is written as the tuple $(\tau, \mathcal{M}, \mathcal{D}, \mathcal{A})$, where:

- $\mathcal{M} = \{m_0, m_1, \ldots, m_T\}$ is the set of metrics collected for the states in the trajectory. Each m_t contains the cleanliness sub-score $C(s_t)$ and the raw signals used to compute it, including PMD violations, CPD duplications, Chidamber–Kemerer measures, build/test outcomes, the action's refactoring spec, and commit and test event metadata.
- $\mathcal{D}$ is the set of divergence points the detector emits for τ (§3.3).
- $\mathcal{A}$ is the set of alternative reference trajectories synthesised at those divergence points (§3.3.2 onwards). Each $\tau^* \in \mathcal{A}$ is represented in the same way as the observed trajectory, which allows the same scoring machinery to be applied to both.

The process-quality score $J(\tau)$ is computed over the full trajectory using $\mathcal{M}$, and is the focus of §3.2.1–§3.2.2. Table 3.1 summarises the notation used throughout the chapter.

Symbol	Meaning
$\tau = (s_0, a_0, s_1, \ldots, a_{T-1}, s_T)$	Refactoring trajectory
s_t	State (code snapshot) at step t
a_t	Action at step t (IDE refactor or manual edit)
$J(\tau)$	Process-quality score, clamped to $[0, 100]$
$C(s)$	Cleanliness sub-score of state s
$\Delta C(\tau) = C(s_T) - C(s_0)$	Cleanliness gain across trajectory
$\beta = 50$	Baseline anchor in the process score
W_{X}	Process-score weight for term X
$r_b,\ r_{st},\ r_{mi}$	Laplace-smoothed rate terms (broken, skip-tests, manual-when-IDE)
$n_{cg}(\tau)$	Commit-gap event count
$\sigma_l(\tau)$	Step-savings bonus for alternative trajectories
DP	Divergence point
τ^*	Alternative reference trajectory
$\mathcal{K}$	Divergence-kind set = $\{$*Ordering*, *Manual-Refactor*, *Rework*, *Hygiene*$\}$

Table 3.1: Notation used throughout the methodology chapter.

3.2 Computing the process score

This section defines the process score $J(\tau)$, the literature-grounded weights it carries, and the cleanliness sub-score it consumes (§3.2.1–§3.2.3); it then describes the event capture, shadow-repo reconstruction, and metrics calculation that produce the per-checkpoint state the score is computed against (§3.2.4–§3.2.6).

3.2.1 The process score

The process score $J(\tau)$ combines seven weighted terms across the trajectory and clamps the result to a $[0, 100]$ range so trajectories of different lengths can be compared on the same scale:

$$J(\tau) = \mathrm{clip}_{[0,100]}\Big(\beta + W_{\mathrm{G}}\,\Delta C(\tau) - W_{\mathrm{B}}\,r_b(\tau) - W_{\mathrm{ST}}\,r_{st}(\tau) - W_{\mathrm{MI}}\,r_{mi}(\tau) - W_{\mathrm{CG}}\,n_{cg}(\tau) - W_{\mathrm{LAG}}\,\ell(\tau) + W_{\mathrm{L}}\,\sigma_l(\tau)\Big) \tag{3.1}$$

where:

- $\beta = 50$ is the baseline anchor. A trajectory with neither cleanliness gain nor process damage scores 50.
- $\Delta C(\tau) = C(s_T) - C(s_0)$ is the cleanliness gain from start to end (§3.2.3).
- $r_b(\tau) = b_{\mathrm{ms}}(\tau)/e_{\mathrm{ms}}(\tau)$ is the time-weighted broken fraction: the proportion of trajectory wall-clock time during which the build or test suite was broken.
- $r_{st}(\tau)$ and $r_{mi}(\tau)$ are Laplace-smoothed event-rate terms of the form $r_\bullet(\tau) = (k + 1)/(n + 2)$, where k is the number of penalised events and n the number of opportunities. Both default to zero when there have been no events of either polarity yet, so a single bad-out-of-one event does not trigger a full penalty.
- $n_{cg}(\tau)$ is the commit-gap event count: one event per stretch of ≥ 6 green-refactor checkpoints between user commits.
- $\ell(\tau) \in [0, 1]$ is the intermediate-cleanliness lag: the per-checkpoint running mean of $\max\big(0,\ C(s_T) - C(s_i)\big)$ over the trajectory, in scalar cleanliness units. It is zero at the terminal checkpoint by construction and positive while the trajectory sits below its final cleanliness target, so it penalises trajectories that linger in a degraded state and rewards getting clean earlier. Two trajectories with the same start and

end cleanliness can still differ here, which is the source of the term's discriminating power.

- $\sigma_l(\tau)$ is the alt-only step-savings bonus. It only contributes when τ is a synthesised alternative trajectory that reaches the same end state in fewer steps than the user.

Three design choices shape this formula. First, an *anti-gaming axiom*: the broken-state penalty W_{B} has to outweigh any single-step cleanliness gain $W_{\text{G}} \cdot \Delta C$ a developer could plausibly engineer by trading transient correctness for structural improvement. The severity ordering $W_{\text{B}} : W_{\text{ST}} : W_{\text{CG}} = 28 : 14 : 7$ enforces this in the weights of §3.2.2. Second, the Laplace smoothing keeps the rate terms from misbehaving on very short trajectories where one bad event would otherwise dominate. Third, the $[0, 100]$ clamp makes scores comparable across trajectories of different lengths, but it does introduce saturation: when a user's trajectory is already near 100, any alt's improvement gets compressed against the ceiling. The sensitivity experiment in §4.1.1 tracks this and reports it as a known limitation of the score formula.

A note on what is claimed and what is not. The process score is presented here as internally consistent and theoretically grounded against the literature reviewed in Chapter 2. We do not claim that the score is uniquely-correct or calibrated against an external outcome variable; no held-out ground truth for "true refactoring-trajectory quality" exists in this problem space. The empirical evidence presented in Chapter 4 (sensitivity sweep, ablation, inter-rater agreement on the kind classifier) speaks to robustness, term-importance, and the kind classifier's accuracy - not to whether the production weight vector is the correct one to use.

3.2.2 Process-score weights

Table 3.2 lists the seven production weights of Equation (3.1) and the literature that backs each one. Their empirical contribution to the formula's ranking output is left to the ablation analysis in §4.1.2, which runs the full power-set of weight subsets ($2^7 = 128$ variants per fixture) and reports both solo and leave-one-out recovery against the production ranking; the literature column below establishes that each weight is grounded in prior work, and the ablation chapter establishes that the weights are doing the work they claim to.

Term	**Weight**	**Cited justification**
W_{G} (gain)	50	Cedrim et al. 2017 [36]; Paixão et al. 2017 [18]
W_{B} (broken)	28	Paixão et al. 2017 [18]; Gautam et al. 2025 [9] (FM #2)
W_{ST} (skip-tests)	14	Murphy-Hill et al. 2009 [5]
W_{MI} (manual)	11	Negara et al. 2013 [37]; Vakilian et al. 2012 [38]
W_{L} (length)	11	Harman & Tratt 2007 [14]; Mohan & Greer 2019 [15]
W_{LAG} (lag)	11	Cunningham 1992 [39]; Kruchten et al. 2012 [40]; Cedrim et al. 2017 [36]; Paixão et al. 2017 [18]
W_{CG} (commit-gap)	7	Tao & Kim 2015 [41]; Di Biase et al. 2019 [42]; Herzig & Zeller 2013 [43]; Kudrjavets et al. 2022 [44]

Table 3.2: Production weights for Equation (3.1) and the literature each weight rests on. Empirical contribution of each term to the formula's ranking output is reported in the ablation analysis of §4.1.2.

The rest of this section walks through each weight. Two of them (W_{MI} and W_{CG}) have a citation gap: the literature supports the idea of the penalty, but does not directly measure exactly what the weight captures. We note this openly below.

W_{G} (gain, 50). The main positive term: a unit of cleanliness gain at the end of the trajectory adds 50 to $J(\tau)$. Cedrim et al. [36] found that refactoring often makes structural metrics no better or even slightly worse, which means the score should reward improvement explicitly rather than assume it. Paixão et al. [18] model the trade-off between disruption and improvement as something developers actively manage, which sets the size of this positive term in relation to the penalty terms below.

W_{B} (broken, 28). The largest single penalty term. Paixão et al. [18] show that developers care a lot about avoiding disruptive intermediate states - they are willing to give up roughly a quarter of the available metric improvement to avoid them. The weight is time-weighted rather than binary: $r_b(\tau) = b_{\mathrm{ms}}/e_{\mathrm{ms}}$, so a brief broken state is penalised proportionally less than a long one. The numerator b_{ms} counts only wall-clock time during which the build or tests are actually failing - not total refactoring time, and not time the developer spends thinking between changes. A developer who pauses to think carefully before making a clean change is not penalised; a developer who leaves the build broken for five minutes is. Time-weighting also makes the broken term robust to the choice of granularity. The state/action/trace formalism in §3.1 leaves what counts as a "state" flexible - a state can be a commit, an IDE event, or a developer-defined checkpoint - and a binary "was this state broken" counter would change by a large factor across those choices (count once per commit vs. once per keystroke and the broken-state count looks very different). Wall-clock time spent broken does not depend on the sampling rate, so $r_b(\tau)$ stays meaningful at any granularity. The time-weighting choice also lines up with Gautam et al.'s 2025 RefactorBench analysis [9]: their failure-mode #2 shows that strictly rejecting any intermediate broken state hurts LM-agent performance, because some broken intermediates are genuinely necessary for multi-file edits. The anti-gaming rule means W_{B} has to outweigh the single-step gain $W_{\mathrm{G}}\,\Delta C$ for any plausible ΔC.

W_{ST} (skip-tests, 14). Penalises combined refactoring windows (60-second batches) that finish without any test run in between. The 60-second cut-off comes from Murphy-Hill et al. [5]: their study of how developers actually refactor shows that tests are run per batch of changes, not after every single step. We set the weight at half W_{B}: skipping tests is weaker evidence of process damage than an observably broken state, but it is still a signal the score should track.

W_{MI} (manual-when-IDE, 11). Penalises steps where the developer refactored manually when the IDE could have applied the same transformation automatically. Mens & Tourwé [4] note that IDE refactoring tools run static precondition checks that hand edits may bypass, making the automated path more reliable. Negara et al. [37] find that manual refactorings remain common nevertheless: across 23 developers and 5,371 refactorings, developers frequently performed them by hand even when an automated equivalent was available. Vakilian et al. [38] show that automated refactorings can also be misused, which motivates a graded penalty rather than a binary one. No published study (that we are aware of) directly measures the fault-rate gap between manual and IDE refactoring, so the weight is not fitted to data; it sits within the severity ordering set out above. The ablation in §4.1.2 shows that `manualIde` sits in the top-three single-knob contributors to session-wide divergence-point magnitude, alongside `length` and `broken` (Table 4.5), and the sensitivity sweep in §4.1.1 indicates that the ranking is robust to perturbations of this weight.

W_{L} (length, 11). A positive term rather than a penalty: it increases $J(\tau)$ for alternative trajectories that reach the same end state as the user in fewer steps. Harman & Tratt [14] and Mohan & Greer [15] treat refactoring count as an objective on equal footing with quality in multi-objective search-based refactoring; Cortellessa et al. [25] continue that line of work, trading effort against code-quality objectives explicitly. The user trajectory is not penalised for length: a longer path may reflect a larger task rather than inferior process, and treating length as a penalty would conflate task size with process quality. The bonus applies only when the synthesiser supplies a shorter alternative that still reaches the same end state.

$W_{\mathbf{LAG}}$ (lag, 11). Penalises trajectories that linger in a degraded cleanliness state on the way to their final value. The gain term $W_{\mathrm{G}}\,\Delta C$ is endpoint-only: two trajectories with the same start and end cleanliness score identically under it, even if one front-loaded the cleanup and the other deferred it. The lag term closes that blind spot. Cunningham's debt metaphor [39] and Kruchten et al.'s subsequent formalisation [40] both frame degraded code as accruing interest while it persists, which motivates a penalty proportional to the time spent below the trajectory's final cleanliness target rather than only to the gap between endpoints. Cedrim et al. [36] provide direct empirical support: their analysis of refactoring sequences shows that intermediate states often degrade structural metrics relative to the start, which the lag term explicitly costs. The weight sits at the same magnitude as W_{L} and W_{MI} — half the broken-state penalty W_{B}, mirroring the severity ordering of §3.2.2 — and the bound $\ell \in [0, 1]$ means the maximum lag penalty is 11 points, smaller than the maximum gain credit of 50. The empirical contribution of this weight is the ablation analysis's strongest signal for the *Ordering* divergence-kind specifically: §4.1.2 reports its solo magnitude-recovery and §4.2.2 reports that the count of *Ordering* divergence-points whose synthesised alternative strictly beats the user's trajectory rises from 4 to 10 of 24 once W_{LAG} is in the formula.

$W_{\mathbf{CG}}$ (commit-gap, 7). Penalises long runs of green checkpoints (refactor passes + tests pass) without a commit in between. The term is based on work on decomposing composite changes. Tao & Kim [41] report that separated commits let reviewers inspect changes more effectively in the same amount of time, and Di Biase et al.'s controlled experiment [42] supports the direction of the penalty: decomposed changes produced fewer comments that wrongly flagged sound code as defective (1 rather than 6, $p = 0.03$). Tangled commits also make later defect attribution less reliable [43]. The claim is deliberately narrow: Kudrjavets et al. [44] find no correlation between PR size and time-to-merge across 1.25M PRs in ten languages, so smaller commit gaps are treated as improving review comprehensibility and rollback locality, not merge speed. Since no published study (that we are aware of) directly links commit cadence during refactoring to rollback or review cost, the weight is not fitted to data; it sits within the severity ordering above. The ablation results support retaining it as a secondary hygiene term: `commitGap` recovers non-zero session-set magnitude on its own (sum/sum 0.126), and removing it changes the ranking (mean $\tau = 0.926$) while leaving most total magnitude intact (sum/sum 0.892).

3.2.3 The cleanliness sub-score

The cleanliness sub-score $C(s)$ in Equation (3.1) combines six per-state quality signals into a single normalised value in $[0, 100]$. The signal set follows the design principle from §2.3: no single quality measure is reliable across contexts, so combining complementary signals reduces sensitivity to the noise of any one tool. Table 3.3 lists each sub-signal, its cited basis, and how it is computed.

Composition rule. Each raw sub-signal is min–max normalised against the main trajectory's range, inverted for "lower is better" measures (cognitive complexity, coupling,

Sub-metric	Weight	Cited basis	Computation source
Cognitive complexity	1.0	Campbell 2018 [45]; Muñoz Barón et al. 2020 [46] (partial validation); Lavazza et al. 2023 [47] (contradicting)	Per-method mean
Coupling (CBO)	1.0	Chidamber & Kemerer 1994 [48]	CK library, per-class mean
Duplication	1.0	Heitlager et al. 2007 [49]; Fowler 1999 [2]	CPD-detected cloned lines on touched files
Readability	1.0	Buse & Weimer 2010 [50] (feature taxonomy only - not the trained model)	Five-feature blend, line-weighted
Smells	1.0	Marinescu 2004 [51]; Arcelli Fontana et al. 2016 [21]; Moha et al. 2010 [19]	PMD violation count on touched files
Cohesion (TCC)	1.0	Bieman & Kang 1995 [52]	Per-class mean, dropped if < 2 methods

Table 3.3: The six sub-signals composing the cleanliness sub-score $C(s)$. All weights are uniform 1.0 following Laplace's principle of insufficient reason (see composition rule below); the production weight bundle in the implementation reflects this.

duplication, smells). Sub-signals with degenerate ranges or missing data on the trajectory are dropped and the remaining weights renormalised. Alternative-trajectory checkpoint values are clamped to $[0, 1]$ against the main trajectory's range, so an extreme alt cannot shift the main trajectory's normalisation. The composite is a weighted sum, then scaled to $[0, 100]$. Wagner et al.'s Quamoco framework [53] and the QMOOD quality model [54] both use this kind of layered weighted-sum aggregation, which is a defensible choice when there is no calibration data to fit the weights against.

Uniform sub-weights, by deliberate choice. All six sub-weights are set to 1.0 because there is no published calibration data for weighting these signals across refactoring trajectories. Calibrated weights do exist for snapshot-level static quality composites - notably Coleman et al.'s Maintainability Index, whose coefficients were regression-fitted against expert maintainability ratings on industrial systems [55] - but those calibrations are fitted to absolute metric magnitudes on a single snapshot, not to normalised deltas across a sequence of checkpoints, so they do not transfer to the trajectory-relative signals used here. In the absence of trajectory-level calibration data, equal weighting follows Laplace's principle of insufficient reason. The sensitivity experiment in §4.1.1 supports this choice: perturbing the cleanliness sub-weights reshuffles only a small fraction of divergence-point rankings, an order of magnitude fewer than the process-score perturbations.

Caveats deliberately disclosed. Two of the sub-signals have weaker empirical backing than the others. Both are detailed below:

- *Readability.* We use Buse & Weimer's feature taxonomy [50] (line length, indentation, identifier characteristics) as a feature blend, but not their trained readability model. Calibrating their trained model to the trajectory-evaluation setting would need domain-specific data we have not collected. The feature blend without the trained model is a weaker signal but it is still defensible as one of six components.

- *Cognitive complexity.* Cognitive complexity is empirically contested. Muñoz Barón et al. [46] re-analysed 24,400 human understandability ratings and partially validated Campbell's claim, finding significant positive correlation with comprehension time and subjective ratings; correlation with correctness was weak. Lavazza et al. [47], on the other hand, find that cognitive complexity predicts understandability about as well as traditional size and cyclomatic-complexity measures, with negligible additional explanatory value. We include cognitive complexity as one of six equal signals, given the mixed evidence around it.

Coupling to the process score. The cleanliness sub-score is consumed twice by the process score: once as the endpoint delta $\Delta C(\tau) = C(s_T) - C(s_0)$, and once as the running reference $C(s_T)$ inside the lag term $\ell(\tau)$. Because $C(s_T)$ feeds the lag denominator, a perturbation of any cleanliness sub-weight propagates into the process score through both terms rather than only through the endpoint delta. This coupling is by design — the lag term is meaningful only in the cleanliness-scalar units used elsewhere — but it has a measurable empirical signature: on the agent corpus, the sensitivity sweep shows cleanliness-weight τ dropping by 3–10 % under $\times 0.5/\times 1.5$ perturbation relative to the pre-lag formula (§4.1.1). The injection and user corpora absorb this leakage without measurable rank disruption.

The process score and its cleanliness sub-score are computed against the per-checkpoint state of a recorded session. The next three subsections describe how that state and the metrics it supplies to the score are produced: event capture in the IDE plugin (§3.2.4), shadow-repo reconstruction that materialises the codebase at every checkpoint (§3.2.5), and per-checkpoint metrics calculation (§3.2.6).

3.2.4 Event capture

The Intellij plugin is responsible for capturing the Intellij events which are emitted during the user's refactoring process. The captured event types cover editor activity (files opened, closed, saved, modified, renamed, moved), refactoring activity (refactoring start / finish), build activity (build start / finish), test activity (test-run start / finish with pass and fail counts), git activity (one event per new commit observed in the underlying repository), and the session lifecycle (refactoring session start / end).

File modification activity is the noisiest source. IntelliJ emits an event for every keystroke, but we debounce these to avoid noise. A debouncing layer accumulates keystrokes into an *edit burst* across all files: when a 2-second quiet window elapses without further typing, a single burst event is emitted carrying the file's post-edit contents and the structured set of program elements touched.

Although a handful of the fine-grained file-level events (file open, save, and similar) are not consumed by any downstream stage, they are recorded because their capture is essentially free at the plugin layer, and they remain obvious candidates for future work on navigation patterns, file engagement patterns, and read-versus-edit behaviour.

Event normalisation. The captured event stream undergoes three normalisations before downstream processing. First, events are re-sorted by timestamp to reflect actual order. Second, spurious events are removed: FILE_CREATED events that arrive late, and EDIT_BURST events with no actual changes. The result is the *normalised event stream* consumed by every subsequent stage.

3.2.5 Shadow repo reconstruction and worktree pool

Analysing a user's trajectory requires the ability to reconstruct, and cheaply check out, the exact state of the codebase at any point during the session. This is made possible by the shadow repo and worktree pool described below.

Shadow repo reconstruction. The shadow repo is built by replaying the normalised event stream into a fresh per-session git repository, one commit per state-bearing event. The baseline commit is the initial source snapshot taken at session start; each subsequent event that carries file changes applies its snapshots to the working tree, stages them, and commits. No-op events – those that carry no file changes, or whose staged diff is blank-line-only – map to the previous SHA without producing an empty commit, so the unique-SHA count is typically much smaller than the event count. The output is a per-event map from event id to SHA, which gives every downstream stage a stable (from-SHA, to-SHA) pair to check out.

Worktree allocation. Borrowers – the validator, the reorder synthesiser, the IDE-replay runner – ask for a fresh worktree at a target SHA, run their operation, and release it back when done. Fresh-per-borrow rather than reuse-across-SHAs is deliberate: it guarantees no leftover build artefacts, stale Gradle cache, or other cross-SHA pollution between consecutive borrows.

3.2.6 Metrics calculation

For every unique state in the shadow repo the pipeline computes a checkpoint metric record: the build and test outcome, the cleanliness signals (PMD violations, CPD duplications, code readability, and Chidamber–Kemerer structural measures), and the file-level diff against the previous checkpoint.

The build and test outcomes are produced by checking the project state out into a worktree and invoking the project's own Gradle build and test tasks against it, recording for each checkpoint whether the build succeeded and whether the test suite passed. These outcomes are computed for every checkpoint, independent of whether the user actually ran the build or tests at that point in time. This is the stage that dominates the wall-clock cost of the pipeline. Two engineering decisions on the build side bring this cost down: all checkpoints share a single persistent worktree (the only stage that bypasses the fresh-per-borrow rule of §3.2.5), and the pipeline walks the unique SHAs in order using detached checkouts to switch between them. The build directory of that worktree is preserved across SHAs, so the Gradle daemon, the Gradle build cache, and incremental compilation all have prior state to reuse on the next checkpoint.

The static-analysis signals are tracked across checkpoints rather than just computed per checkpoint. Each PMD violation and CPD clone group is threaded through the unified diffs so the report can identify when a violation was first introduced, when it was resolved, and which checkpoint transition is responsible. This is what lets the dashboard show, for any refactoring step, the smells it introduced or eliminated. Both user-study participants found this inspection feature valuable during interviews (§4.3.1).

3.3 Divergence points

A *divergence point* is a step in the user's trajectory where an alternative trajectory exists that scores strictly better than the user's under the process score. The detector emits divergence points of four kinds:

$$\mathcal{K} = \{\textit{Ordering},\ \textit{Manual-Refactor},\ \textit{Rework},\ \textit{Hygiene}\}.$$

Each divergence point has four fields: a *kind*, an *anchor step* (the user step where divergence occurs), a *magnitude*, and per-kind context (the reorder window for *Ordering*; the replaced refactoring identifier for *Manual-Refactor*; the originating–terminal step pair, file, scope, and reverted-line count for *Rework*; the hygiene sub-kind and stretch length for *Hygiene*).

The magnitude is uniform across all four kinds:

$$\text{magnitude} = J(\tau^*_{\text{final}}) - J(\tau_{\text{final}}),$$

where τ^*_{final} is the alt's process score rolled forward through the user's post-convergence activity - i.e. the user's trajectory with the alt's path substituted in at the divergence and the rest of the trajectory unchanged - and τ_{final} is the user's actual trajectory-final score. The comparison is therefore at the user's trajectory-final state on both sides, making the magnitude directly comparable across all kinds: *"if you had taken this one different path at the divergence point and continued with the rest of your trajectory unchanged, by how many process-score points would your final score have moved?"*

For *Rework* specifically, the magnitude is non-trivial even though the alt reaches the same end state as the user: the alt has fewer steps because the round-trip work is omitted, so the W_{L} step-savings bonus contributes positively to the alternative trajectory's score. The reverted-line count is still attached to the divergence point as a separate field (`reworkLineCount`) for explanatory use on the dashboard, but it is not directly used in the ranking magnitude.

Before emitting a divergence point, the detector runs kind-specific filters. Three threshold constants gate detection: $min\_rework\_lines = 2$ drops single-line *Rework* noise; $min\_commit\_gap = 6$ requires at least six green-refactor checkpoints between commits before *Commit-Gap* fires; and the 60 s composite-window boundary [5] defines the refactoring batch boundary used by both *Ordering* and *Hygiene*. The constant 6 is derived from Murphy-Hill's batching heuristic rather than empirical calibration. Since the 60 s composite window already groups tight refactor sequences, setting the threshold at 6 ensures a stretch of consecutive green checkpoints represents a real no-commit gap, not just tight in-batch activity. No published study calibrates this particular threshold, so it is presented as a tunable parameter; the sensitivity analysis in §4.1.1 tests the associated weight W_{CG} rather than the threshold itself.

The rest of this section pairs each kind's detection criterion with the synthesiser that produces the alternative trajectory. §3.3.1 first describes the Eclipse JDT substrate that the *Ordering* and *Manual-Refactor* synthesisers both build on; §3.3.2–§3.3.5 cover the four kinds; §3.3.6 describes the bracket-level validator that gates the reorder synthesiser.

3.3.1 Applying refactorings via Eclipse JDT

Synthesising counterfactual trajectories requires actually applying refactorings, not just describing them. Building a refactoring engine from scratch was out of scope, so we rely on an existing one. Eclipse JDT was chosen for two reasons. First, operationally: JDT can be linked as a library and driven programmatically from a JVM, whereas IntelliJ's refactoring engine requires launching a full IDE instance via a custom plugin, adding UI dependencies, process boundaries, and startup cost. Second, capability-wise: JDT covers the refactoring catalogue this analysis needs at a maturity level comparable to its use in the Eclipse IDE.

JDT was designed to run inside Eclipse but can be embedded in other JVMs by hosting the Equinox OSGi framework. The Eclipse JDT FAQ [56] distinguishes two deployment modes: bare-classpath (for parser and AST features) and runtime-workbench (for workspace-dependent features). Refactorings require the runtime-workbench mode. The reference implementation is the Eclipse JDT Language Server [57], maintained by the Eclipse Foundation as a headless JDT distribution.

Batch sessions. JDT maintains an indexed project model in memory. Building this model takes roughly a second, so repeatedly rebuilding it between refactorings would be expensive. Instead, the pipeline wraps a whole refactoring window in a single *batch session*: the model is initialized once at the start, reused across all operations, and torn down at the end. Each refactoring then sees a fully indexed model without paying the initialization cost repeatedly.

Anchor resolution. Refactorings need to identify their target, but line numbers shift whenever surrounding code changes. The pipeline addresses this with content-addressed anchors. On the host side, RefactoringMiner's line and column coordinates are used once to locate the AST node, and that node is then summarised as the enclosing type's fully qualified name, the host method's name and parameter-type list, and a canonical hash of the node's own AST subtree. The wire-level spec sent to the JDT bundle carries that anchor; it does not carry the original line and column for range-based operations, and for point-based operations it carries them only as a tiebreaker hint in the rare case that two declarations inside the same method hash equal. The bundle finds the host method by name plus parameter types inside the named type, then searches inside it for a node whose subtree hash matches. The same hashing routine is computed on both sides, so the lookup is robust to formatting changes and to line shifts elsewhere in the file: a method that moved from line 42 to line 47 still resolves correctly, as long as its own subtree was not itself rewritten.

3.3.2 Ordering

Ordering detection works on bracket-validated windows of consecutive refactoring steps (§3.3.6). For each window, the reorder synthesiser enumerates valid permutations of the steps and computes the process-score delta of each permutation against the user's. One divergence point is emitted per window at its terminal step; the divergence point's magnitude is the best delta across the window's admitted permutations, and all admitted permutations are attached to the divergence point as alternatives. Permutations whose delta is ≤ 0 are retained on the divergence point for informational use (the dashboard surfaces them so the user can inspect the search space), but downstream analyses - the divergence-detection experiment of §4.2.2 and the "beats user" counts reported with it - apply a strict magnitude > 0 filter so only divergence points whose best alternative actually improved on the user's trajectory are counted.

The reorder synthesiser produces those alternatives by reshuffling the user's refactoring steps into a different order while still reaching the same end state. Correctness is established in six steps: window-splitting on validator outcomes, dependency-DAG construction over the program entities each step reads and writes, single-static-assignment (SSA) versioning across renames, topological enumeration, depth-first search over a prefix trie of candidate permutations, and a terminal AST-equivalence audit.

Window splitting on validator verdicts. Only bracket-validated windows (§3.3.6) are eligible for reordering. The validator returns one of three non-valid verdicts for brackets the synthesiser engine cannot reproduce: *untyped* when RefactoringMiner cannot map the user's edit to a typed JDT refactoring spec (the typical case for unstructured manual edits); *refactor_failed* when JDT attempts the spec but errors out; and *ast_diverged* when JDT applies the spec successfully but the resulting AST does not match the user's. Any of the three disqualifies the containing bracket, which is then treated as individual steps that cannot be reordered. The wrap-and-patch layer (§3.3.3) closes the most common JDT-IntelliJ divergences before this check runs, expanding the set of brackets that pass validation.

Dependency-DAG construction. Each refactoring step carries a structured *spec* describing what entities it reads, writes, creates, or deletes. The synthesiser builds a dependency graph where an edge from step u to step v means v depends on u's effects. This graph defines the constraints on any valid reordering.

SSA versioning across renames. If a method is renamed at step t from `foo` to `bar`, it shows up in later steps under its new name, which breaks simple name-based equality across the trajectory. The synthesiser threads version identifiers through the DAG using single-static-assignment-style versioning: each rename produces a fresh version, and subsequent steps that target the renamed entity bind to the right version. This handles trajectories like Extract Method followed by Rename Method on the extracted method.

Topological enumeration. The synthesiser enumerates topological orderings of the DAG via recursive backtracking. Each enumeration is a candidate permutation of the window's steps. The enumerator carries a hard size budget - windows with more than 7 refactoring steps are skipped, and within an admitted window enumeration stops after $7! = 5040$ permutations - because the cost of enumerating all topological orderings grows factorially with window size. In practice the recorded sessions of Chapter 4 rarely produced reorder windows large enough to exceed this budget, but a session with longer contiguous refactor sequences would have its widest windows silently dropped from *Ordering* enumeration (the threats-to-validity chapter at §5.6 discusses this).

Depth-first search over a prefix trie. The enumeration is implemented as a depth-first walk over a prefix trie of candidate permutations, so orderings that share a prefix reuse the project state across that prefix rather than re-applying it from scratch. Each forward edge applies one refactoring, each back edge rolls the worktree to the parent commit. Two orderings that agree on their first j steps share the first j trie nodes, so each unique prefix is materialised exactly once no matter how many leaves reuse it.

Terminal AST-equivalence audit. For each permutation that reaches a terminal state, the synthesiser admits it only if its terminal AST is equivalent to the user's modulo syntactic differences that do not change behaviour. This is the correctness guarantee: by construction, an admitted reorder reaches the same end state as the user. At each leaf of the trie – the terminal state of one full ordering – the synthesiser hashes the touched files' ASTs into a canonical form and compares those hashes against the user's terminal AST hashes, which are precomputed from the user's own trajectory via `git show` at the start of the window (no second worktree is needed). The canonicalisation collapses cosmetic differences (whitespace, comment positioning, formatting variation) and reorderings that do not change behaviour - notably method-declaration order within a class - so two ASTs that are structurally identical produce the same hash. The method-declaration-order canonicalisation matters here because permutations that apply, say, an Extract Method at a different position in the bracket place the new declaration at a different lexical offset, even though the resulting class is functionally identical. An ordering passes if and only if its terminal hashes match the user's. Otherwise, it is discarded. Admitted permutations are then scored under Equation (3.1).

Three engineering choices make this search possible at session-scale. First, the whole window runs against a single borrowed worktree, so forward edges, back edges, and the final AST audit all reuse the same on-disk checkout rather than paying the cost of a fresh worktree per ordering. Second, the whole window also runs inside a single JDT batch session (§3.3.1), so every refactoring inside it sees a warm project model and pays only the per-call cost. Third, back edges in the search roll the worktree back via a detached git checkout and then ask the JDT engine to re-stat its files via the same "files changed under us" pathway an IDE user exercises constantly.

3.3.3 Manual-Refactor

A *Manual-Refactor* divergence point is emitted at any user step where (i) the action was performed by hand, and (ii) the resulting diff matches the structural pattern of an IDE-supported refactoring. IDE-driven refactorings arrive pre-tagged in the captured event stream, but a manual refactoring is just a series of edit bursts whose net effect happens to constitute a structured refactoring. The pipeline recovers them by running Refactoring-Miner [7] – a standard tool that detects refactorings between two code states by AST-level structural matching – over a sliding window of commit pairs in the shadow repo. The window matters because a manual refactoring may unfold across several intermediate edit bursts, each of which on its own looks like an unrelated change; only by widening the window to cover the full span does the refactoring become visible to the miner. Each detected refactoring is then cross-checked against the captured event stream: a detection whose window contains a matching IDE-emitted refactoring event is recorded as IDE-driven, and one whose window contains no such event is flagged as manual. Adjacent candidate steps that share the same `(fromSha, toSha)` bracket are grouped into a single composite that the synthesiser treats as one refactoring.

The manual-refactor synthesiser applies each detected spec via its IDE-equivalent operator on the JDT engine and then runs a residual three-way merge to layer the user's non-refactoring edits (for example, unrelated bug fixes interleaved with the refactor) on top of the synthesised refactor. The resulting tree is committed to a shadow repository. The residual merge ensures the alt trajectory keeps all the user's non-refactoring edits, so both trajectories end at the same code state and the comparison measures the refactoring method (hand vs IDE), not whether the final states match. Without the residual merge, the alt would diverge on non-refactoring changes and the process-score comparison would be meaningless.

The wrap-and-patch layer. A complication which had to be resolved is that the user records sessions in IntelliJ but the pipeline applies refactorings via JDT, and the two engines sometimes produce different terminal ASTs even when they agree on a refactoring's semantics. A wrap-and-patch layer closes this gap between the two engines. Two concrete divergences observed during testing: JDT only adds the `static` modifier to an extracted method when the body requires it, while IntelliJ adds it whenever the body permits; and JDT under-detects outer-scope-variable liveness in conditional branches, producing a void-returning method where IntelliJ would produce a value-returning one. A host-side post-pass detects each pattern and rewrites the bundle's output to match. Where the residual gap cannot be patched safely, the bracket is marked as diverged and not reordered: the wrap-and-patch layer expands the set of allowed brackets but does not fully bridge the two engines. Fully covering all JDT-IntelliJ structural differences remains future work (discussed in §3.3.6).

3.3.4 Rework

Rework detection looks for pairs of changes within a trajectory where content is added and later removed (or removed and later re-added) such that the two halves net to a no-op, modulo whitespace. The detector parses each step's unified diff into hunks, resolves each hunk to its enclosing method or class scope, content-hashes the affected lines (whitespace-normalised), and greedily pairs add/remove hunks that share the same `(file, scope,` `content-hash`). Each paired `(originating step, terminal step)` becomes a *Rework* candidate; pairs below the *min_rework_lines* floor are dropped as line-noise.

The rework synthesiser builds an alternative trajectory in which the originating–terminal hunk pair is skipped. For each pair, it replays the [originating, terminal] window as a sequence of surgical patches that omit the reworked chunk while preserving every other change in the window. The operation is atomic per trajectory: either all patches apply, or the alt is thrown away - a partial replay never produces a half-baked alternative.

A key issue is that the intermediate edits have shifted line numbers around the chunk, so the terminal-step surgery has to translate its line coordinates from the user's tree (where the chunk reappears) to the synthesised tree (where the chunk was never written, so the file is several lines shorter). A per-file drift tracker maintains the divergence between the two trees as a set of zones; each intermediate user hunk shifts every zone past it by the hunk's net line delta, so each zone stays anchored at the same logical content as the user's tree grows or shrinks around it.

After surgery, some of the intermediate checkpoints have no remaining content of their own – their only change was the wasted work the surgery has now excised – and some only contain whitespace changes, such as blank-line edits that surrounded the reworked chunk. Empty checkpoints are dropped from the alt trajectory entirely rather than being committed as no-op steps; checkpoints whose surviving change is purely whitespace are folded into the preceding checkpoint. Both moves shorten the alt's step count, which lifts its process score directly through the step-savings term defined in §3.2.1.

3.3.5 Hygiene

Hygiene detection has two sub-kinds. *Tests-Skipped* fires once per composite window (60-second batch [5]) that contains at least one refactoring step and no successful test run in between. *Commit-Gap* fires once per stretch of ≥ 6 green-refactor checkpoints (checkpoints whose tests passed and whose refactoring applied cleanly) without an intervening user commit. Both sub-kinds emit one divergence point per finding.

The hygiene synthesiser builds alternatives where the user ran tests or committed at points where they didn't, without changing the code itself. For *Tests-Skipped*, the alt represents a trajectory in which the user ran the test suite at the anchor checkpoint; the synthesiser injects a synthetic `TEST_RUN_FINISHED` event at that checkpoint but does not touch the test outcome (`tests.success` or `tests.wasSkipped` are left as the user recorded them). The counterfactual is "you should have run the tests here", not "they would have passed if you had": W_{ST}, which is gated on whether tests were run, correctly lifts; W_{B}, which is gated on the broken/passing outcome, is untouched. For *Commit-Gap*, the alt represents a trajectory in which the user committed at the anchor checkpoint, breaking what would otherwise have been a long stretch of uncommitted refactor checkpoints.

Both alter the process metadata at the anchor checkpoint, not the code-because hygiene is fundamentally a process property (when did tests run, when did commits happen). Synthesis amounts to flipping the relevant process flag on the anchor checkpoint and feeding the trajectory back through Phase B (§3.4). By rescoring the trajectory with adjusted metadata and comparing final scores, we measure the cost in process-score points of the skipped hygiene event.

3.3.6 Validator and behaviour preservation

The bracket-level validator checks that replaying the user's specs in order reaches the user's final state. For each `(fromSha, toSha)` bracket, it replays the specs and verifies that (i) the set of `.java` files changed matches `git diff fromSha..toSha`, and (ii) the terminal AST hash of each file matches the user's. Brackets that pass both checks are admitted as *valid* and become reorder windows; those that fail are treated as individual steps that can't be reordered. The replay reuses the same worktree-borrow and JDT-batch-session machinery as the reorder engine (§3.3.2).

The validator's admission rate is limited by JDT-IntelliJ engine differences (detailed in §3.3.3): when the wrap-and-patch layer covers a divergence, the bracket is admitted; when it doesn't, the bracket fails validation even though the refactoring was correct. Covering all JDT-IntelliJ structural differences is future work and is the main reason *Ordering* recall isn't higher (reported in §4.2.2).

3.4 System architecture and pipeline phases

This section sketches the four JVM modules that make up the tool and how they are staged across the analysis pipeline. §3.4.1 shows the modules and their dataflow; §3.4.2 describes the Phase A / Phase B split that the implementation uses.

3.4.1 Architecture at a glance

The tool consists of four JVM modules (`:ide-plugin`, `:analysis`, `:refactoring-bundle`, `:shared`) plus a standalone React/TypeScript dashboard (`dashboard/`):

- `ide-plugin/` – an IntelliJ plugin that records the developer's session as a stream of typed events and uploads it to the analysis server. The user manually starts and stops recording a button in the IDE.
- `analysis/` – the Kotlin pipeline; it ingests sessions, reconstructs a shadow git repo, mines and synthesises trajectories, computes per-checkpoint metrics, and emits an `AnalysisReport`.
- `refactoring-bundle/` – a headless Eclipse JDT distribution wrapped in an embedded Equinox OSGi framework; the analysis module talks to it across the OSGi classloader boundary (details in §3.3.3).
- `dashboard/` – a React/TypeScript single-page app that renders the `AnalysisReport` (details in §3.5).

Figure 3.1 shows the dataflow at a glance.

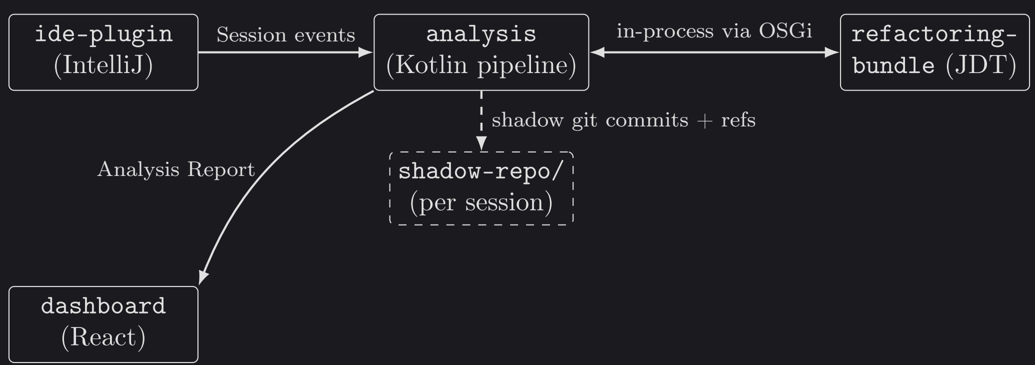

Figure 3.1: High-level dataflow between the recording side (IDE plugin), the analysis pipeline, the embedded Eclipse JDT engine, and the dashboard. The shadow repo holds every reconstructed state – both user checkpoints and synthesised alternatives – as git commits in a per-session repository.

3.4.2 Phase A / Phase B split

The detector and synthesisers above run as stages of an offline analysis pipeline split into two phases on either side of a single kotlinx-serialisable boundary type, `PhaseAResult`.

Phase A. Phase A covers every stage above – event ingestion, normalisation, shadow-repo reconstruction, refactoring mining, the four synthesisers, the validator, the per-checkpoint metrics computation, and the PMD / CPD cross-checkpoint tracking. It takes on the order of minutes per session, dominated by the per-checkpoint Gradle build and JUnit test runs of §3.2.6 and the Equinox-hosted JDT applications of §3.3.1. Phase A emits a `PhaseAResult` containing the trace, the reconstruction, the mined steps and their typed refactoring specs, the per-checkpoint metrics for both the user's trajectory and every synthesised alt SHA, the diff bundle, the PMD / CPD tracking edges, and the reorder trajectories.

Phase B. Phase B takes that record together with a `ScoringConfig` (the six process-score weights of §3.2.2) and runs the `ReportAssembler`, which calculates and clamps the trajectory's process score to the [0,100] range, attaches alt-vs-user deltas at every checkpoint, and emits the final `AnalysisReport`. Phase B does no filesystem I/O, no JDT work, and no test runs; it is pure in-memory tree-walking and finishes in milliseconds.

This split matters because weights only enter at Phase B: changing them won't break any cached results from Phase A. The sensitivity experiment of Chapter 4 exploits this directly: $12 \times 9 \times 45 = 4{,}860$ scoring configurations are evaluated against the same cached set of `PhaseAResult` dumps in roughly 2.6 seconds of wall-clock time, because each configuration only re-runs `ReportAssembler`. Two Gradle entrypoints make the boundary visible at the CLI layer: `:analysis:phaseA` dumps a `PhaseAResult` JSON to disk, and `:analysis:phaseB` re-runs the cheap second phase against a Phase-A dump and a chosen `ScoringConfig`.

3.5 Dashboard

The detector and synthesisers above produce an analysis report; the dashboard surfaces it to participants. The dashboard is a React web app that consumes the analysis report and presents the session interactively. It is hosted inside the IDE rather than served as a separate web page: the plugin opens a non-modal IntelliJ dialog containing a `JBCefBrowser`, the IDE's JCEF (Java Chromium Embedded Framework) wrapper, and points it at the React app. Once the page finishes loading, the plugin injects the freshly produced `analysis-report.json` into the page as `window.__REPORT__` and dispatches a `refdash:report-loaded` event; a `useReport()` hook on the React side picks the report up from there. Keeping the renderer inside JCEF means the dashboard is always shown next to the IDE state that produced it (no external server, no cross-origin handshake), while keeping the React + TypeScript developer ergonomics of a normal SPA. Figure 3.2 shows the dashboard in use on one of the user-study sessions.

The trajectory chart plots the user's process score over time, with the synthesised alternative trajectories overlaid as comparison lines so the developer can see how each counterfactual would have scored against their own. Clicking on a refactoring step opens a detail panel showing what the step did, which smells it introduced or eliminated, and which code-quality metrics shifted across it. Each sub-signal of the cleanliness score (PMD violations, CPD duplications, readability, Chidamber–Kemerer measures) is itself click-through: the panel drills down to the underlying per-file readings so the developer can inspect why a step's cleanliness score moved, not just by how much. Divergence points appear as flagged moments on the trajectory, each one a click-through to the counterfactual that scored higher. A separate panel summarises the highest-impact suggestions for the session as a whole, so the developer has both a per-step view and a session-level takeaway.

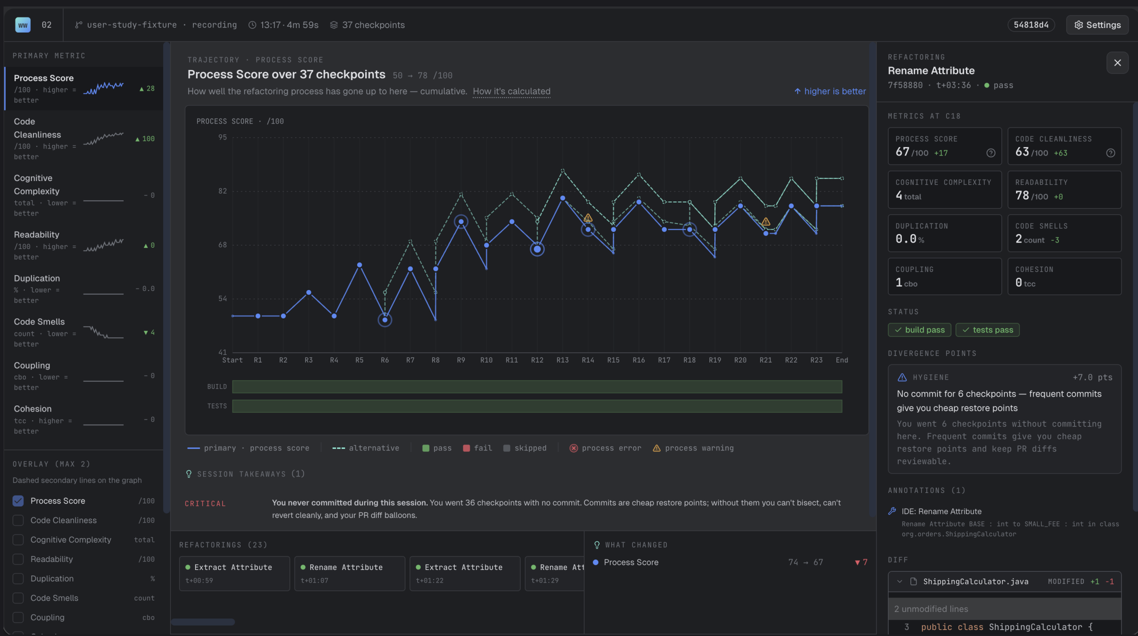

Figure 3.2: The dashboard hosted inside an IntelliJ JCEF window. The trajectory chart (centre) plots the user's process score over the session's 37 checkpoints, with the synthesised alternative trajectory overlaid as a dashed comparison line and divergence-point flags marked along the path. The left rail lists each per-state cleanliness signal with its session-level sparkline; the right rail shows the per-checkpoint detail panel for the currently selected refactoring (a `Rename Attribute`*), including the divergence point it produced, the IDE-event annotation, and the diff of the underlying file. The bottom strip summarises the session's highest-impact takeaway (left) and the per-metric delta across the selected step (right).*

3.6 Session corpus and labelling

The primary test data the experiments in Chapter 4 run against is a set of 45 refactoring sessions we recorded by hand on a Java library-style fixture in IntelliJ. This section describes how the sessions were assembled, the two label streams each session carries, and the rater protocol that establishes those labels as a defensible ground-truth signal.

The IDE-event capture surveyed in §2.6 demonstrates that traces of this kind can be collected, but no existing corpus combines per-event code content with IDE-side refactoring-action events on a Java/IntelliJ target. The IDE-versus-manual signal in particular is a capture-time property: RefactoringMiner can recover the structural refactoring from a diff pair, but it cannot say whether the developer invoked the IDE menu or hand-typed the edit, and that distinction drives the manual-when-IDE penalty in §3.2.1. A corpus that did not record IDE-emitted refactoring events at the time the developer ran them loses that signal permanently. Separately, the precision and recall claims of §4.2.2 are computed against a controlled-injection design in which each session deliberately exercises one specific behaviour - a property of the study, not of the dataset, but one a free-form corpus does not directly support. For both reasons we recorded our own corpus, instrumented at the IDE-event level via the plugin of §3.2.4.

3.6.1 The 45-session injection set

Each session follows a scripted playbook scenario that deliberately induces one specific "bad" refactoring behaviour - a step ordering the playbook author intended as worse than its alternatives, an undo-redo cycle on the same chunk of code, a skipped test cadence, a manual edit where an IDE refactor would have worked, and so on. The playbook covers all four divergence kinds, with 5 ordering injections, 9 rework, 13 hygiene, and the remainder split between IDE-replay injections and clean-control sessions where no bad behaviour is induced.

3.6.2 Two label streams

Each session carries two distinct labels, used for different parts of the detector evaluation. The first names the deliberately-injected divergence kind - what the experiment was designed to trigger. This is the signal the *injection prominence* claim of §4.2.2 is computed against. The second is a separate hand-label naming the full set of divergence kinds the session should fire, computed independently and allowed to name multiple kinds where a session legitimately exercises more than one. This is the signal the per-kind precision and recall claims of §4.2.2 are computed against, and is also the label set whose inter-rater agreement is reported in §4.2.1.

The multi-label schema operates over $\mathcal{K} = \{\textit{IDE-Replay}, \textit{Rework}, \textit{Hygiene}, \textit{Ordering}\}$. Each session carries a row in `fixtures/sessions/manifest-v2.csv` whose `expected_kinds` cell holds a semicolon-separated subset of $\mathcal{K}$. A session may legitimately exhibit multiple kinds - for example, both an ordering divergence and a hygiene flag at the same step - and the manifest captures all of them.

3.6.3 Rater protocol

To check the schema against someone other than us, two independent annotators (denoted P1 and P2 throughout the results chapter) labelled all 45 sessions against the same schema. P1 and P2 produced their own independent labelling before either performed a user-study session of their own. Each rater received the schema definition and the session artefacts for every recording - the raw event log `events.jsonl` and the wrapping session metadata `session.json`. They did not receive the playbook prompts that drove each recording, our `manifest-v2.csv` labels, the per-session `analysis-report.json` produced by Phase A, or any view of the dashboard. The raters read the sessions without seeing the playbook prompts or the detector's analysis, ensuring an unbiased assessment. Because each `events.jsonl` entry stores the full post-event file contents rather than a delta, raters were also given a small read-only helper script (`tool/scripts/session-event-diffs.py`) that prints a per-event unified diff for every *edit_burst* and *refactoring_finished* event. The script is a presentation aid only; it derives its output from the same `events.jsonl` the raters already had and surfaces no additional information.

3.6.4 Cohen's κ computation

Per-kind agreement is reported using Cohen's κ [58], applied separately to each kind in $\mathcal{K}$. For each kind k, the rater's label is treated as the binary judgement "$k \in$ `expected_kinds` for this session", and the resulting 2×2 confusion matrix yields a single κ per rater pair. The Landis and Koch thresholds [59] are used for interpretation: $\kappa < 0.20$ slight, 0.21–0.40 fair, 0.41–0.60 moderate, 0.61–0.80 substantial, 0.81–1.00 almost perfect. With only 45 sessions, the raw agreement counts (yes/yes, no/no, and disagreements from each 2×2 table) are reported alongside κ, since these cell counts are small enough to be informative on their own.

3.6.5 Treatment of disagreements

When the two raters disagree with us on a particular kind, the disagreement is reported as a finding rather than edited into the manifest. Our labels are not changed post-hoc, and neither are the raters' labels. When both raters agree with each other but disagree with us (as happens for rework on session 043), those sessions are named explicitly in the results chapter, with a note that our labels may be the outlier.

The rater study results-per-kind κ values, raw agreement counts, and disagreement patterns-are reported in Table 4.8 (§4.2.1).

Chapter 4

Experimentation and Evaluation

This chapter tests the system component-by-component. The previous chapter proposed (i) a process-quality score that ranks refactoring trajectories on process behaviours, and (ii) a divergence-point detector that uses that score to identify and synthesise concrete alternative trajectories at points where the user could have done better. Each is testable on its own terms, and end-to-end behaviour can be tested on top.

§4.1 tests the score formula directly: whether its divergence-point ranking is stable to weight perturbations, whether every term in the formula actually contributes to the ranking, and an overall finding about what the formula does and does not detect on the session set. §4.2 tests the detector against external ground truth: whether three independent labellers agree on which divergence kinds each session contains, then per-kind precision and recall against those labels, then one known recall weakness that is deliberately left open. §4.3.1 tests the tool end-to-end: a 12-session user study with two participants on a new fixture, and a 6-session comparison run on the same fixture by an automated coding agent. Together the three parts test each contribution against the question that makes it credible - the formula against weight stability and term contribution, the detector against external labels, the tool against use by people other than us.

The 45-session injection corpus and its two-label scheme are defined in §3.6. Per-cell counts across the four kinds and 45 sessions are small, so raw counts are reported alongside percentages everywhere they appear. Every magnitude number in this chapter is the trajectory-final J delta defined in §3.2.1, uniform across all four divergence kinds.

4.1 Testing the score formula

This section evaluates the process-quality score on its own terms. The score's weights were set within a literature-supported severity ordering (§3.2.2) rather than fitted to an external outcome dataset, so the rankings the formula produces are credible only if (a) they are stable to plausibly-different weight choices within that ordering, and (b) every term in the formula actually contributes to those rankings. §4.1.1 tests stability via single-knob and multi-knob weight perturbations. §4.1.2 tests term contribution via a full power-set ablation across $2^6 = 64$ weight subsets. §4.1.3 then reports one cross-cutting observation that emerges from both: a structural limit on what the formula can detect even when it is otherwise well-behaved.

4.1.1 Sensitivity sweep

The first experiment asks how the divergence-point ranking responds to perturbations of the weights in the process-score formula. If tweaking a weight significantly reshuffles the ranking, then the ranking depends on choices we cannot justify. The experiment runs against three session sets and uses two perturbation approaches. The first is a single-knob sweep that varies one weight at a time. The second is a multi-knob Monte Carlo that draws every weight simultaneously from a log-normal distribution centered around the production weights. The headline result is top-1 stability across all three session sets and both approaches, reported below alongside the places where the formula does respond to weight perturbations.

Design. The single-knob sweep fixes every weight in `ScoringConfig.PRODUCTION` except one, scales the chosen weight by one of nine factors in $\{0.1, 0.25, 0.5, 0.75, 1.25, 1.5, 2.0, 4.0, 10.0\}$, reassembles the analysis report, and compares the perturbed divergence-point ranking against the production baseline. There are thirteen weights (7 process-side and 6 cleanliness-side) and nine factors per weight, so the design produces $13 \times 9 \times |\mathcal{C}|$ rows per session set $\mathcal{C}$. The multi-knob setup draws 200 independent samples per fixture. Each weight W_i is scaled by $\exp(\sigma \cdot Z_i)$ with $Z_i \sim \mathcal{N}(0, 1)$ drawn independently per weight, and $\sigma = \ln 2$, so around 95% of samples land inside $[\times 0.25, \times 4]$ on each weight, and the distribution is symmetric in log-space.

The experiment is run against three session sets: the 45 hand-labelled injection sessions of §4.2.2, the 12 user-study sessions of §4.3.1, and the 6 agent sessions of §4.3.2. These differ in their per-fixture divergence-point profiles. The injection sessions have at most 5 divergence points per fixture, 66 in total, and concentrates on single-kind exercises. The user-study sessions have up to 8 per fixture across naturalistic sessions, 50 in total. The agent sessions hves up to 3 per fixture across structurally simpler sessions, 10 in total.

Three ranking-stability statistics are reported per row. Kendall's τ-b [60, 61] is a rank correlation coefficient designed for short rankings on small samples, reported as the percentage of rows where $\tau = 1.0$ (ranking unchanged) and separately as the percentage where $\tau < 0$ (ranking inverted). Top-N hit rate is the fraction of the production top-N that survives in the perturbed top-N, reported at $N = 1, 3, 5$. Top-1 is the only metric that works across all three session sets, since it bites on any fixture with at least one divergence point. Top-3 and top-5 are reported on the user-study sessions only, where per-fixture sizes are large enough to make them informative; on the other two session sets every fixture has at most 5 divergence points and the top-5 comparison collapses to set-preservation [62]. Both metrics require a deterministic ranking, but several divergence-point kinds produce ties: every *Hygiene* and *Commit-Gap* point carry the same magnitude (exactly W_{CG}, since the commit-flip alt drops the gap count from one to zero at trajectory-final), and divergence points saturated against the $[0, 100]$ score clamp share a clamp-induced magnitude that is weight-invariant. Ties are broken by ascending step index, applied uniformly across both the production baseline and every perturbed ranking, so the comparison is well-defined.

Single-knob results. Table 4.1 reports top-1 stability across the three session sets. The user-study sessions is the most demanding test: it has the longest per-fixture rankings, the highest divergence-point density, and zero clipped baseline divergence points against the $[0, 100]$ score clamp. The injection sessions shows the highest stability, but only because it has shorter rankings to reshuffle in the first place. The agent sessions sit between them, but 30% of its baseline divergence points hit the score suppression clamp, which suppresses the observable effect of weight perturbations. On the user-study sessions the top-1 recommendation is preserved on 1,239 of 1,296 rows (95.6%).

Sources of ranking instability. Table 4.2 reports per-knob top-1 disruption counts on the user-study sessions. The pattern is clear. The lag term and five of the six cleanliness sub-weights (cognitive, coupling, duplication, readability, smells) never disrupt the top-1 recommendation across any of the 108 rows per knob. The cohesion sub-weight disrupts

session set	rows	$\tau = 1.0$	top-1 = 1	$\tau < 0$	baseline saturation
Injection (45)	5,265	99.0%	99.7%	0.2%	13.6%
User study (12)	1,404	87.3%	**96.1%**	0.7%	**0.0%**
Agent (6)	702	95.6%	95.9%	4.1%	30.0%

Table 4.1: Single-knob sensitivity across three session sets. The user-study sessions is the cleanest benchmark: longest per-fixture rankings, zero saturated baseline divergence points. Top-3 and top-5 stability on the user-study sessions are 93.0% and 98.1% respectively; top-5 on the other two session sets is trivially 100% because no fixture there has more than 5 divergence points.

it exactly once, the smallest non-zero entry in the table — a single rank flip attributable to the cleanliness↔lag coupling described in §3.2.3 (the lag denominator uses the cleanliness scalar). Every other top-1 shift is driven by a process-side weight. The two largest disruptors are the commit-gap weight W_{CG} (18 of 108 rows) and the manual-when-IDE weight W_{MI} (11 rows). The remaining process-side weights account for between 4 and 8 shifts each, including the cleanliness-gain weight W_{G}, which only disrupts the top-1 at $\times 2$, $\times 4$, or $\times 10$. This decomposition tells us the score formula's *effective parameter dimension* is closer to four or five than to thirteen. The cleanliness composite usually adds the same amount to both paths so it does not change the ranking, and the process-side weights do the rank-shaping.

knob	top-1 disruptions / 108 rows
`commitGap`	18
`manualIde`	11
`length`	8
`skipTests`	8
`broken`	5
`gain`	4
`lag`	0
cleanliness sub-weights: `cohesion` 1, *all others* 0	

Table 4.2: Per-knob top-1 disruption counts on the user-study sessions. The lag term and five of the six cleanliness sub-weights never reshuffle the highest-magnitude divergence point in the sweep; the single `cohesion` *entry reflects the lag-denominator coupling. Process-side weights account for almost every top-1 shift.*

Multi-knob Monte Carlo. The single-knob sweep only tests one weight at a time. The Monte Carlo approach perturbs every weight together: 200 samples per fixture, with each weight drawn independently from $\exp(\ln 2 \cdot Z)$ for $Z \sim \mathcal{N}(0, 1)$. Table 4.3 reports the distribution of τ and top-N hit rate across $200 \times |\mathcal{C}|$ joint-perturbation samples per session set.

The multi-knob result is meaningfully weaker than the single-knob headline. On the user-study sessions, the top-1 recommendation changes on 19% of joint-perturbation samples, against 4% under single-knob, and mean τ falls to 0.785. The injection sessions are much more stable at $\tau = 0.975$ mean, but again only because their per-fixture rankings are too short to allow much reshuffling. The agent sessions sits between them on τ, and its top-3 stability is trivially 100% because every agent fixture has at most 3 divergence points. In short, the formula is robust to small joint perturbations and degrades smoothly with larger ones. It is not robust to arbitrary combinations of weights, even within the realistic range we tested.

Saturation and score-clamp bias. One concern is whether the τ and top-N numbers reflect the $[0, 100]$ clamp on the process score rather than true stability. When both user

session set	samples	mean τ	top-1 $= 1$	top-3 $= 1$	top-5 $= 1$
Injection (45)	9,000	0.975	98.7%	97.6%	100.0% †
User study (12)	2,400	**0.785**	**81.3%**	**75.0%**	92.3%
Agent (6)	1,200	0.748	87.1%	100.0% †	100.0% †

† trivially 100%: no fixture in that session set has $> N$ baseline divergence points.

Table 4.3: Multi-knob Monte Carlo robustness across three session sets (200 samples per fixture, log-normal $\sigma = \ln 2$). The user-study sessions are the discriminating benchmark; the multi-knob top-1 stability is ~ 14 percentage points lower than the single-knob result on the same set.

and alt scores hit the ceiling, their magnitude is zero regardless of weights, which would inflate τ. Saturation is therefore tracked per fixture. The right-hand column of Table 4.1 shows the three session sets differ substantially. The injection sessions has 13.6% saturated baseline divergence points, concentrated on seven short fixtures. The agent sessions has 30.0%, because most agent sessions push the score close to the ceiling on the easier playbook sessions. The user-study sessions has zero saturation. This is the main reason for leading with the user-study sessions: its τ and top-N values are clean perturbation responses, not clipping artifacts, and the multi-knob top-1 $= 81\%$ figure isn't explained by saturation.

Caveats and scope. Three main limitations. First, the experiment establishes that the formula is stable near the production weights. It does not establish that the production weights themselves are uniquely correct, only that small changes to them mostly preserve the ranking. To properly validate this, we'd need to refit the weights against an external outcome dataset, e.g. longitudinal code-quality measurements, but no such dataset exists for this problem. Second, the multi-knob test is centered around the production weights and tests the neighborhood within roughly $\pm 2\sigma$ (about $\times 0.25$ to $\times 4$). The formula visibly degrades on the user-study sessions even inside this range ($\tau = 0.78$ mean), so the robustness claim is local rather than global. Third, the user-study sessions are $n = 12$ sessions across 2 participants; we haven't tested it on a broader set of naturalistic sessions. We document these limits as threats to validity in §4.2.3 and the discussion chapter. The chapter does not claim more than the experiments support.

TODO: MOVE ABOVE TO VALIDITY SECTION

4.1.2 Ablation sweep

The sensitivity sweep shows the ranking is robust to weight perturbations. This experiment asks the different question of whether every term in the score formula carries substantial weight. If removing any one term left the ranking unchanged, that term would be decorative, and if zeroing every process term simultaneously did not collapse the divergence-point magnitudes to zero, then some weight-independent signal would be doing the work and the formula would just restate that signal. A full power-set sweep across the seven process-side weights ($2^7 = 128$ variants per fixture, 5,760 rows total) rules out both possibilities: sum-of-magnitude recovery rises monotonically with the count of active terms from 0.000 (every process term zeroed) to 1.000 (full formula), and mean τ rises in parallel from 0.883 to 1.000.

Design. The experiment enumerates every subset of the seven process-side weights, testing each subset by keeping those weights at production and zeroing the rest. With seven weights the full $2^7 = 128$-variant power set fits comfortably and runs in roughly 3 seconds of wall-clock time. This approach draws from two research communities. Composite-indicator sensitivity analysis [63] establishes systematic weight perturbation as standard practice for composite indices. Shapley-value feature attribution [64, 65] uses enumeration over all 2^n

subsets and is exact rather than approximate when n is small; sampling approximations such as KernelSHAP only become necessary once n becomes large and enumeration becomes intractable. With $n = 7$ here, a full enumeration is the appropriate choice. The cleanliness sub-weights are kept at production across all variants because the sensitivity sweep already showed cleanliness perturbations almost never reshuffle the ranking; ablating them would inflate the variant count from 128 to 8,192 for minimal new information.

Two recovery statistics are reported. Mean recovery is the per-fixture ratio of perturbed total absolute magnitude to the baseline total absolute magnitude, averaged across fixtures, and is shown for consistency with earlier analysis. Sum-over-sum recovery is the sum of |magnitude| across every divergence point in every fixture, divided by the same sum at production, and is the cleaner measure of the fraction of the sessions' total magnitude that survives a given ablation. Sum-over-sum is better than per-fixture mean because it avoids the $0/0 = 1.0$ problem on empty fixtures. We treat it as our main metric.

Sanity check. The full-formula variant (all seven terms active) reproduces production exactly: $\tau = 1.000$, recovery $= 1.000$, and mean $|\Delta\text{magnitude}| = 0.000$ on every fixture. Comparison and production use the same system.

Monotone by active-term count. Table 4.4 groups the 5,760 rows by how many process-score terms are active. Both τ and sum-over-sum recovery increase with each added term, with no jumps. The key finding is when all terms are zeroed (active-count $= 0$): the total magnitude across all 45 fixtures drops to zero. This means there's no baseline magnitude independent of the weights; every term does real work, and removing all seven leaves nothing behind. (The mean-recovery value is 0.311 at that point, which is why we use sum-over-sum instead.)

active terms	**mean τ**	**mean recovery**	**sum/sum**	**n**
0 (all stripped)	0.883	0.311	**0.000**	45
1	0.886	0.430	0.170	315
2	0.893	0.534	0.327	945
3	0.904	0.632	0.473	1,575
4	0.920	0.726	0.612	1,575
5	0.942	0.817	0.745	945
6	0.968	0.908	0.874	315
7 (full)	1.000	1.000	1.000	45

Table 4.4: Mean τ and magnitude recovery grouped by how many process-score terms are active. Both statistics increase with the number of active terms; sum/sum recovery reaches 0 when all terms are zeroed.

Per-term single-knob recovery. Table 4.5 shows how much magnitude each term contributes when it's the only active term. Four observations follow. The `length` and `broken` terms are the strongest contributors by sum/sum, each recovering roughly a third of the total; these match the values in Table 3.2 (§3.2.2), connecting methodology to results. The new `lag` term lands fourth by mean recovery (0.423) and fourth by sum/sum (0.132), confirming the methodology's prediction that the intermediate-cleanliness penalty does measurable work on its own without dominating any single existing contributor. The `gain` term alone recovers 0.000 despite having the largest weight (50), because gain relies on the cleanliness delta ΔC, which is near zero when user and alternative reach the same state; gain has no effect alone but matters in combination, as Table 4.4 shows. Finally, the gap between `commitGap` (0.080) and `length` (0.328) shows that recovery depends on both weight and how much the signal varies across sessions, not solely weight alone.

term	mean τ	mean recovery	sum/sum
`length`	0.883	0.578	**0.328**
`broken`	0.919	0.553	0.302
`manualIde`	0.883	0.375	0.261
`lag`	0.778	0.423	0.132
`skipTests`	0.883	0.398	0.089
`commitGap`	0.972	0.368	0.080
`gain`	0.883	0.311	**0.000**

Table 4.5: Per-term single-knob recovery (each term active alone, others zeroed). The `length` and `broken` terms are the strongest contributors by both mean recovery and sum/sum; the new `lag` term is fourth on both. `gain` alone recovers 0.000 because the cleanliness delta is near zero on most fixtures, so it has no effect when isolated.

Per-term leave-one-out recovery. Table 4.6 shows what happens when you remove one term from the full formula. Removing `length` causes the largest drop in recovery (0.718), so the step-savings bonus from reorder and rework contributes more than any other single term. Removing `gain` pushes recovery above 1.000 to 1.060 - the most surprising result: without `gain`, total magnitude is larger than at production. The unit being summed here is the per-divergence-point magnitude defined in §3.2.1, i.e. the trajectory-final $J(\tau^*) - J(\tau)$ delta between each alternative and the user it is compared against; the sum is over |magnitude| across every surfaced divergence point, including the ones whose alternative scored worse than the user (negative magnitudes contribute their absolute value). The mechanism behind the inflation is that `gain` $\cdot\Delta C$ is small but typically positive for alternatives, so it partially offsets the penalty terms; remove it and the penalty terms determine the alt-versus-user delta unopposed, producing larger per-DP absolute magnitudes which accumulate in the sum. Were the experiment to filter to alt-beats-user DPs only, removing `gain` would in fact lower the total, since alternatives sitting just above the beats-user threshold flip below it without the offset. The `gain` term therefore plays a regularising role against the penalty terms rather than acting as a positive contributor in isolation. Removing the new `lag` term costs 0.158 on sum/sum (production $1.000 \to 0.842$) while preserving the ranking ($\tau = 0.996$); the lag knob carries real magnitude into the divergence-point ranking without itself reshaping which divergence points top the list. Removing `skipTests` or `commitGap` reshuffles the ranking somewhat but moves recovery less than removing `length` or `manualIde` does, so these terms primarily affect which divergence point is biggest rather than how big.

removed term	mean τ	mean recovery	sum/sum
`length`	1.000	0.817	**0.718**
`manualIde`	1.000	0.911	0.822
`broken`	0.887	0.966	0.833
`lag`	0.996	0.880	0.842
`skipTests`	0.970	0.903	0.899
`commitGap`	0.930	0.956	0.943
`gain`	0.993	0.925	**1.060**

Table 4.6: Per-term leave-one-out recovery (six terms active, one removed). Removing `length` causes the largest sum/sum drop; removing `gain` inflates sum/sum above 1.000, indicating that `gain` regularises against the penalty terms rather than acting as a positive contributor in isolation. Removing the new `lag` term costs 0.158 on sum/sum while leaving the ranking essentially intact ($\tau = 0.996$).

Cross-set rerun on the user-study sessions. The ablation above runs on the 45 injection sessions, which have few divergence points per fixture (max 5, mean 1.5). Running the same test on the 12 user-study sessions, which have larger rankings (max 8, mean 4.2) and no saturation, produces different results. Table 4.7 compares leave-one-out τ across both sets. On user-study sessions, the most important term is commit-gap (W_{CG}): removing it drops τ from 1.000 to 0.599, roughly twice the drop of any other term. On injection sessions, W_{CG} is one of the smaller-impact terms ($\tau = 0.930$ on removal). The reverse holds for W_{MI} and W_{L}: they drive injection-session ranking but have smaller effects on user-study sessions. This difference matters: which terms matter depends on the type of sessions, so the weights have different effects on different session types. The new lag term sits near the bottom of the user-study disruption list ($\tau = 0.992$ on removal), confirming that the cleanliness-deficit signal it introduces is more visible on the controlled injection sessions than on the naturalistic user sessions. Removing the cleanliness-gain term W_{G} still has no effect on either set, so it acts as a regulariser, not a rank-shaper.

removed term	injection τ	user-study τ
`commitGap`	0.930	**0.599**
`manualIde`	1.000	0.842
`skipTests`	0.970	0.844
`length`	1.000	0.877
`broken`	0.887	0.901
`lag`	0.996	0.992
`gain`	0.993	1.000

Table 4.7: Leave-one-out ranking stability against the injection sessions (Table 4.6) and the user-study sessions. The term-importance ordering inverts between the two session sets: W_{CG} is the dominant rank-carrier on the user-study sessions, W_{MI} and W_{L} on the injection sessions. The lag term has near-zero ranking impact on the user-study set ($\tau = 0.992$) but contributes magnitude on the injection set.

Caveats. Five key limits apply. The cleanliness sub-weights are not ablated because they had almost no effect (§4.1.1). Saturation has only a small effect on this experiment: the mean saturated-divergence-point count is 0.20 to 0.29 across variants, so saturation doesn't greatly reduce the signal. τ-b is unstable on small divergence-point counts, and many fixtures have one or two divergence points where τ is 1.0; mean τ at active-count = 5 partly reflects this floor, not pure ranking quality. The `gain`-alone row equals the empty baseline on recovery (0.000); the regularizer role shown in the leave-one-out table explains this. Finally, with the earlier line-count approach the active-count = 0 floor was 0.119 because line-count magnitudes were weight-invariant by design; with the trajectory-final approach used here the floor collapses to 0.000. The relative importance of terms is consistent across both approaches.

4.1.3 Reordering as a window onto intermediate quality

The most informative finding is what reordering tells us about *when* cleanliness arrives. The reorder synthesiser tested 194 permutations across the 45 sessions, which collapsed into 24 distinct reorder windows. Under the production score formula, 10 of those 24 windows (41.7%; Table 4.10) have a permutation that strictly beats the user's order; a further 2 windows acquired *negative* magnitude, meaning the synthesised alternatives front-loaded *less* cleanliness than the user did. Both numbers depend critically on the lag term W_{LAG} in the score: with the lag term removed, only 4 of the 24 windows produce a positive magnitude and the negative-magnitude verdicts collapse to zero. The lag term is therefore the operative lever for the *Ordering* divergence-kind, and the headline finding of this experiment.

The mechanism is straightforward. Most IDE refactoring pairs are designed to be independent [66]: they commute, and every permutation reaches the same final state with the same final cleanliness metrics. Under an endpoint-only score, the gain term $W_{\text{G}}\,\Delta C$ is identical across permutations and ordering becomes invisible. The lag term changes this: it measures the per-checkpoint deficit $\max(0, C(s_T) - C(s_i))$, so a permutation that front-loads the cleanliness-raising step accumulates a smaller running mean than one that defers it. Two permutations with the same endpoints can now score differently if their intermediate cleanliness trajectories differ – which is precisely the discrimination an endpoint-only formula was blind to. The negative-magnitude verdicts on 2 of the 24 windows are the same mechanism running in reverse: on those windows the user front-loaded cleanliness better than the synthesised alts, and the lag term correctly reports the user's order as cheaper.

This has theoretical support that cuts both ways. Mens, Taentzer and Runge [66] show that refactorings as graph transformations with parallel independence (empty critical pairs) reach the same final state under every permutation, and IDE refactorings are designed to have this property – which explains why endpoint-only formulations score them identically. Prior empirical work either claims order matters based on interactions between code smells [67] or tests on a single small fixture without checking AST equivalence [68]. Our 45-session, 194-permutation result is, to our knowledge, the first systematic evidence on real IDE sessions that independent refactorings differ measurably in process quality even when they converge on the same final code – as long as the score formula has an intermediate-quality term.

Three things follow. First, ordering of independent refactorings matters more than endpoint-only formulations suggested, but only when the score notices intermediate quality; the $4 \to 10$ shift in positive-magnitude windows when W_{LAG} is included is the direct empirical signal. Second, the synthesiser still finds 7 windows of non-commuting pairs (e.g. inline-then-rename versus rename-then-inline) where the final state itself differs; these account for most of the remaining magnitude. Third, the chapter's central methodological argument – that process-quality evaluation cannot be reduced to endpoint comparison – is supported here in the strongest way the data permits: a single weight added to the formula moves a controlled measurable quantity (the count of strictly-beats-user reorder windows) from 4/24 to 10/24, with the rest of the formula untouched.

4.2 Testing the divergence-point detector

This section evaluates the divergence-point detector against external ground truth. Precision and recall numbers on detector output are only meaningful if the per-kind labels they are computed against are themselves defensible, so §4.2.1 first establishes that three independent labellers (us, P1, P2) agree on which divergence kinds each session contains - inter-rater agreement that gives the labels standing as a ground-truth signal. §4.2.2 then measures the detector's per-kind precision and recall against those labels on the 45 injection sessions. §4.2.3 discusses one known recall weakness in the resulting numbers that is deliberately left open.

4.2.1 Inter-rater reliability

P1 and P2 each labelled all 45 sessions against the multi-label schema in §3.6, working only from `events.jsonl`, `session.json`, and a written copy of the schema. Neither rater saw the playbook prompts, our own labels, the divergence-experiment analysis reports, or any dashboard output; the labelling pass ran before either rater performed a user-study session. Table 4.8 reports per-kind Cohen's κ [58] for the three rater pairs against the

45-session set. All four kinds reach at least substantial agreement by the Landis and Koch thresholds [59]. Ordering and IDE-replay land at $\kappa = 1.00$ across every pair: all three labellers agree on every one of the 45 sessions. Rework lands at $\kappa = 0.86$ between us and each of the independent raters, and at $\kappa = 1.00$ between P1 and P2. Hygiene lands between $\kappa = 0.72$ and $\kappa = 0.86$, depending on which pair is compared.

kind	Author vs P1	Author vs P2	P1 vs P2
Ordering	1.000	1.000	1.000
IDE-Replay	1.000	1.000	1.000
Rework	0.861	0.861	1.000
Hygiene	0.848	0.723	0.862

Table 4.8: Per-kind Cohen's κ on the 45-session set between us and the two independent raters and between the two independent raters. Ordering and IDE-replay are perfect on every pair; rework and hygiene reach substantial-to-almost-perfect agreement on every pair under Landis and Koch's interpretation thresholds.

The two imperfect kinds show specific disagreement patterns. Rework disagreements occur on exactly two sessions out of 45: session 024 (borderline suboptimal-ordering) has a rework label from both raters but not from us, while session 043 (borderline add-then-revert) has a rework label from us but neither rater. Since P1 and P2 agree with each other on both, their consistent judgment suggests our labels on these two sessions may be outliers. Hygiene disagreements cluster on three manual-move-method sessions (014, 015, 016) where P2 added a hygiene label that neither we nor P1 did; plus session 039 (borderline no-commit-stretch) where we added hygiene but neither rater did. We report these as findings rather than revise the labels, following our protocol (§3.6) not to edit labels after the fact to artificially boost agreement scores.

4.2.2 Divergence experiment

The third experiment evaluates the divergence-point detector itself against ground truth. For each of the 45 recorded sessions, the analysis pipeline generates a report using the production weights and compares the report's divergence points against the session's `expected_kinds` label (the set of kinds a perfect detector would surface alternatives for) and the implied injection kind from the session's playbook pattern. The output is a 54-column row per session covering per-kind classification (true and false positives and negatives), magnitude statistics, and injection prominence; the full sweep takes roughly 5 seconds of wall-clock time. The headline numbers, expanded in the parts below, are: perfect precision on all four kinds (rework, hygiene, ordering, and IDE-replay all at 1.00); 26 of 45 injections caught with a positive-magnitude alternative, every one of them ranked at top-1 within its own session; and – the most informative single signal – the count of *Ordering* divergence-points whose synthesised alternative strictly beats the user's trajectory rises from 4 to 10 once the lag term W_{LAG} is in the formula, while the corresponding counts for the other three kinds are unchanged. The lag term is therefore the operative lever for the *Ordering* kind: it does not manufacture new divergence-points (the total count of 24 is the same with or without it), it turns previously-flat (magnitude = 0) ordering alternatives into recognisably-better ones.

Precision and recall. We report precision and recall as separate numbers rather than rolling them into F1 or accuracy. The reason is that the two kinds of mistake the detector can make are not equally bad: if the detector shows the user an alternative that doesn't really exist (a precision miss), it actively misleads them and wastes their time; if the detector quietly fails to show an alternative that would have helped (a recall miss), the user is no worse off than if they had never opened the dashboard. Treating these as the same kind of error, the way F1 does, would hide the difference. Accuracy is also unhelpful here because most of the cells in the per-kind confusion matrix are true negatives - a detector that does nothing at all would still score around 75% accuracy on these refactoring sessions, which doesn't tell the reader anything useful.

Table 4.9 reports per-kind precision and recall. Rework and hygiene are perfect on both axes: 9 of 9 rework injections and 13 of 13 hygiene injections are caught at 1.00 precision. IDE-replay is also precision-perfect. Ordering has perfect precision but only 0.36 recall on any-expected (and 0.40 recall on injection-only), because the reorder synthesiser's `splitOnInvalid` validator check (§3.3.2) disqualifies any window containing a step the synthesiser engine cannot reproduce, and on this session set most ordering-injection windows contain at least one such step. The recall gap is documented and deliberate - its rationale is in §4.2.3. Per-kind inter-rater reliability of these labels, against two independent raters, is reported in §4.2.1.

kind	precision	recall (injection)	recall (any expected)
Ordering	1.00	0.40	0.36
IDE-Replay	1.00	0.76	0.76
Rework	1.00	1.00	1.00
Hygiene	1.00	1.00	1.00

Table 4.9: Per-kind precision and recall on the 45-session set. The "injection" recall column counts only the kind the playbook explicitly injected; "any expected" counts any kind the session's `expected_kinds` label includes.

Detection quality. A precision/recall number says nothing about whether the alternatives the detector surfaces are actually useful improvements over the user's own trajectory. Table 4.10 reports the number of divergence points the detector fires per kind, the number of alternatives it enumerates, and the fraction of fires whose best alternative beats the user's trajectory under the production score formula. Across all kinds, best-alt magnitudes range from -2 to $+23$ with mean $+6.07$, so the percentages in the "beats user" column below can be read against an absolute scale of how much each winning alternative actually improved on the user's trajectory.

Hygiene never proposes a useless alternative: every one of its 13 divergence points is a real improvement over the user's trajectory (100%). IDE-replay proposes a real improvement in 12 of 16 fires (75.0%); the four misses break down as three sessions where the user's process score was already pinned at the lower end of the $[0, 100]$ range so neither the user nor the IDE-driven alternative can score below it (the magnitude is mechanically zero rather than evidence about the `manualIde` penalty), and one session where the IDE alt scored 5 points lower than the user's manual edit. The four misses are a property of the score-range floor, not a calibration finding about `manualIde`.

Rework beats the user in 6 of 13 fires (46.2%); one of the seven non-beats is a magnitude-zero no-op (stripping the rework loop did not move the final score) and the other six produce an alternative with negative magnitude. Two readings sit on top of each other here. The first is implementation: the rework synthesiser is intentionally simple, stripping out the user's add-then-undo edits without modelling what the intermediate states were for - if the user added a guard, ran the tests, decided the guard was unnecessary and removed it, the synthesiser treats the whole excursion as wasted motion, but the user's trajectory may have legitimately benefited from the validation step (one more checkpoint, one fewer commit-gap, a test run that pushed cleanliness up). The second is methodological: undoing work is not inherently a bad behaviour. A careful user who paths through a slightly longer trajectory to verify intermediate correctness can, and on these sessions sometimes does, score higher than a trajectory that takes the shortest line to the same final state. The 46.2% beat rate is therefore not evidence that the rework detector is miscalibrated, but evidence that the score formula correctly distinguishes "rework that wasted process signal" (the six beats) from "rework that earned process signal en route" (the seven non-beats) - and that the simple loop-stripping synthesiser cannot tell the two apart by inspection of the diff alone.

Ordering surfaces 24 distinct divergence points from 194 enumerated permutations, of which 10 (41.7%) have a best permutation that strictly beats the user's order and a further 2 have a best permutation that scores *worse* than the user's order under the production formula (negative magnitude; the remaining 12 are tied at magnitude zero). The 194/24/10/2 breakdown is the operational evidence behind the headline in §4.1.3: the synthesiser enumerates 194 permutations across the session set, those collapse to 24 reorder windows (one divergence point per from-SHA / to-SHA window, magnitude set to the best permutation in the window), and the lag term W_{LAG} is what discriminates between the windows whose intermediate cleanliness trajectories differ. Without W_{LAG}, only 4 of 24 ordering windows would show a positive magnitude and the negative-magnitude verdicts would collapse to zero.

kind	DPs	alts enum.	beats user	beats fraction	max magnitude
Rework	13	13	6	**46.2%**	8 points
Hygiene	13	13	13	**100%**	9 points
IDE-Replay	16	16	12	**75.0%**	23 points
Ordering	24	194	10	**41.7%**	9 points

Table 4.10: Per-kind detection quality. "DPs" is the number of divergence points the detector fires; "alts enum." is the total number of alternative paths enumerated across those DPs (a single ordering DP enumerates up to $k!$ permutations of a k-step window); "beats user" is the number of DPs whose best alternative trajectory beats the user's trajectory. The Ordering *beats count rises from* 4 *to* 10 *once the lag term W_{LAG} is in the formula; without W_{LAG} the count collapses to* 4.

Two-way ordering discrimination. Two of the 24 ordering windows now carry negative magnitude under the production formula; session 043 is the cleanest example. The synthesised alternative converges to the same final state as the user but front-loads less cleanliness en route, so its lag term $W_{\text{LAG}}\,\ell(\tau)$ accumulates more penalty than the user's. Under the pre-lag formula the two trajectories scored identically (same endpoint, same gain term), and the divergence-point magnitude was structurally zero. The new formula correctly reports the user's order as the cheaper one. This is the same mechanism that drives the $4 \to 10$ positive-magnitude lift, running in reverse: W_{LAG} enables two-way discrimination on intermediate quality, not just one-way inflation of positive magnitudes.

Injection prominence. The most user-facing claim in this chapter is in Table 4.11: of the 26 injections the detector caught with a positive-magnitude alternative (8 hygiene, 12 IDE-replay, 5 rework, 1 ordering), every single one was the highest-magnitude divergence point in its session. The mean rank is 1.00 across all four kinds. A user who only opens the top recommendation in the dashboard of §3.5 therefore always sees the deliberately bad behaviour the playbook injected, and no useful alternative of another kind masks the injection. The single ordering catch is session 022, whose ordering-injection magnitude flips from 0 to +1 once the lag term is in the formula – the same lever that drives the $4 \to 10$ shift in positive-magnitude ordering windows reported in §4.1.3. The remaining four ordering injections still surface with magnitude zero or below, consistent with the synthesiser's reach on those windows being limited by the `splitOnInvalid` validator check rather than by score-formula insensitivity (§4.2.3). The prominence number is partly a property of the controlled-injection design, since the playbook injects one dominant bad behaviour per session and that behaviour is structurally likely to be the largest divergence point; the prominence number on uncontrolled real-world sessions would almost certainly be lower. The participants' sessions described in §4.3.1 deliberately omit pre-declared expected kinds, so no prominence number is reported for them; the per-session per-kind distribution reported there is the closest external check on the prominence claim.

injected kind	n caught	@ rank 1	mean rank
Hygiene	8	8	1.00
IDE-Replay	12	12	1.00
Ordering	1	1	1.00
Rework	5	5	1.00

Table 4.11: Injection prominence: when an injection's kind is caught, where does the corresponding divergence point sit in the session's per-magnitude ranking? Top-1 on every catch across all four kinds. The single ordering catch is session 022, surfaced once W_{LAG} entered the formula.

Caveats. Three limitations qualify the divergence numbers. The session set is 45 hand-recorded sessions, so per-cell counts are small (only 5 ordering injections, for example); the per-kind ratios are directionally correct but not enough to support tight confidence intervals, which is why raw counts are reported alongside percentages throughout. The detector reports each divergence point against the user's recorded step index, but step boundaries are a function of the plugin's edit-burst-flush cadence and are not semantically stable across recordings of the same logical activity; recall is therefore reported under the looser anywhere-in-session policy rather than against the manifest's `target_step` label. The "beats user" computation uses a strict magnitude > 0 threshold against trajectory-final J, so a positive magnitude genuinely means the alternative scores higher under the production formula; hygiene commit-gap divergence points have a discrete magnitude exactly equal to W_{CG} because the commit-flip alt drops the gap count by one at trajectory-final. The 194 enumeration count for ordering is the synthesiser's reach across the session set rather than the user-facing number, which is the 24 surfaced divergence points.

4.2.3 Scoped-out fix: ordering recall

One known weakness in the divergence numbers is deliberately left open. Ordering recall sits at 0.36 on any-expected (equivalently 0.40 on injection-only). The cause is the `splitOnInvalid` validator check of §3.3.2: any reorder window containing a step the synthesiser engine cannot reproduce (one of the three non-valid verdicts defined there) is collapsed into singletons and contributes no permutations. The gap is left open because §4.1.3 has already shown that the score formula is insensitive to commuting reorders. If the validator check were relaxed, more reorder windows would be admitted, but every newly-admitted window would (by the same commuting-refactor argument) carry magnitude zero. Recall would rise, the fraction of windows that beat the user would fall by the same amount, and the substantive finding of §4.1.3 would be reinforced rather than overturned. Relaxing the check is therefore a cosmetic improvement to the recall number that strengthens the headline claim of §4.1.3 rather than changing it.

4.3 User and agent studies

§4.1 and §4.2 test the score formula and the detector in isolation; this section tests the tool end-to-end. The end-to-end question is whether the feedback the dashboard surfaces - divergence points ranked by score-delta magnitude, with concrete alternative trajectories attached - actually changes behaviour for users who did not build the tool. §4.3.1 reports a 12-session user study with two human participants on a new fixture. §4.3.2 reports a 6-session comparison run on the same fixture by an automated coding agent, included as a contrastive observation rather than a co-equal study.

4.3.1 User study

The 12-session user study steps outside the controlled-injection sessions to two new questions. First, on sessions recorded by independent participants on a different codebase and without the playbook's controlled-injection structure, does the detector's per-kind output remain coherent? Second, do per-session divergence-point rates evolve as participants accumulate tool feedback across a multi-session arc? The two participants, referred to throughout as P1 and P2, were recruited as senior MEng Computing students at Imperial College London. Both performed six recorded refactoring sessions each on a new order-processing fixture (`user-study-fixture/`) structurally disjoint from the library fixture used in §4.2.2. Both also produced an independent labelling of the 45-session set before participating; that work feeds the inter-rater agreement reported in §4.2.1 and is not repeated here.

Descriptive findings on the participants' sessions. P1 and P2 each performed six recorded refactoring sessions on the order-processing fixture: 12 sessions in total. The playbook gives each session a what-not-how prompt naming the area of code to address but never the IntelliJ action to use, the testing or commit cadence to follow, or the divergence kinds the detector surfaces.[1] Because the playbook deliberately omits pre-declared `expected_kinds`, the participants' sessions admits no precision/recall claim of the form reported in §4.2.2; what it does admit is a descriptive per-kind distribution and a descriptive per-session count, which Table 4.12 gives in full.

session	*IDE-Replay*	*Rework*	*Hygiene*	*Ordering*	total
P1 S1	5	0	1	1	7
P1 S2	0	0	3	0	3
P1 S3	5	2	1	0	8
P1 S4	0	0	0	0	0
P1 S5	1	0	1	0	2
P1 S6	0	0	0	0	0
P2 S1	3	0	2	0	5
P2 S2	2	0	4	0	6
P2 S3	1	0	2	1	4
P2 S4	0	0	0	0	0
P2 S5	2	0	1	0	3
P2 S6	1	3	1	0	5
total	**20**	**5**	**16**	**2**	**43**

Table 4.12: Per-session divergence-point counts across the 12-session participants' sessions. Both participants registered 0 *divergence points on session S04, a cross-class duplication consolidation task that both executed as straight manual edits; the remaining sessions exercise all four kinds in proportions broadly consistent with the 45-session set.*

The aggregate per-kind distribution across the 43 divergence points is IDE-replay 46.5%, hygiene 37.2%, rework 11.6%, ordering 4.7%. Both participants registered 0 divergence points on session S04, the cross-class validation consolidation prompt. The play-

[1] Quotes throughout this section are paraphrased from session-end notes taken during the post-study interview; full audio transcription was not feasible within the study budget. The source notes are retained as private study artefacts and are not reproduced here.

book for that session asks the participant to consolidate three near-identical validation blocks into a single home - a delete-an-inline-block plus delete-a-dead-method task. Both participants executed it as straight manual edits with no rework, no skipped tests, no manual-where-IDE-could-replay candidate, and no reorder window, so the detector legitimately had nothing to surface. Used cautiously, this is the closest the chapter comes to an external true-negative observation: when the task can be discharged in a couple of clean edits and the participants do so, the detector stays quiet. Both participants identified IDE-replay as the most actionable kind in interview, with one participant noting that they "didn't realise that IntelliJ could do all of these refactorings" so didn't use them; both identified ordering as the least actionable, with one asking "how can I actually apply that in the future without knowing".

Behavioural trajectory across the six-session arc. A second question from the participants' sessions is whether per-kind divergence-point rates evolve as participants accumulate feedback. Table 4.13 aggregates per-kind divergence-point counts across the two participants, session by session. Two kinds show trajectory signal. The IDE-replay aggregate falls from 8 in S1 to 1 in S6, an 88% reduction with a mid-arc spike in S3; per-participant, P1 falls from 5 to 0 and P2 falls from 3 to 1, both trending down across the arc. The hygiene aggregate peaks at 7 in S2 - consistent with both participants first seeing commit-cadence and test-cadence feedback at the end of S1 and applying it from S3 onwards - and then falls to 1 in S6. Both participants self-reported the corresponding behavioural change in interview: one said they "ran tests after every change and committed more and used the provided automated tools", and the other said they "used the IDE refactoring methods more frequently". The IDE-replay and hygiene patterns align with what the participants said they did, but with only $n = 2$ participants over six sessions, this is a descriptive observation rather than evidence of a causal effect.

kind	**S1**	**S2**	**S3**	**S4**	**S5**	**S6**	**Δ**
IDE-Replay	8	2	6	0	3	1	−7
Hygiene	3	7	3	0	2	1	−2
Ordering	1	0	1	0	0	0	−1
Rework	0	0	2	0	0	3	+3

Table 4.13: Per-kind divergence-point trajectory across the six-session arc, summed across both participants. IDE-replay and hygiene show a downward trajectory consistent with self-reported behavioural change; ordering and rework are episodic, firing on the sessions whose prompts inherently involve them.

The remaining two kinds do not show trajectory signal. Ordering fires rarely on the participants' sessions - 2 positive-magnitude divergence points in total across the two participants and six sessions each, where the synthesiser found a permutation that strictly beats the user's order - consistent with the headline finding of §4.1.3 that the score formula is insensitive to commuting reorders and that positive-magnitude reorder alternatives are concentrated on the small fraction of windows containing genuinely non-commuting pairs. Rework is episodic, spiking on the participants' hardest sessions where contiguous undo/redo cycles drive several flags from a single sequence of mis-clicks. One participant raised this as a specific metric-design complaint in interview, observing that rework "picked up on undos + redos a lot (in a bad way) [-] over penalised" on contiguous-step sequences caused by mistyped keys. The Table 4.13 pattern is consistent with that account: the 3 rework flags concentrated in S6 cluster within the single most complex session of the arc, where P2 produced all of those flags from a contiguous sequence of small edits.

The S3 and S6 rework spikes are a reminder that the six sessions are not interchangeable: S2 (magic-number renames) and S4 (cross-class consolidation) are short, mechanical sessions where divergence-point counts collapse for both participants regardless of where they sit in the arc, while S3 and S6 are structurally harder sessions where the opportunity for rework is built into the playbook prompt. Cross-session comparisons of raw divergence-point counts or of the within-session process score are therefore confounded by task difficulty, and the per-kind columns of Table 4.13 are the cleaner trajectory signal: IDE-replay and hygiene fall across the arc on the kinds where behavioural change was possible, while rework rises on the hard sessions for both participants. The same confound applies similarly to the agent-comparison reading below.

Process scores under the production and gain-stripped weightings. The within-session process score itself can be partially separated from task difficulty by inspecting it twice: once under the production weights, and once under a weighting that sets the cleanliness-gain weight W_g to zero. The cleanliness-gain term is the only term in the score formula that scales with how much the code itself improved over the session, and the ablation in §4.1.2 already showed that this term acts as a regularizer on divergence-point ranking rather than reshaping it. W_g is the largest weight in the production formula by design, since the anti-gaming axiom of §3.2.1 requires the cleanliness reward to outweigh any process-side shortcut a developer could plausibly take by trading off code quality for procedural discipline; zeroing W_g in a free-form deployment would invite exactly that gaming. The participants' sessions are not free-form: the playbook fixes the structural change each session must produce, so the cleanliness outcome is set by whether the participant completed the prompt rather than by anything the participant could trade off, and the gain-stripped view is a legitimate read on the process-discipline terms alone for this experiment. Zeroing W_g leaves the broken-time, skipped-tests, manual-when-IDE, length-bonus, and commit-gap terms in place, so the remaining score asks "did the user execute the process well" rather than "did the code end up cleaner". Table 4.14 reports both views across the six-session arc for P1, P2, and the agent.

actor	S1	S2	S3	S4	S5	S6	mean
P1 (production)	25	78	18	97	56	57	55.2
P1 (gain = 0)	25	28	35	47	40	50	37.5
P2 (production)	16	68	35	97	31	31	46.3
P2 (gain = 0)	16	18	31	47	31	40	30.5
Agent (production)	0	79	30	100	72	39	53.3
Agent (gain = 0)	37	40	40	50	42	35	40.7

Table 4.14: Trajectory-final process score across the six-session arc for each actor under the production weighting and under a copy of the weighting with the cleanliness-gain weight W_g set to zero. The production view reads the actor's outcome and process together; the gain-stripped view reads the process alone.

Two patterns sit in the table. First, the shape of the production trajectory is structurally similar across all three actors: S2 and S4 are the high points, S1 and S3 are the low points, S5 and S6 land in between. This is the task-difficulty signature. S2 (magic-number extraction) and S4 (cross-class duplication consolidation) reward the actor with a large positive cleanliness delta for a small number of edits; S1 (renames) and S3 (within-class duplication) demand careful intermediate-state management and offer less cleanliness gain to compensate. The agent and the participants land on the same shape because the shape is set by the playbook prompts, not by how well the actor executed them.

Second, the gain-stripped trajectory shows a pattern that the production trajectory hides. P1's process score under $W_g = 0$ rises nearly monotonically across the arc, from 25 at S1 to 50 at S6, with one -7 dip at S5. S6 sits at the baseline ceiling of 50, meaning P1 incurred essentially no process-discipline penalty on the structurally hardest session of the arc. P2 trends the same direction less consistently: $16 \rightarrow 18 \rightarrow 31 \rightarrow 47 \rightarrow 31 \rightarrow 40$. The agent's gain-stripped scores show no arc-position trend at all: $37, 40, 40, 50, 42, 35$, varying with task structure rather than session number. The playbook orders the six sessions in roughly increasing structural complexity (S1 renames through to S6 multi-step reshape), so the participants' improvement is happening despite progressively harder tasks, not riding on easier ones. The agent shows no such trend, suggesting that the qualitative between-session course-corrections discussed in §4.3.2, Thread 1 do not compound into a monotonic improvement on the process-discipline subscore over the arc.

The direction of the participants' improvement is what the tool's feedback is designed to produce, but the design has no control condition (no participant performed the arc without dashboard feedback between sessions) and $n = 2$ participants is too small to support a causal claim. The gain-stripped subscore is largely insulated from how familiar the participant is with the codebase itself - the terms active under $W_g = 0$ are commit cadence, skipped tests, and manual-when-IDE rate, none of which depend on knowing the code better. What the design does not separate is the participants intuiting what the dashboard rewards and adjusting behaviour to satisfy it (a score-gaming reading), or becoming more comfortable with the playbook idiom across the arc (a task-style-familiarity reading). The threats-to-validity chapter (§5.5) discusses these. What the table does support is a narrower claim: the production score's session-level value is dominated by the cleanliness-gain term, so the production view alone does not show what changes in the participants' process across the arc, and the gain-stripped view does.

One additional qualitative finding does not show in the trajectory table but matters for future tuning: hygiene also produces a false-positive shape, firing when a participant is thinking rather than stuck. One participant flagged this in the interview as was "penalised for not running tests when wasn't quite finished but just was thinking", and the pattern is reflected in the S2 peak for hygiene more generally. Both this finding and the rework over-penalisation are reported here as candidate metric-tuning directions rather than detector bugs: the divergence points are accurate readings of what the trajectory contains, but the metric design treats some legitimate-pause and mistyped-key cases more harshly than the participant would.

Per-step inspection as a usability win. Both participants singled out the dashboard's per-checkpoint detail panel as a positive part of the interface, independently of the four detector kinds. One participant said the per-checkpoint view made it easy to "see what action you made to cause the issues to happen", and that the trajectory graph showing the highest score reachable from each point onwards was "egging you on to be a better person" (i.e. the alternative-trajectory overlays were read as constructive rather than punitive). The other participant said they could "see the specific point you went wrong", valued the cleanliness-signal "breakdowns" on each step, and called the dashboard "easy to track session at specific points and code smells". These observations validate the per-step smell-attribution and metric-shift breakdown described in §3.2.6 as a usability feature the participants engaged with, not just an architectural by-product of how the report is assembled.

4.3.2 Agent comparison

An automated coding agent - Claude Code (Anthropic, model `claude-opus-4-7`, run on 2026-05-21) - performed the same six-session playbook on the same fixture. Between sessions, the human facilitator pasted the markdown output of the `feedbackForAgent` task (`tool/analysis/.../experiment/FeedbackForAgent.kt`) into the agent's prompt at the start of the next session. The task renders an analysis report's divergence points and trajectory advice into the same plain-text summary that a human participant would read on the dashboard, so the agent received the same information the human participants did, just in textual rather than rendered form. The agent honoured a small setup contract before any session ran: tests were invoked via a wrapper script that brackets the gradle call with `TEST_RUN_STARTED` / `TEST_RUN_FINISHED` events in the plugin trace, and commits were made via standard `git commit` which the plugin's reflog watcher captured as `GIT_COMMIT` events. Apart from starting and stopping the plugin's recording at session boundaries and pasting the previous session's feedback into the next prompt, no human intervened during a session.

Table 4.15 reports the agent's per-kind divergence-point totals against each participant's six-session total from Table 4.13. The agent produced 5 divergence points across six sessions, against 20 from P1 and 23 from P2 over the same six sessions each - but the shape of the comparison is the headline, not the count. All five of the agent's divergence points are IDE-replay; rework, hygiene, and ordering fired zero times across the entire six-session arc. Both participants exercise all four kinds.

kind	**Agent**	**P1**	**P2**	**Agent / P1**	**Agent / P2**
IDE-Replay	5	11	9	0.45	0.56
Rework	0	2	3	0.00	0.00
Hygiene	0	6	10	0.00	0.00
Ordering	0	1	1	0.00	0.00
total	**5**	**20**	**23**	**0.25**	**0.22**

Table 4.15: Per-kind divergence-point totals for the agent over six sessions on the user-study fixture, against each participant's six-session total separately so the comparison is run on the same session count for each row.

Table 4.16 mirrors the human trajectory table for direct visual comparison. The agent's divergence-point counts session by session are $0, 0, 1, 0, 2, 2$, and the final-checkpoint process scores are $0, 79, 30, 100, 72, 39$. Neither row shows monotonic improvement, for the same task-difficulty reason noted earlier in this section: S2 and S4 are short, mechanical sessions where everyone - both participants and the agent alike - produces near-zero divergence points and runs the score close to the ceiling; S3 and S6 are structurally harder sessions where the human cohort's rework count spikes as well. The cleaner trajectory signal is the per-kind columns rather than the per-session score endpoints.

kind	**S1**	**S2**	**S3**	**S4**	**S5**	**S6**	**Δ**
IDE-Replay	0	0	1	0	2	2	+2
Rework	0	0	0	0	0	0	0
Hygiene	0	0	0	0	0	0	0
Ordering	0	0	0	0	0	0	0
final score	0	79	30	100	72	39	-

Table 4.16: Agent per-session divergence-point trajectory and final-checkpoint process scores, mirroring Table 4.13 for the human cohort.

Reading the per-kind columns left-to-right tells a different story from the score row. Rework, hygiene, and ordering held at zero across all six sessions, consistent with the procedural-discipline contract written into the agent's setup prompt being honoured throughout: tests via the wrapper after every change, commits at the boundaries the playbook prescribes, no undo cycles. The S1 dashboard did surface an admitted-but-not-improving ordering window (a permutation existed that reached the same end state but did not strictly beat the agent's; under the strict magnitude > 0 filter of §3.3.2 this is informational rather than a divergence point), and the agent acknowledged it in writing after the session; the kind never re-surfaced, but no controlled-comparison claim is made on a window whose magnitude is non-positive.

Behavioural threads from the agent's own reflections. The aggregate trajectory table flattens what is, on inspection, a substantively richer story. The full transcript reproduced in Appendix A.1 shows the agent reading each session's feedback, diagnosing its own anti-patterns in writing, and adjusting approach between sessions in ways the per-kind counts only partially capture. Three threads stand out.

Thread 1: commit cadence, over-corrected and then resolved. S1 produced the worst-scoring session of the arc: one bulk commit at the end of all three renames, a session-long stretch without committing, a 28% build-broken rate (the build was broken on 2 of 7 checkpoints), one test regression, and a process score that fell from 50 to 0 - the largest single-session drop on the arc. The post-session feedback fired three advice items at critical and warning levels. The agent's response, written before S2 began, was a direct procedural correction: "commit smaller and more often - one commit per rename (or per logical step), not one at the end. Test after each step." S2 implemented exactly this. The agent did three steps as three separate `git commit` calls, each preceded by `./agent-test`; the resulting session produced 0 divergence points, 0 advice items, and the process score climbed from 50 to 79 - a +79-point swing across one session boundary attributable to a single concrete procedural change.

The agent then over-corrected in the opposite direction. Its pre-S3 statement was "I'll aim to commit even more granularly (each individual constant/edit, not each step)", and S3 fired three IDE-replay divergence points (one at magnitude -6), `REFACTORING_INTRODUCED_SMELLS`, and a process score drop from 50 to 30. By S6 the agent had revised the rule a second time, but in a more interesting direction: the S6 advice flagged an 8-checkpoint commit stretch and a 37% build-broken rate (3 of 8 checkpoints), and the agent's diagnosis identified the cause as a structural tension the rule itself created: "for an Extract Method, do it as a single atomic edit (insert new method first, then replace call site) so the build never breaks mid-step." The agent had discovered, by being penalised for it twice, that "commit more often" and "don't break the build" are in conflict when an edit is structurally multi-step, and resolved the conflict by switching to larger atomic diffs rather than finer-grained commits. This is the most substantive behavioural-trajectory finding in the arc: the agent did not just reduce its DP count, it revised its operational model of the trade-off the advice was implicitly pointing at. Placing the broken-build percentages alongside the human cohort's puts the size of the issue in context: across the arc the agent's per-session broken-build percentage (broken wall-clock time over total session wall-clock time) averages 6.5% with a peak of 24% on S6, while P1 averages 7.3% with a peak of 23% on S1 and P2 averages 14.0% with a peak of 52% on S1. The agent is comparable to P1 on average but P2's worst session is more than twice the agent's worst. These per-session percentages are what surface as the `BUILD_OFTEN_BROKEN` dashboard advice when they cross the warning or critical thresholds; the agent's wrapper script forces a test run at every committed checkpoint but does not prevent intermediate broken states between checkpoints, which is exactly the failure mode the atomic-edit rule above addresses.

Thread 2: smell-introduction feedback produced a specific self-critique. After S3 the agent fired `REFACTORING_INTRODUCED_SMELLS` alongside the process-score drop, and the agent's written response identified its own anti-pattern by name: "methods like `chargeCash`/`refundVoucher` are arguably trivial wrappers that didn't earn their keep," and "could have made `formatAddress` cleaner." The follow-on sessions support the diagnosis: S4 produced zero divergence points and zero advice items, S5 produced two IDE-replay flags but no `REFACTORING_INTRODUCED_SMELLS`, and the smell-introduction advice did not fire again until S6 - where the agent attributed it to multi-edit extract-method mechanics rather than the trivial-wrapper anti-pattern from S3. The smell-introduction feedback drove a course correction that holds for two consecutive clean sessions before resurfacing under a different cause.

Thread 3: agent acknowledges the structural IDE-replay limitation. The persistent failure mode is IDE-replay. All five of the agent's divergence points are IDE-replay flags, and the count does not decline across the arc; it is concentrated on the structurally heavier sessions (S3, S5, S6) where the agent performed enough structural refactoring to surface any divergence points at all. The agent has no IDE refactor surface to switch to: Claude Code edits files via text tool calls, not through IntelliJ's Refactor menu, so every refactoring it performs is classified as a manual edit by the detector. The agent itself acknowledges the structural nature of this in its S6 reflection, observing that "the IDE-replay flags persist - manual extracts consistently underperform the IntelliJ refactoring; in a real run I'd use the Extract Method action." The signal is real and the recommendation correct, but the agent lacks the tool to execute it. The agent cannot drive the IDE-replay count toward zero however well it works, and this is the cleanest example in the arc of feedback being correctly interpreted but structurally inactionable. A setup that exposed IDE refactor actions as agent tool calls would close the gap - that work is named in the future-work section.

The two failure profiles are nearly complementary. Humans are procedurally noisy and architecturally capable: high IDE-replay and hygiene counts, scattered rework on the hard sessions, but able to switch to IntelliJ refactor actions when they remember to. The agent is procedurally disciplined and architecturally limited: zero on rework, hygiene, and ordering - the three kinds where the tool measures process cadence and sequencing - and IDE-replay only on the sessions where it performed structural refactoring at all, because the actuator that would resolve IDE-replay does not exist in its interface. The tool surfaces both kinds of failure correctly; neither profile is the "right" one to target, and the comparison demonstrates that the detector's four kinds are pulling apart genuinely different dimensions of refactoring practice rather than collapsing onto a single dimension.

The agent comparison is descriptive at $n = 1$ agent over six sessions with one model version on one date; it is illustrative of the detector's behaviour on a non-human actor rather than a generalisable claim about coding agents. A multi-agent extension, and an agent-side IDE refactor capability that would let the IDE-replay count actually respond to feedback, are flagged as future work.

Chapter 5

Threats to validity

This chapter collects the limitations mentioned throughout Chapter 4 and discusses design choices that affect the validity of our claims. The discussion follows the standard four-axis framework from empirical software engineering [69]: construct validity, internal validity, external validity, and statistical conclusion validity. A per-experiment section then identifies threats specific to each of the four empirical studies (the kind-classifier, the user study, the agent comparison, and the rater study), followed by implementation-side concerns and an out-of-scope list. The mitigations the design already carries (three-way inter-rater reliability on the injection sessions, deterministic ranking tie-breaks, multi-knob robustness alongside the single-knob sweep, honest score-floor and validator-check disclosures, reproducible notebook sources for every reported number, and an explicit per-session-not-per-step true-positive policy) are stated where the corresponding design choice or result is presented, rather than restated here.

5.1 Construct validity

There is no external ground truth for "true refactoring quality". The process-quality score J measures process behaviours the literature identifies as bad practice: leaving the build broken, going long stretches without committing, refactoring manually where an IDE refactor would have worked, and so on. It does not predict bug rates, time-to-merge, or any other downstream-of-refactoring outcome, and Chapter 3 does not claim it does. Calibrating a formula like this against a held-out outcome dataset would require a dataset linking session-level process signals to those outcomes, and we are not aware of one that exists. The weights are grounded in the literature-supported severity orderings of §3.2.2 rather than fitted to data; the chapter addresses robustness and inter-rater agreement empirically, but not predictive validity.

The score is not a maintainability index. Snapshot composites like the Maintainability Index [55] were regression-fitted against expert ratings of completed code. Our score is trajectory-level and process-side; it asks "did the refactoring follow good practice on the way there", not "is the code good now". The two are different constructs and we do not conflate them.

The four-kind taxonomy is a methodology choice, not a discovered structure. The divergence-point kinds (*IDE-Replay*, *Rework*, *Hygiene*, *Ordering*) were chosen for the rater-design rationale set out in §3.6: they are operationally distinguishable behaviours the literature flags as bad practice, and they admit a synthesiser that can produce a concrete alternative for each. A different taxonomy is defensible. Collapsing *Hygiene* into separate top-level *Tests-Skipped* and *Commit-Gap* kinds, or merging *Rework* into *Ordering* on the grounds that both are about wasted steps, would change the precision/recall numbers without changing the chapter's headline claims.

`SuboptimalOrdering` is author-claimed, not measured. The five ordering-injection sessions in the playbook carry the `SuboptimalOrdering` pattern label because the playbook author selected an ordering they believed was worse than its alternatives. The score formula cannot independently verify all five: §4.1.3 reports that 1 of the 5 "suboptimal" orderings produces a positive-magnitude reorder alternative under the production formula (which is the largest single signal the lag term W_{LAG} produces on the injection set), while the other 4 remain at magnitude zero because their permutations commute and the synthesiser cannot reach an AST-distinct end state. The label reflects what we designed the session to contain rather than objective ground truth, so interpret the precision/recall numbers with that in mind.

The AST-equivalence audit uses an informal definition. A candidate permutation is accepted by the reorder synthesiser only if its terminal AST is equivalent to the user's. Equivalence is defined as "modulo syntactic differences that do not change behaviour" (§3.3.2) and implemented using an AST hash. The exact set of differences the hash treats as behaviour-preserving (e.g. comment placement, annotation order, whitespace) is implicit in the hash implementation rather than spelled out here. A hash that is too loose would admit reorders that change behaviour and a hash that is too tight would reject reorders that are genuinely equivalent. It catches the obvious failures, but the exact boundary is a design choice we do not formally specify.

5.2 Internal validity

The single-knob sensitivity sweep is combined with a multi-knob design rather than replaced. The headline top-1 stability number in §4.1.1 is computed under single-knob perturbation: one weight changes per row, the rest stay at production. Real misspecification of weights rarely happens one at a time, so a multi-knob Monte Carlo experiment runs alongside (every weight perturbed simultaneously from $\exp(\sigma \cdot \mathcal{N}(0,1))$ with $\sigma = \ln 2$). Top-1 stability falls from 96.1% under single-knob to 81.3% under multi-knob on the user-study sessions. The formula is robust to plausibly-sized perturbations of any single weight, and robust to roughly four-in-five plausibly-sized simultaneous perturbations of all weights, but is not robust to arbitrarily-large simultaneous moves.

Kind-presence agreement does not imply rank-order agreement. The Cohen's κ reported in §4.2.1 establishes that three independent labellers agree on which divergence kinds fire in each session. It does not establish that they agree on the order the score puts those divergence points in, which is what the dashboard surfaces first to the user. Closing this gap would require a separate rater-rank study (engage raters to rank the divergence points within each session, compute Kendall τ-b against the score's production ranking). This was out of scope for our project. Our strongest claim from κ alone is that the score's kind labels match what the rater identifies, but the ranking that orders those labels is not externally validated.

Cleanliness sub-weights are uniform 1.0 by Laplace's principle of insufficient reason. The six sub-signals that feed the cleanliness sub-score (§3.2.3) are weighted equally because no published calibration data exists for weighting these signals across refactoring trajectories. The sensitivity sweep shows the cleanliness sub-weights have essentially no effect on the top of the divergence-point ranking on the user-study set (Table 4.2: five of six sub-knobs produce zero top-1 disruptions, and `cohesion` produces a single disruption attributable to the lag-denominator coupling). On the agent corpus the cleanliness sub-weights' Kendall τ drops by 3–10 % under $\times 0.5/\times 1.5$ perturbation – the same coupling running through a smaller corpus where one shifted rank dominates. The uniform choice is defensible in practice on the current sessions but it is not a positive calibration result.

The lag term couples the cleanliness sub-score to the process score. The lag term $W_{\text{LAG}}\,\ell(\tau)$ uses the trajectory's final cleanliness scalar $C(s_T)$ as the reference against which intermediate-state deficits are measured (§3.2.3). This means a perturbation of any cleanliness sub-weight propagates into the process score through both the endpoint-gain term and the lag denominator, instead of only through the endpoint. The agent-corpus sensitivity result above is the empirical signature of this coupling. The injection and user corpora absorb the same leakage without measurable rank disruption because their per-fixture rankings are longer and more diverse, but the underlying coupling is the same. A cleanliness composite that mis-weights one sub-signal could in principle shift the lag-driven part of the ranking by a small amount we cannot disentangle from the gain-driven part. We treat this as an acceptable design cost: the lag term is meaningful only in the cleanliness-scalar units used elsewhere, and detaching it would introduce a second scalar with no independent calibration anchor.

Two batching primitives are heuristic and not sensitivity-tested. The score formula and the divergence-point detector share two batching constants: `min_commit_gap` = 6 (the green-refactor checkpoint count at which a *Commit-Gap* divergence point fires and the score formula applies the commit-gap penalty), and the 60-second composite-window boundary that bundles refactoring steps into batches for *Tests-Skipped* and *Ordering* detection. Both come from Murphy-Hill et al.'s 2009 study of how developers actually refactor [5], and neither is sensitivity-tested directly. The sensitivity sweep perturbs the weights on these terms but not the thresholds themselves. A different typing speed (slower or faster than the participants in this study) would group steps differently and could produce a different mix of fired kinds.

The production weight vector is hand-picked within a literature-grounded ordering. The exact production weights (§3.2.2, Table 3.2) sit within the severity ordering the literature supports ($W_b > W_{st} > W_{cg}$ etc.), but no calibration step pins them to those specific numbers rather than any other vector inside the same ordering. The robustness experiments establish that the formula sits in a locally stable region around this vector; they do not establish that no other defensible vector exists in the same region.

Which terms carry the ranking varies by session type. The ablation analysis on the injection sessions identifies `broken`, `length` and `commitGap` as the dominant rank-carriers (Table 4.6: removing each drops Kendall τ to between 0.887 and 0.930). On the user-study sessions the dominant carrier is `commitGap` (τ = 0.599 on removal), with `manualIde` and `skipTests` next. The new lag term has near-zero ranking impact on the user-study set (τ = 0.992) but contributes magnitude on the injection set, so its rank-shaping role is narrower than its magnitude role. The formula's effective parameter dimension therefore depends on what kind of session is being scored. This connects to the gain-stripped finding in §4.3.1: the cleanliness-gain term dominates the absolute score, but doesn't affect the ranking of divergence points, so the formula effectively has two modes (an outcome-side mode that drives the absolute level and a process-side mode that drives the ranking). Whether a weight matters depends on which mode is being read - the absolute score or the ranking.

The reorder dependency analysis is deliberately coarse, and anchors are content-addressed. The dependency analyser that decides which refactoring steps can be safely reordered uses two design choices that affect the *Ordering* recall reported in §4.2.2. First, `Extract*` and `InlineMethod` specs are modelled optimistically: two extracts on the same host method, or two inlines of distinct methods on the same class, are not auto-conflicted, on the grounds that the downstream AST-equivalence check will reject any reorderings that conflict in practice. Second, the four specs that rewrite call sites of a named method (`RenameMethod`, `ChangeMethodSignature`, `ParameterizeVariable`, `ParameterizeAttribute`) are modelled coarsely: each writes the entire declaring class, which chains every later operation on the same class after it by construction, even when the operations would have been genuinely independent. Other rename specs (`RenameClass`, `RenameField`, `RenameLocalVariable`) only consume and produce the specific named entity rather than writing the whole class. On top of this, the entities that anchor each refactoring (a specific region of code, a specific local declaration) are identified by AST subtree hashes, so an identifier rename inside an anchor's region between two reorderable steps invalidates the second step's anchor. The optimistic model over-enumerates but the AST-equivalence check filters out the resulting false positives; the coarse model under-enumerates and removes some legitimate independence by construction. The 0.36 *Ordering* recall number reflects these two design choices. The headline §4.1.3 finding ("reordering rarely improves the score under the production formula") is robust to them, because that finding follows from commuting refactorings producing identical end states under the formula, which does not depend on how the dependency analyser decides what commutes.

Cleanliness sub-signals are clamped to the user's range on alternative trajectories. Alt-trajectory cleanliness sub-signals are min-max normalised against the main (user) trajectory's observed range, then clamped to $[0, 1]$. An alternative whose intermediate checkpoints sit outside the user's range is masked at the boundary. We mention the clamp in §3.2.3 but do not flag it as a threat there; it is named here for completeness. The clamp does not affect the divergence-point magnitudes we report because the validator requires every accepted alternative to reach the same terminal AST as the user's trajectory, so alt and user share the same terminal-state cleanliness by construction; the trajectory-final J computed from $\Delta C = C(s_T) - C(s_0)$ is identical on both. The clamp only changes how alt intermediate checkpoints are displayed on the dashboard's cleanliness chart, which has no downstream effect on the scoring numbers.

5.3 External validity

The injection sessions are scoped exercises, not naturalistic sessions. The 45 sessions in the primary test set were recorded by us against a scripted playbook that injects one specific bad behaviour per session. They are scoped exercises rather than naturalistic refactoring activity; precision and recall claims drawn from this corpus are conditional on that controlled-injection structure, not on the variety of session shapes a free-form developer would produce in practice.

The user study has two participants and twelve sessions. This is the smallest meaningful sample for a multi-recorder study. The chapter shows each participant's trajectory and divergence-point counts, but does not report statistical tests, confidence intervals, or claims like "the median participant improved on metric X". The improvement in procedural discipline visible in the gain-stripped trajectory is real and matches what the feedback is designed to produce; the gain-stripped subscore is constructed precisely to isolate process behaviours that are not a function of how familiar the participant is with the codebase. The threats that remain after that decomposition - demand characteristics and task-style familiarity - are discussed under per-experiment threats in §5.5; the $n = 2$ sample size limits the strength of any claim drawn from the trajectory regardless.

The agent comparison is a single agent over six sessions. This comparison uses a single agent (Claude Code, model `claude-opus-4-7`, tested on 2026-05-21). The comparison portrays the detector's behaviour on this particular coding agent rather than a generalisable claim about coding agents. A multi-agent extension is named as future work in §5.7.

All sessions are single-author IntelliJ sessions on a Gradle Java project. The detector's behaviour on team pull-request-driven workflows, on other IDEs, or on other languages is not studied. The plugin's instrumentation is IntelliJ-specific: it uses the IDE's PSI and VFS hooks to capture refactoring events, which have no direct equivalent in VSCode or Eclipse, so a port would require re-implementing the event-capture layer rather than just retargeting a build.

Refactoring kinds the synthesiser cannot handle are excluded by construction. The *IDE-Replay* synthesiser builds alternative trajectories by replaying refactoring specs through Eclipse JDT. A user trajectory built entirely from refactoring operations outside JDT's supported set (or outside RefactoringMiner's detection capability, which feeds the synthesiser) would produce no *IDE-Replay* divergence points. The detector cannot flag what it cannot synthesise an alternative for. This is a structural limitation of the detection design, not a finding specific to the sessions we used.

5.4 Statistical conclusion validity

Cohen's κ in §4.2.1 is reported as a point estimate with raw cell counts. Confidence intervals are not reported because with only 45 sessions per rater pair, they would be too wide to be useful and would overshadow the κ value itself. Instead, the specific disagreement cases are listed (sessions 024 and 043 for *Rework*; sessions 014, 015, 016, 039 for *Hygiene*), which tells the reader more.

Kendall's τ-b is brittle on small per-fixture rankings. A single pair swap on a three-point ranking moves τ by 0.333. τ is reported on all three session sets, but the injection sessions' per-fixture rankings are short enough that the metric is dominated by the small-N artefact; the sweep therefore reports top-1 hit rate alongside τ as a more stable surrogate, and the user-study sessions are treated as the canonical benchmark in §4.1.1.

No multiple-comparison correction is applied across the weight $\times$ factor sweep ($12 \times 9 = 108$ cells per fixture). The headline result is a descriptive statistic - what fraction of cells preserve the top-1 recommendation - not a hypothesis test, so a correction would be appropriate only if individual cells were being interpreted as significance claims, which they are not. The multi-knob Monte Carlo uses a fixed seed and the 200-sample size was chosen heuristically rather than from a power calculation; distribution shape (mean τ, top-N hit rate) is reported rather than a point estimate with confidence interval.

5.5 Per-experiment threats

The kind classifier (§4.2.2). The 1.00 precision figure across all four kinds is conditional on the sessions used. `expected_kinds` is a hand-label computed independently of the playbook's injection pattern, and the inter-rater κ study (§4.2.1) establishes that three labellers agree on it for these sessions. The threat is not that the labels are author-biased; it is that the corpus consists of controlled-injection sessions whose divergence behaviours are bounded by what the playbook injected and what the four detector kinds can describe. A free-form session set could contain bad behaviours outside those four kinds, and even within the four kinds the labelling shows rater variance on this corpus ($\kappa = 0.72$ on *Hygiene* between two of the three pairs, §4.2.1). Precision on a free-form session set is therefore unstudied. Separately, IDE-replay precision was 0.80 before the plugin-instrumentation fix described in §4.2.3: the false positives in the pre-fix recordings were upstream plugin-capture issues, not detector-logic errors, but the current 1.00 figure would not hold on the pre-fix recordings.

The user study (§4.3.1). The external-validity limitations above ($n = 2$, no control) apply here too. The main concern with the gain-stripped score is what it can actually show us. The metrics in the gain-stripped score (commit cadence, skipped tests, manual-when-IDE rate) don't depend much on knowing the codebase, so improvement can't be explained by just getting more familiar with the fixture. But there are two other explanations we can't rule out. First is the Hawthorne effect: after seeing their dashboard in S1, a participant figures out what the system rewards-frequent commits, test runs, IDE actions-and adjusts their behavior to game the score. This looks like improvement, but it's behavior change to fit the metric, not actual skill improvement. We can't tell the two apart from the data. Second is learning the task format: all six sessions follow the same structure (a what-not-how prompt, a small change to make, look at the dashboard), so participants get better at the format itself, not at refactoring. We can't separate either confound from the study design, and we don't claim we can.

The agent comparison (§4.3.2). The agent has no IDE-refactor surface available to it: Claude Code edits files via text tool calls, not through IntelliJ's Refactor menu, so every structural refactoring the agent performs is classified as manual by the detector. The *IDE-Replay* count is therefore structurally bounded above zero whenever the agent performs any structural refactoring at all. This is a genuine measurement limitation, not a finding about the agent's capability. The agent itself acknowledges this in its S6 reflection (Thread 3 of §4.3.2). The behavioral changes described in §4.3.2 are observational only ($n = 1$ agent, no control). We didn't run the agent without feedback, so while the data is consistent with "feedback caused the change," we can't prove it.

The rater study (§4.2.1). Three labellers, one of whom is us. The chapter reports pairwise κ so us-vs-rater values are visible alongside the rater-vs-rater pair, which makes our self-bias detectable rather than hidden. The two participant raters are the same two participants who later performed the user-study sessions, so familiarity with the broader project is not zero; the labels were assigned before any participant performed a user-study session, however, so the labelling cannot have been biased by what the participants later did. Two of the four kinds (*Rework* and *Hygiene*) show $\kappa < 1.0$ with specific disagreement structure named in §4.2.1; reconciliation was deliberately not applied to inflate κ, since the disagreements are themselves a finding about where the kind boundaries are fuzziest.

5.6 Implementation threats

The wrap-and-patch layer of §3.3.3 closes residual gaps between IntelliJ and Eclipse JDT for accepted reorder windows but is not exhaustive: windows where the gap cannot be patched are marked `ast_diverged` and excluded from reordering.

The reorder enumerator carries a hard size budget: any window with more than 7 refactoring steps is skipped entirely, and within an admitted window enumeration stops after $7! = 5040$ permutations (`TopologicalEnumerator.kt`). The budget exists because the cost of enumerating all topological orderings grows factorially with window size, and an 8-step window would already require $\geq$ 40,320 enumerations before the AST-equivalence audit ran on each. In practice the recorded sessions rarely produced reorder windows large enough to exceed this budget, but a session with longer contiguous refactor sequences would have its widest windows silently dropped from the *Ordering* enumeration. A future implementation could replace the exhaustive enumeration with a beam search or heuristic pruning to lift the size cap.

5.7 Out of scope

The following are explicitly outside the scope of our work. We do not claim to address them.

- **Multi-agent agent comparison.** Only Claude Code was run, with one model version on one date. A multi-agent or longitudinal agent study is named as future work in Chapter 6.
- **Longitudinal validation against external code-quality outcomes.** No outcome dataset linking refactoring-trajectory signals to downstream maintenance cost was available, and constructing one was treated as outside our scope.
- **Cross-IDE and cross-language validation.** The plugin is IntelliJ-specific and the refactoring engine is JDT-specific (Java). Ports to other IDEs or languages are named as future work and not attempted.
- **Production-weight refitting against an outcome dataset.** See the outcome-dataset point above.
- **Real-time, live-feedback dashboard mode.** The dashboard is a session-replay view of completed analysis reports. Live feedback during refactoring is flagged in §4.3.1 as a natural next step but is not implemented.
- **Rater-rank validation.** As named in §5.2, the κ study validates kind-presence agreement, not rank-order agreement. A Kendall τ-b study on per-session rankings is the natural follow-up and is not done here.

Chapter 6

Discussion and Conclusions

This chapter consolidates the work of the previous chapters: what was built, what Chapter 4 establishes and what it does not, and which directions are most worth pursuing next.

6.1 Summary of contributions

We produce two main contributions. The first is the process-quality score J, a formula that measures refactoring quality by combining the endpoint cleanliness gain with five penalty terms from the literature – broken builds, skipped tests, manual edits when IDE refactoring would work, long gaps between commits, and an intermediate-cleanliness lag penalty that costs trajectories which linger below their final cleanliness target – and a step-savings bonus that rewards alternatives reaching the same end state in fewer steps. The second is the divergence-point detector, which finds points in a refactoring session where a better alternative exists, classifies them into four kinds (*IDE-Replay*, *Rework*, *Hygiene*, *Ordering*), and produces a concrete alternative trajectory for each. The alternatives are generated by four per-kind synthesisers internal to the detector: a reorder enumerator that finds valid step reorderings and checks them for AST equivalence, an IDE replayer that converts manual edits into IDE refactor calls, a rework stripper that removes self-cancelling step pairs, and a hygiene synthesiser that models what would happen if tests were run or commits made at different points.

Supporting these are: a 45-session hand-recorded corpus on a purpose-built Java fixture with multi-label ground-truth annotations; a 12-session user study and a 6-session agent comparison on a separate fixture; the IntelliJ plugin that captures the event traces; the JCEF dashboard that surfaces the divergence points; and a reproducible notebook from which every number in Chapter 4 can be regenerated.

6.2 Headline findings

Five findings carry our main claims.

First, the lag term $W_{\text{LAG}}\,\ell(\tau)$ is the operative lever for the *Ordering* divergence-kind. Across the 45 injection sessions the reorder synthesiser admitted 194 permutations over 24 windows; 10 of those windows produced a positive-magnitude alternative (41.7%), and a further 2 produced a negative-magnitude alternative (the user front-loaded cleanliness better than the synthesised alts). On the five sessions whose playbook author intended an ordering they believed was worse than its alternatives, 1 of 5 produced a strictly better permutation. Without the lag term in the formula, the corresponding numbers collapse to 4/24 positive-magnitude windows, 0 negative-magnitude verdicts, and 0 ordering injections caught – the lift is mechanically attributable to W_{LAG}, since the rest of the formula was held fixed across the comparison (§4.1.3, §4.2.2).

Second, the kind classifier is precise on the controlled-injection sessions. Per-kind precision is 1.00 across all four kinds against the inter-rater-agreed `expected_kinds` labels, with Cohen's $\kappa = 1.00$ for *Ordering* and *IDE-Replay* on all three rater pairs and $\kappa = 0.72$–1.00 for *Rework* and *Hygiene* (§4.2.1, §4.2.2). Every injection caught with a positive-magnitude alternative trajectory ranked at top-1 within its session.

Third, the score formula sits in a locally stable region around its production weights. Top-1 stability under single-knob perturbation is 96.1% on the user-study sessions. The multi-knob Monte Carlo design (every weight perturbed simultaneously from $\exp(\sigma \cdot \mathcal{N}(0, 1))$ with $\sigma = \ln 2$) lowers this to 81.3% (§4.1.1). The formula is robust to plausibly-sized single-weight changes and to a majority of plausibly-sized simultaneous changes, and degrades smoothly under larger ones.

Fourth, both human participants showed near-monotonic improvement in their process discipline across the user-study arc. Decomposing the process score with the cleanliness-gain weight zeroed isolates terms that do not depend on codebase familiarity. P1's score rises from 25 at S1 to 50 at S6 despite the tasks getting progressively harder; P2 trends in the same direction less consistently (§4.3.1). The agent's gain-stripped score doesn't improve over time; it varies by task difficulty, not session number.

Fifth, the agent (Claude Code, model `claude-opus-4-7`, tested on 2026-05-21) course-corrected between sessions on the procedural-discipline behaviours within its tool surface, and correctly identified the one behaviour that lay outside it. Three threads from the transcript (§4.3.2) show this: commit cadence was revised twice (under-committing in S1, over-correcting in S3, resolved by S6 atomic-edit synthesis); smell-introduction self-diagnosis after S3 prevented recurrence in S4–S5; IDE-replay flags persisted across the arc not because the agent failed to read the feedback but because it edits files via text tool calls and has no IDE refactor surface available to it, a limitation it diagnoses correctly in its own S6 reflection.

6.3 Discussion

The chapter substantiates four claims: a divergence-point classifier whose precision and rank-1 prominence are corpus-conditional but reproducible; a score formula whose ranking is locally robust to weight choices, with each term carrying measurable influence; a descriptive observation that two participants showed gain-stripped process-discipline improvement across the six-session arc after receiving feedback; and an observation that a coding agent adjusted its behaviour between sessions in response to the same written feedback, within the limits of its tool surface.

A few stronger claims are left for follow-up work. The user study and agent comparison are observational rather than controlled, so the process-discipline arc is consistent with the feedback driving it but is not reported here as a causal finding. The plugin and synthesisers are written against IntelliJ and JDT, so generalisation to other IDEs or languages is a port rather than a configuration change. The weights are grounded in literature-supported severity orderings rather than fit against an external outcome dataset, since no dataset linking session-level process signals to downstream maintenance outcomes exists at the granularity the score operates on. Chapter 5 discusses each of these in detail.

6.4 Future work

The suggestions below are ordered by what would most strengthen the main findings, with the scale of the human study the largest single gap:

1. **Larger and controlled human study.** The strongest descriptive claim in the chapter (the gain-stripped process-discipline arc in §4.3.1) rests on two participants without a control condition. A study with more participants and a feedback-on / feedback-off arm would establish causation rather than just describe the effect, and would enable better estimation of between-participant variance.

2. **Multi-agent comparison.** The agent-comparison chapter rests on one agent run with one model version on one date. Extending it to multiple models across multiple dates would separate model-specific behaviour from agent-class behaviour and would let the qualitative threads in §4.3.2 become quantitative.

3. **Rater-rank validation.** The Cohen's κ study validates kind-presence agreement only. A Kendall τ-b study on per-session divergence-point rankings would validate that the score's ordering of divergence points within a session matches a human expert's ranking, which the κ figures alone do not establish.

4. **IDE-refactor-as-agent-tool deployment.** The IDE-replay structural limitation in §4.3.2 is the cleanest example in the chapter of feedback being correctly interpreted but inactionable: the agent acknowledges the IDE refactoring would score better but has no way to invoke it. Exposing IntelliJ's refactor actions as agent tool calls would close that gap and would let the *IDE-Replay* count actually respond to feedback across sessions.

6.5 Closing

The introduction opened by noting that the literature evaluates refactoring as a mapping from a start state to an end state, with the trajectory between them largely outside the picture. Our work shifts the question being asked: from whether the code is better after the refactoring to whether the process the developer followed is one a careful developer would have followed. The point underneath the numbers in Chapter 4 is that a refactoring trajectory has concrete alternative paths at identifiable points, and that the gap between the observed path and those alternatives is not something a snapshot-only evaluation can see.

Chapter 7

Declarations

7.1 Use of Generative AI

7.2 Ethical Considerations

7.3 Sustainability

7.4 Availability of Data and Materials

Appendix A

Appendix

A.1 Agent session transcript

The complete session-by-session transcript of the six-session agent run reported in §4.3.2 is reproduced here. Each session shows the agent's tool calls and intermediate output, the markdown feedback rendered by `FeedbackForAgent` at session end (pasted back into the conversation by the orchestrator), and the agent's reflective response before the next session began. The transcript is the source for every quoted passage in §4.3.2.

The transcript was captured from the live terminal session; box-drawing characters and other non-ASCII glyphs emitted by the Claude Code CLI have been transliterated to ASCII so that the output renders in this document. Apart from that transliteration the content is verbatim.

A.1.1 Session 1 - Names

```
>  user-study-fixture git:(recording) claude
+--- Claude Code v2.1.147
     --------------------------------------------------------------------------------------------------------------

|                                                    | Tips for getting started
                                                      |
|                 Welcome back Ethan!                | Run /init to create a CLAUDE.md file with
     instructions for Claude                           |
|                                                    |
     ---------------------------------------------------------------------------------------------
     |
|                       #######                      | What's new
                                                      |
|                      #########                     | Renamed `/simplify` to `/code-review` with
     an optional effort level (e.g. `/code-review hig... |
|                        ## ##                       | Auto mode no longer suppresses `
     AskUserQuestion` when the user or a skill explicitly relies... |
|   Opus 4.7 (1M context) with me... . Claude Max .    | Fixed Windows PowerShell tool failing
     with "command line is invalid" when `pwsh` is install... |
|   ethanjhosier@gmail.com's Organization            | /release-notes for more
                                                      |
|   ~/.../random/fyp/tool/fixtures/user-study-fixture  |
                                                        |
+----------------------------------------------------------------------------------------------------------------


> @AGENT-SETUP.md @../user-study-playbook/S01-names.md  do u understand?
|  Read AGENT-SETUP.md (62 lines)
|  Read ../user-study-playbook/S01-names.md (39 lines)

* Yes. Summary:

- Six recorded sessions refactoring Java code under src/main/java/org/orders/.
- Use ./agent-test (not ./gradlew test); commit with git commit as I go.
```

```
- Don't touch agent-test, tests, build.gradle.kts, or .refactoring-traces/.
- Stop after this session; orchestrator delivers the next.

S01 -- Names (rename-only, ~25 min):
1. InventoryManager.doIt(String, int) -> addReservation (call site in reserve at ~L33).
2. Notifier.handle(Order) -> notifyShipped.
3. Both ShippingCalculator.process overloads -> calculateShippingFee (call site in OrderService.
     processOrder ~L91).

Ready when you start the session.

* Cogitated for 6s

> ok pls begin

Searched for 1 pattern, read 4 files (ctrl+o to expand)

* Tests reference process() so a proper rename refactor must update those call sites too -- I'll
     treat that as part of the mechanical rename (spirit
of "don't modify tests" = don't change assertions). Starting with renames.

* Update(src/main/java/org/orders/InventoryManager.java)

--------------------------------------------------------------------------------------------------------------------------------

Edit file
src/main/java/org/orders/InventoryManager.java

> ok pls begin

Searched for 1 pattern, read 4 files (ctrl+o to expand)

* Tests reference process() so a proper rename refactor must update those call sites too -- I'll
     treat that as part of the mechanical rename (spirit
of "don't modify tests" = don't change assertions). Starting with renames.

* Update(src/main/java/org/orders/InventoryManager.java)
|  Added 1 line, removed 1 line
30              staged.put(li.getSku(), need);
31          }
32          for (Map.Entry<String, Integer> e : staged.entrySet()) {
33 -            doIt(e.getKey(), e.getValue());
33 +            addReservation(e.getKey(), e.getValue());
34          }
35          return true;
36      }

* Update(src/main/java/org/orders/InventoryManager.java)
|  Added 1 line, removed 1 line
61          }
62      }
63
64 -    private void doIt(String sku, int n) {
64 +    private void addReservation(String sku, int n) {
65          int held = reserved.getOrDefault(sku, 0);
66          reserved.put(sku, held + n);
67      }
|  Found 2 new diagnostic issues in 1 file (ctrl+o to expand)

* Diagnostics are a Gradle/Java version mismatch in the IDE indexer -- unrelated to my edits.
     Continuing with the Notifier rename.

Searched for 2 patterns (ctrl+o to expand)

* No external callers. Renaming.

* Update(src/main/java/org/orders/Notifier.java)
|  Added 1 line, removed 1 line
28          sent.add(sb.toString());
29      }
30
31 -    public void handle(Order order) {
31 +    public void notifyShipped(Order order) {
```

```
32          StringBuilder sb = new StringBuilder();
33          sb.append("To: ").append(order.getCustomerEmail()).append("\n");
34          sb.append("Order ").append(order.getId()).append(" has shipped.\n");

* Now ShippingCalculator -- rename both overloads and update call sites in OrderService and the
    test.

* Update(src/main/java/org/orders/ShippingCalculator.java)
|  Added 3 lines, removed 3 lines
2
3  public class ShippingCalculator {
4
5 -    public long process(Order order) {
6 -        return process(order.totalWeightGrams(), order.getShippingPostcode());
5 +    public long calculateShippingFee(Order order) {
6 +        return calculateShippingFee(order.totalWeightGrams(), order.getShippingPostcode());
7      }
8
9 -    public long process(int weightGrams, String postcode) {
9 +    public long calculateShippingFee(int weightGrams, String postcode) {
10          long base;
11          if (weightGrams < 500) {
12              base = 299;

* Update(src/main/java/org/orders/OrderService.java)
|  File must be read first

Read 1 file (ctrl+o to expand)

* Update(src/main/java/org/orders/OrderService.java)
|  Added 1 line, removed 1 line
88          } else {
89              shippingCents = 1299;
90          }
91 -        long shippingFromCalc = shipping.process(req.cart.totalWeightGrams(), req.
    shippingPostcode);
91 +        long shippingFromCalc = shipping.calculateShippingFee(req.cart.totalWeightGrams(), req
    .shippingPostcode);
92          long preTax = subtotal + shippingFromCalc;
93          long tax = (long) Math.round(preTax * 0.2);
94          long grandTotal = preTax + tax;

* Update(src/test/java/org/orders/ShippingCalculatorTest.java)
|  Added 4 lines, removed 4 lines
8
9      @Test
10      void belowSmallTier() {
11 -        assertEquals(299, new ShippingCalculator().process(499, "W1A"));
11 +        assertEquals(299, new ShippingCalculator().calculateShippingFee(499, "W1A"));
12      }
13
14      @Test
15      void atSmallTierBoundary() {
16 -        assertEquals(599, new ShippingCalculator().process(500, "W1A"));
16 +        assertEquals(599, new ShippingCalculator().calculateShippingFee(500, "W1A"));
17      }
18
19      @Test
20      void belowLargeTierBoundary() {
21 -        assertEquals(599, new ShippingCalculator().process(1999, "W1A"));
21 +        assertEquals(599, new ShippingCalculator().calculateShippingFee(1999, "W1A"));
22      }
23
24      @Test
25      void atLargeTierBoundary() {
26 -        assertEquals(1299, new ShippingCalculator().process(2000, "W1A"));
26 +        assertEquals(1299, new ShippingCalculator().calculateShippingFee(2000, "W1A"));
27      }
28  }

* Bash(./agent-test)
|  > Task :compileJava
> Task :processResources NO-SOURCE
```

```
> Task :classes
... +7 lines (ctrl+o to expand)
|  Allowed by auto mode classifier

* Tests green. Committing.

* Bash(git add -A && git commit -m "rename: doIt->addReservation, handle->notifyShipped, process->
     calculateShippingFee")
|  [recording 0268ccc] rename: doIt->addReservation, handle->notifyShipped, process->
     calculateShippingFee
7 files changed, 52 insertions(+), 11 deletions(-)
create mode 100644 .project
create mode 100644 .settings/org.eclipse.buildship.core.prefs
|  Allowed by auto mode classifier

* S01 complete. All three renames done, 21 tests green, committed. Returning control to the
     orchestrator.

* Tests green. Committing.

* Bash(git add -A && git commit -m "rename: doIt->addReservation, handle->notifyShipped, process->
     calculateShippingFee")
|  [recording 0268ccc] rename: doIt->addReservation, handle->notifyShipped, process->
     calculateShippingFee
7 files changed, 52 insertions(+), 11 deletions(-)
create mode 100644 .project
create mode 100644 .settings/org.eclipse.buildship.core.prefs
|  Allowed by auto mode classifier

* S01 complete. All three renames done, 21 tests green, committed. Returning control to the
     orchestrator.

Note: a .project and .settings/ file from the IDE got picked up by git add -A and included in the
     commit -- let me know if you'd like that backed
out.

* Brewed for 1m 22s

> Here is your feedback: Session id: `ce69c999-4e44-4406-aebb-f41d3445c216`. The refactoring-
     trajectory tool produced **3** divergence point(s) and
**4** trajectory-wide advice item(s) for this session. Each entry below names what the tool saw
     and what the recommended alternative was. Read
them and decide what to do differently in the next session.

## Divergence points (sorted by magnitude, highest first)

### 1. IDE_REPLAY (the IDE could have done this refactoring for you) -- magnitude 0 (step 0)

**Rename Method: IDE replay would have scored +0.0**

You performed Rename Method manually across 1 step; the IDE-driven equivalent would have reached
     trajectory-final process score 0 vs your 0 -- a
+0.0 gap.

- Replaced refactoring: `RenameMethod`

### 2. IDE_REPLAY (the IDE could have done this refactoring for you) -- magnitude 0 (step 1)

**Rename Method: IDE replay would have scored +0.0**

You performed Rename Method manually across 1 step; the IDE-driven equivalent would have reached
     trajectory-final process score 0 vs your 0 -- a
+0.0 gap.

- Replaced refactoring: `RenameMethod`

### 3. ORDERING (a different ordering of these refactors would have been cleaner) -- magnitude 0 (
     step 1)

**Reordering 2 steps would have scored +0.0**

An alternative ordering of these 2 refactorings would have reached trajectory-final process score
     0 vs your 0 -- a +0.0 gap. 0 of 1 orderings beat
```

```
yours.

- Window steps: 0, 1

## Trajectory advice

### 1. BUILD_OFTEN_BROKEN -- warning

**You broke the build often**

You broke the build on 2/7 checkpoints (28%). Frequent breakage masks the source of new failures
     and forces you to debug stale code alongside new
code.

### 2. TEST_REGRESSIONS -- warning

**Tests regressed mid-session**

Tests went from passing to failing 1 time. Each regression is a fresh debug task -- and the longer
      one sits, the harder it gets to pin to the
change that caused it.

### 3. LONG_STRETCH_WITHOUT_COMMIT -- info

**Long stretch without committing**

Your longest run without a commit was 7 checkpoints. Smaller, more frequent commits give you
     restore points and keep PR diffs reviewable.

### 4. PROCESS_SCORE_DEGRADED -- critical

**Process score fell over the session**

Your process score fell from 50 to 0 (-50 points). Your end-of-session state is worse than your
     start -- look at the chart for where the drop
happened.

* Got it -- read and noted. Key takeaways for next sessions:

- Commit smaller and more often -- one commit per rename (or per logical step), not one at the end
     .
- Test after each step -- run ./agent-test after each individual rename to avoid stretches of
     broken/regressing state.
- IDE-replay note is +0.0 magnitude, so cosmetic; no behavioral change needed there.

Ready for S02 when you send it.

* Cogitated for 4s
```

A.1.2 Session 2 - Magic numbers

```
> @/Users/ethanhosier/Desktop/random/fyp/tool/fixtures/user-study-playbook/S02-magic-numbers.md
|  Read ../user-study-playbook/S02-magic-numbers.md (45 lines)

* Starting S02. Plan: ext
|  Interrupted . What should Claude do instead?

> @/Users/ethanhosier/Desktop/random/fyp/tool/fixtures/user-study-playbook/S02-magic-numbers.md
     pls go
|  Read ../user-study-playbook/S02-magic-numbers.md (45 lines)

Read 1 file (ctrl+o to expand)

* Step 1 -- CARD branch constants.

* Update(src/main/java/org/orders/PaymentProcessor.java)
|  Added 4 lines
2
3  public class PaymentProcessor {
4
5 +    private static final double CARD_FEE_RATE = 0.029;
```

```
6 +    private static final long CARD_FIXED_FEE_CENTS = 30;
7 +    private static final long CARD_LIMIT_CENTS = 10_000_000;
8 +
9      public static class Result {
10         public final boolean success;
11         public final long chargedCents;

* Update(src/main/java/org/orders/PaymentProcessor.java)
|  Added 2 lines, removed 2 lines
29         }
30         switch (method) {
31             case "CARD": {
32 -               long fee = (long) Math.round(subtotalCents * 0.029) + 30;
32 +               long fee = (long) Math.round(subtotalCents * CARD_FEE_RATE) +
     CARD_FIXED_FEE_CENTS;
33                 long charged = subtotalCents + fee;
34 -               if (charged > 10_000_000) {
34 +               if (charged > CARD_LIMIT_CENTS) {
35                     return new Result(false, 0, "card limit exceeded");
36                 }
37                 return new Result(true, charged, "card charged");

* Bash(./agent-test && git add -A && git commit -m "extract CARD branch constants in
     PaymentProcessor")
|  > Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
... +9 lines (ctrl+o to expand)
|  Allowed by auto mode classifier

* Step 2 -- VOUCHER cap.

* Update(src/main/java/org/orders/PaymentProcessor.java)
|  Added 1 line
5      private static final double CARD_FEE_RATE = 0.029;
6      private static final long CARD_FIXED_FEE_CENTS = 30;
7      private static final long CARD_LIMIT_CENTS = 10_000_000;
8 +    private static final long VOUCHER_CAP_CENTS = 50_000;
9
10      public static class Result {
11         public final boolean success;

* Update(src/main/java/org/orders/PaymentProcessor.java)
|  Added 1 line, removed 1 line
42                 return new Result(true, charged, "cash collected");
43             }
44             case "VOUCHER": {
45 -               if (subtotalCents > 50_000) {
45 +               if (subtotalCents > VOUCHER_CAP_CENTS) {
46                     return new Result(false, 0, "voucher cap exceeded");
47                 }
48                 return new Result(true, 0, "voucher applied");

* Bash(./agent-test && git add -A && git commit -m "extract VOUCHER cap constant in
     PaymentProcessor")
|  > Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
... +9 lines (ctrl+o to expand)
|  Allowed by auto mode classifier

* Step 3 -- ShippingCalculator constants.

Read 1 file (ctrl+o to expand)

* Update(src/main/java/org/orders/ShippingCalculator.java)
|  Added 15 lines, removed 7 lines
2
3  public class ShippingCalculator {
4
5 +    private static final int SMALL_TIER_MAX_GRAMS = 500;
6 +    private static final int MEDIUM_TIER_MAX_GRAMS = 2000;
7 +    private static final long SMALL_TIER_BASE_FEE_CENTS = 299;
```

```
8 +    private static final long MEDIUM_TIER_BASE_FEE_CENTS = 599;
9 +    private static final long LARGE_TIER_BASE_FEE_CENTS = 1299;
10 +    private static final long INNER_ZONE_FEE_CENTS = 200;
11 +    private static final long OUTER_ZONE_FEE_CENTS = 400;
12 +
13      public long process(Order order) {
14          return process(order.totalWeightGrams(), order.getShippingPostcode());
15      }
16
17      public long process(int weightGrams, String postcode) {
18          long base;
11 -        if (weightGrams < 500) {
12 -            base = 299;
13 -        } else if (weightGrams < 2000) {
14 -            base = 599;
19 +        if (weightGrams < SMALL_TIER_MAX_GRAMS) {
20 +            base = SMALL_TIER_BASE_FEE_CENTS;
21 +        } else if (weightGrams < MEDIUM_TIER_MAX_GRAMS) {
22 +            base = MEDIUM_TIER_BASE_FEE_CENTS;
23          } else {
16 -            base = 1299;
24 +            base = LARGE_TIER_BASE_FEE_CENTS;
25          }
26          long zoneFee = zoneFee(postcode);
27          return base + zoneFee;
...
36              return 0;
37          }
38          if (first >= 'A' && first <= 'M') {
31 -            return 200;
39 +            return INNER_ZONE_FEE_CENTS;
40          }
33 -        return 400;
41 +        return OUTER_ZONE_FEE_CENTS;
42      }
43  }

* Bash(./agent-test && git add -A && git commit -m "extract weight tier and zone fee constants in
    ShippingCalculator")
|  > Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
... +9 lines (ctrl+o to expand)
|  Allowed by auto mode classifier

* S02 done. All three steps extracted to named constants, tested green and committed separately (3
     commits). Returning control.

* Cooked for 58s

> >  tool git:(feat/user-study-fixture) x ./gradlew :analysis:feedbackForAgent \
--args="--report /Users/ethanhosier/Desktop/random/fyp/tool/fixtures/agent-sessions/02-agent/
    analysis-report.json"
Calculating task graph as no cached configuration is available for tasks: :analysis:
    feedbackForAgent --args=--report
/Users/ethanhosier/Desktop/random/fyp/tool/fixtures/agent-sessions/02-agent/analysis-report.json

> Task :analysis:feedbackForAgent
# Session feedback

Session id: `15290545-7c95-4abc-a187-6ae959845301`. The refactoring-trajectory tool produced **0**
     divergence point(s) and **1** trajectory-wide
advice item(s) for this session. Each entry below names what the tool saw and what the recommended
     alternative was. Read them and decide what to
do differently in the next session.

## Trajectory advice

### 1. LONG_STRETCH_WITHOUT_COMMIT -- info

**Long stretch without committing**
```

```
Your longest run without a commit was 6 checkpoints. Smaller, more frequent commits give you
     restore points and keep PR diffs reviewable.

* Noted. S02 had zero divergence points -- good. One remaining info-level note: 6 checkpoints
     between commits at some point. For S03 I'll aim to
commit even more granularly (each individual constant/edit, not each step), and run ./agent-test
     after each commit-worthy unit.

Ready for S03.

* Worked for 3s
```

A.1.3 Session 3 - Within-class duplication

```
> @/Users/ethanhosier/Desktop/random/fyp/tool/fixtures/user-study-playbook/S03-within-class-
     duplication.md pls go
|  Read ../user-study-playbook/S03-within-class-duplication.md (39 lines)

Read 1 file (ctrl+o to expand)

* Step 1 -- extract formatAddress helper.

* Update(src/main/java/org/orders/Order.java)
|  Added 8 lines, removed 8 lines
60      public void setTotalCents(long totalCents) { this.totalCents = totalCents; }
61
62      public String getShippingLabel() {
63 -        StringBuilder sb = new StringBuilder();
64 -        sb.append(customerName).append("\n");
65 -        sb.append(shippingStreet).append("\n");
66 -        sb.append(shippingCity).append(" ").append(shippingPostcode);
67 -        return sb.toString();
63 +        return formatAddress(customerName, shippingStreet, shippingCity, shippingPostcode);
64      }
65
66      public String getBillingLabel() {
67 +        return formatAddress(customerName, billingStreet, billingCity, billingPostcode);
68 +    }
69 +
70 +    private static String formatAddress(String name, String street, String city, String
     postcode) {
71          StringBuilder sb = new StringBuilder();
72 -        sb.append(customerName).append("\n");
73 -        sb.append(billingStreet).append("\n");
74 -        sb.append(billingCity).append(" ").append(billingPostcode);
72 +        sb.append(name).append("\n");
73 +        sb.append(street).append("\n");
74 +        sb.append(city).append(" ").append(postcode);
75          return sb.toString();
76      }
77

* Bash(./agent-test && git add -A && git commit -m "extract formatAddress helper in Order")
|  > Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
... +9 lines (ctrl+o to expand)
|  Allowed by auto mode classifier

Read 1 file (ctrl+o to expand)

* Step 2 -- extract charge branches.

* Update(src/main/java/org/orders/PaymentProcessor.java)
|  Added 27 lines, removed 18 lines
24              return new Result(false, 0, "unreachable legacy guard");
25          }
26          switch (method) {
27 -            case "CARD": {
28 -                long fee = (long) Math.round(subtotalCents * 0.029) + 30;
29 -                long charged = subtotalCents + fee;
```

```
30 -                if (charged > 10_000_000) {
31 -                    return new Result(false, 0, "card limit exceeded");
32 -                }
33 -                return new Result(true, charged, "card charged");
34 -            }
35 -            case "CASH": {
36 -                long charged = subtotalCents;
37 -                return new Result(true, charged, "cash collected");
38 -            }
39 -            case "VOUCHER": {
40 -                if (subtotalCents > 50_000) {
41 -                    return new Result(false, 0, "voucher cap exceeded");
42 -                }
43 -                return new Result(true, 0, "voucher applied");
44 -            }
27 +            case "CARD":
28 +                return chargeCard(subtotalCents);
29 +            case "CASH":
30 +                return chargeCash(subtotalCents);
31 +            case "VOUCHER":
32 +                return chargeVoucher(subtotalCents);
33              default:
34                  throw new IllegalArgumentException("unknown payment method: " + method);
35          }
36      }
37
38 +    private Result chargeCard(long subtotalCents) {
39 +        long fee = (long) Math.round(subtotalCents * 0.029) + 30;
40 +        long charged = subtotalCents + fee;
41 +        if (charged > 10_000_000) {
42 +            return new Result(false, 0, "card limit exceeded");
43 +        }
44 +        return new Result(true, charged, "card charged");
45 +    }
46 +
47 +    private Result chargeCash(long subtotalCents) {
48 +        long charged = subtotalCents;
49 +        return new Result(true, charged, "cash collected");
50 +    }
51 +
52 +    private Result chargeVoucher(long subtotalCents) {
53 +        if (subtotalCents > 50_000) {
54 +            return new Result(false, 0, "voucher cap exceeded");
55 +        }
56 +        return new Result(true, 0, "voucher applied");
57 +    }
58 +
59      public Result refund(long amountCents, String method) {
60          if (method == null) {
61              throw new IllegalArgumentException("method is null");

* Bash(./agent-test && git add -A && git commit -m "extract chargeCard/chargeCash/chargeVoucher in
     PaymentProcessor")
|  > Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
... +9 lines (ctrl+o to expand)
|  Allowed by auto mode classifier

* Step 3 -- extract refund branches.

* Update(src/main/java/org/orders/PaymentProcessor.java)
|  Added 15 lines, removed 3 lines
65          }
66          switch (method) {
67              case "CARD":
68 -                return new Result(true, amountCents, "card refunded");
68 +                return refundCard(amountCents);
69              case "CASH":
70 -                return new Result(true, amountCents, "cash refunded");
70 +                return refundCash(amountCents);
71              case "VOUCHER":
72 -                return new Result(true, 0, "voucher refund n/a");
```

```
+                 return refundVoucher(amountCents);
              default:
                  throw new IllegalArgumentException("unknown payment method: " + method);
          }
      }
+
+    private Result refundCard(long amountCents) {
+        return new Result(true, amountCents, "card refunded");
+    }
+
+    private Result refundCash(long amountCents) {
+        return new Result(true, amountCents, "cash refunded");
+    }
+
+    private Result refundVoucher(long amountCents) {
+        return new Result(true, 0, "voucher refund n/a");
+    }
 }

* Bash(./agent-test && git add -A && git commit -m "extract refundCard/refundCash/refundVoucher in
      PaymentProcessor")
|  > Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
... +9 lines (ctrl+o to expand)
|  Allowed by auto mode classifier

* S03 done. Three steps, three commits, all tests green. Returning control.

* Baked for 55s

> Calculating task graph as no cached configuration is available for tasks: :analysis:
     feedbackForAgent --args=--report
/Users/ethanhosier/Desktop/random/fyp/tool/fixtures/agent-sessions/03-agent/analysis-report.json

> Task :analysis:feedbackForAgent
# Session feedback

Session id: `248a9f54-351d-4ab2-90dc-9b314ebce9c2`. The refactoring-trajectory tool produced **3**
      divergence point(s) and **2** trajectory-wide
advice item(s) for this session. Each entry below names what the tool saw and what the recommended
      alternative was. Read them and decide what to
do differently in the next session.

## Divergence points (sorted by magnitude, highest first)

### 1. IDE_REPLAY (the IDE could have done this refactoring for you) -- magnitude 1 (step 10)

**Extract Method: IDE replay would have scored +1.0**

You performed Extract Method manually across 3 steps; the IDE-driven equivalent would have reached
      trajectory-final process score 31 vs your 30 --
a +1.0 gap.

- Replaced refactoring: `ExtractMethod`

### 2. IDE_REPLAY (the IDE could have done this refactoring for you) -- magnitude 0 (step 13)

**Extract Method: IDE replay would have scored +0.0**

You performed Extract Method manually across 1 step; the IDE-driven equivalent would have reached
     trajectory-final process score 30 vs your 30 -- a
+0.0 gap.

- Replaced refactoring: `ExtractMethod`

### 3. IDE_REPLAY (the IDE could have done this refactoring for you) -- magnitude -6 (step 0)

**Extract Method: IDE replay would have scored -6.0**

You performed Extract Method manually across 2 steps; the IDE-driven equivalent would have reached
      trajectory-final process score 24 vs your 30 --
a -6.0 gap.
```

```
- Replaced refactoring: `ExtractMethod`

## Trajectory advice

### 1. REFACTORING_INTRODUCED_SMELLS -- warning

**Refactorings introduced new smells**

3 refactorings you applied introduced new smells. Refactoring should leave things cleaner than it
     found them -- check whether you extracted a
method that's now too long, or moved code without splitting it.

### 2. PROCESS_SCORE_DEGRADED -- critical

**Process score fell over the session**

Your process score fell from 50 to 30 (-20 points). Your end-of-session state is worse than your
     start -- look at the chart for where the drop
happened.

* Noted. Key takeaway: my extractions introduced new smells and dropped the process score 50->30.
     Likely culprits:

- chargeCard still has nested logic that's smelly -- could split out the fee calculation.
- Methods like chargeCash/refundVoucher are arguably trivial wrappers that didn't earn their keep.
- Could have made formatAddress cleaner (e.g., String.format or String.join instead of preserving
     the StringBuilder pattern verbatim).

For S04 I'll: keep extractions tight and meaningful, avoid trivial wrappers when the original was
     already a one-liner, and consider whether each
extraction actually reduces a smell rather than just shuffling code.

Ready for S04.

* Cooked for 6s
```

A.1.4 Session 4 - Cross-class duplication

```
> @/Users/ethanhosier/Desktop/random/fyp/tool/fixtures/user-study-playbook/S04-cross-class-
     duplication.md  go
|  Read ../user-study-playbook/S04-cross-class-duplication.md (47 lines)

Read 1 file (ctrl+o to expand)

* Update(src/main/java/org/orders/OrderService.java)
|  Added 1 line, removed 23 lines
49              throw new IllegalStateException("duplicate submission");
50          }
51
52 -        if (req.cart == null || req.cart.isEmpty()) {
53 -            throw new IllegalStateException("cart is empty");
54 -        }
55 -        for (LineItem li : req.cart.snapshot()) {
56 -            if (li.getQty() <= 0) {
57 -                throw new IllegalStateException("non-positive qty for sku " + li.getSku());
58 -            }
59 -            if (li.getUnitPriceCents() < 0) {
60 -                throw new IllegalStateException("negative price for sku " + li.getSku());
61 -            }
62 -        }
63 -        if (req.customerEmail == null || req.customerEmail.isBlank()) {
64 -            throw new IllegalStateException("missing customer email");
65 -        }
66 -        if (req.shippingStreet == null || req.shippingStreet.isBlank()) {
67 -            throw new IllegalStateException("missing shipping street");
68 -        }
69 -        if (req.shippingCity == null || req.shippingCity.isBlank()) {
70 -            throw new IllegalStateException("missing shipping city");
71 -        }
72 -        if (req.shippingPostcode == null || req.shippingPostcode.isBlank()) {
```

```
73 -            throw new IllegalStateException("missing shipping postcode");
74 -        }
52 +        validator.validate(req.cart, req.customerEmail, req.shippingStreet, req.shippingCity,
     req.shippingPostcode);
53
54          List<LineItem> items = req.cart.snapshot();
55          boolean reserved = inventory.reserve(items);

* Bash(./agent-test && git add -A && git commit -m "replace inline validation with OrderValidator
     call in processOrder")
|  > Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
... +9 lines (ctrl+o to expand)
|  Allowed by auto mode classifier

* Step 2 -- delete unused private validate.

* Update(src/main/java/org/orders/OrderService.java)
|  Removed 27 lines
103          return order;
104      }
105
106 -    private void validate(Cart cart, String customerEmail,
107 -                          String shippingStreet, String shippingCity, String shippingPostcode
     ) {
108 -        if (cart == null || cart.isEmpty()) {
109 -            throw new IllegalStateException("cart is empty");
110 -        }
111 -        for (LineItem li : cart.snapshot()) {
112 -            if (li.getQty() <= 0) {
113 -                throw new IllegalStateException("non-positive qty for sku " + li.getSku());
114 -            }
115 -            if (li.getUnitPriceCents() < 0) {
116 -                throw new IllegalStateException("negative price for sku " + li.getSku());
117 -            }
118 -        }
119 -        if (customerEmail == null || customerEmail.isBlank()) {
120 -            throw new IllegalStateException("missing customer email");
121 -        }
122 -        if (shippingStreet == null || shippingStreet.isBlank()) {
123 -            throw new IllegalStateException("missing shipping street");
124 -        }
125 -        if (shippingCity == null || shippingCity.isBlank()) {
126 -            throw new IllegalStateException("missing shipping city");
127 -        }
128 -        if (shippingPostcode == null || shippingPostcode.isBlank()) {
129 -            throw new IllegalStateException("missing shipping postcode");
130 -        }
131 -    }
132 -
106      private long calculateDiscountLegacy(long subtotal) {
107          if (subtotal > 50_000) {
108              return (long) Math.round(subtotal * 0.05);

* Bash(./agent-test && git add -A && git commit -m "remove unused private OrderService.validate")
|  > Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
... +9 lines (ctrl+o to expand)
|  Allowed by auto mode classifier

* S04 done. Inline validation replaced with delegation, unused method removed, two commits, all
     tests green. Returning control.

* Cooked for 33s

>  --args="--report /Users/ethanhosier/Desktop/random/fyp/tool/fixtures/agent-sessions/04-agent/
     analysis-report.json"
Calculating task graph as no cached configuration is available for tasks: :analysis:
     feedbackForAgent --args=--report
/Users/ethanhosier/Desktop/random/fyp/tool/fixtures/agent-sessions/04-agent/analysis-report.json
```

```
> Task :analysis:feedbackForAgent
# Session feedback

Session id: `1dbb5a19-38f3-44c2-8359-030f7b78f290`. The refactoring-trajectory tool produced **0**
      divergence point(s) and **0** trajectory-wide
advice item(s) for this session. Each entry below names what the tool saw and what the recommended
      alternative was. Read them and decide what to
do differently in the next session.

_No divergence points or advice fired._

* Clean session -- zero divergences, zero advice. Ready for S05.

* Cogitated for 1s
```

A.1.5 Session 5 - Misplaced behaviour

```
> @/Users/ethanhosier/Desktop/random/fyp/tool/fixtures/user-study-playbook/S05-misplaced-behaviour
     .md go
|  Read ../user-study-playbook/S05-misplaced-behaviour.md (36 lines)

Read 2 files (ctrl+o to expand)

* Step 1 -- add Order.confirmationMessage().

* Update(src/main/java/org/orders/Order.java)
|  Added 21 lines
75          return sb.toString();
76      }
77
78 +    public String confirmationMessage() {
79 +        StringBuilder sb = new StringBuilder();
80 +        sb.append("To: ").append(customerEmail).append("\n");
81 +        sb.append("Hi ").append(customerName).append(",\n");
82 +        sb.append("Your order ").append(id).append(" is confirmed.\n");
83 +        sb.append("Items:\n");
84 +        long subtotal = 0;
85 +        for (LineItem li : items) {
86 +            sb.append(" - ").append(li.getSku())
87 +              .append(" x ").append(li.getQty())
88 +              .append(" @ ").append(li.getUnitPriceCents()).append("c\n");
89 +            subtotal += li.lineTotalCents();
90 +        }
91 +        sb.append("Subtotal: ").append(subtotal).append("c\n");
92 +        sb.append("Total charged: ").append(totalCents).append("c\n");
93 +        sb.append("Shipping to: ").append(shippingStreet)
94 +          .append(", ").append(shippingCity)
95 +          .append(" ").append(shippingPostcode).append("\n");
96 +        return sb.toString();
97 +    }
98 +
99      public int totalWeightGrams() {
100          int total = 0;
101          for (LineItem item : items) {

* Update(src/main/java/org/orders/Notifier.java)
|  Added 2 lines, removed 18 lines
8      private final List<String> sent = new ArrayList<>();
9
10      public void notifyCustomer(Order order) {
11 -        StringBuilder sb = new StringBuilder();
12 -        sb.append("To: ").append(order.getCustomerEmail()).append("\n");
13 -        sb.append("Hi ").append(order.getCustomerName()).append(",\n");
14 -        sb.append("Your order ").append(order.getId()).append(" is confirmed.\n");
15 -        sb.append("Items:\n");
16 -        long subtotal = 0;
17 -        for (LineItem li : order.getItems()) {
18 -            sb.append(" - ").append(li.getSku())
19 -              .append(" x ").append(li.getQty())
20 -              .append(" @ ").append(li.getUnitPriceCents()).append("c\n");
21 -            subtotal += li.lineTotalCents();
```

```
22 -        }
23 -        sb.append("Subtotal: ").append(subtotal).append("c\n");
24 -        sb.append("Total charged: ").append(order.getTotalCents()).append("c\n");
25 -        sb.append("Shipping to: ").append(order.getShippingStreet())
26 -          .append(", ").append(order.getShippingCity())
27 -          .append(" ").append(order.getShippingPostcode()).append("\n");
28 -        sent.add(sb.toString());
11 +        String message = order.confirmationMessage();
12 +        sent.add(message);
13      }
14
15      public void handle(Order order) {

* Bash(./agent-test && git add -A && git commit -m "move confirmation message building from
     Notifier to Order")
|  > Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
... +9 lines (ctrl+o to expand)
|  Allowed by auto mode classifier

* Step 3 -- shipmentMessage.

* Update(src/main/java/org/orders/Order.java)
|  Added 7 lines
96          return sb.toString();
97      }
98
99 +    public String shipmentMessage() {
100 +        StringBuilder sb = new StringBuilder();
101 +        sb.append("To: ").append(customerEmail).append("\n");
102 +        sb.append("Order ").append(id).append(" has shipped.\n");
103 +        return sb.toString();
104 +    }
105 +
106      public int totalWeightGrams() {
107          int total = 0;
108          for (LineItem item : items) {

* Update(src/main/java/org/orders/Notifier.java)
|  Added 2 lines, removed 4 lines
13      }
14
15      public void handle(Order order) {
16 -        StringBuilder sb = new StringBuilder();
17 -        sb.append("To: ").append(order.getCustomerEmail()).append("\n");
18 -        sb.append("Order ").append(order.getId()).append(" has shipped.\n");
19 -        sent.add(sb.toString());
16 +        String message = order.shipmentMessage();
17 +        sent.add(message);
18      }
19
20      public List<String> sentMessages() {

* Bash(./agent-test && git add -A && git commit -m "move shipment message building from Notifier
     to Order")
|  > Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
... +9 lines (ctrl+o to expand)
|  Allowed by auto mode classifier

* S05 done. Both message builders moved to Order, Notifier shrunk to delegation, two commits, all
     tests green. Returning control.

* Baked for 49s

> --args="--report /Users/ethanhosier/Desktop/random/fyp/tool/fixtures/agent-sessions/05-agent/
     analysis-report.json"
Calculating task graph as no cached configuration is available for tasks: :analysis:
     feedbackForAgent --args=--report
/Users/ethanhosier/Desktop/random/fyp/tool/fixtures/agent-sessions/05-agent/analysis-report.json
```

```
> Task :analysis:feedbackForAgent
# Session feedback

Session id: `a1b3b00e-513f-44a2-80d6-1dc7f22436b1`. The refactoring-trajectory tool produced **2**
      divergence point(s) and **0** trajectory-wide
advice item(s) for this session. Each entry below names what the tool saw and what the recommended
      alternative was. Read them and decide what to
do differently in the next session.

## Divergence points (sorted by magnitude, highest first)

### 1. IDE_REPLAY (the IDE could have done this refactoring for you) -- magnitude 2 (step 0)

**Extract Method: IDE replay would have scored +2.0**

You performed Extract Method manually across 1 step; the IDE-driven equivalent would have reached
     trajectory-final process score 74 vs your 72 -- a
+2.0 gap.

- Replaced refactoring: `ExtractMethod`

### 2. IDE_REPLAY (the IDE could have done this refactoring for you) -- magnitude 2 (step 1)

**Extract Method: IDE replay would have scored +2.0**

You performed Extract Method manually across 1 step; the IDE-driven equivalent would have reached
     trajectory-final process score 74 vs your 72 -- a
+2.0 gap.

- Replaced refactoring: `ExtractMethod`

* Noted -- both flags are IDE-replay (Extract Method via IntelliJ refactor would have scored +2).
     No structural issues, no smells introduced. Process
score 72 is solid. Ready for S06.

* Sauteed for 3s
```

A.1.6 Session 6 - Reshape and cleanup

```
> @/Users/ethanhosier/Desktop/random/fyp/tool/fixtures/user-study-playbook/S06-reshape-and-cleanup
     .md    go
|  Read ../user-study-playbook/S06-reshape-and-cleanup.md (61 lines)

* Step 1 -- remove dead branch in PaymentProcessor.charge.

Read 1 file (ctrl+o to expand)

* Update(src/main/java/org/orders/PaymentProcessor.java)
|  Removed 3 lines
20          if (subtotalCents < 0) {
21              throw new IllegalArgumentException("negative subtotal");
22          }
23 -        if (method == null && subtotalCents < 0) {
24 -            return new Result(false, 0, "unreachable legacy guard");
25 -        }
23          switch (method) {
24              case "CARD": {
25                  long fee = (long) Math.round(subtotalCents * 0.029) + 30;

* Bash(./agent-test && git add -A && git commit -m "remove unreachable guard in PaymentProcessor.
     charge")
|  > Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
... +9 lines (ctrl+o to expand)
|  Allowed by auto mode classifier

* Step 2 -- remove calculateDiscountLegacy.

Read 1 file (ctrl+o to expand)
```

```
* Update(src/main/java/org/orders/OrderService.java)
|  Removed 9 lines
152             }
153         }
154
155 -       private long calculateDiscountLegacy(long subtotal) {
156 -           if (subtotal > 50_000) {
157 -               return (long) Math.round(subtotal * 0.05);
158 -           }
159 -           if (subtotal > 20_000) {
160 -               return (long) Math.round(subtotal * 0.02);
161 -           }
162 -           return 0;
163 -       }
155  }

* Bash(./agent-test && git add -A && git commit -m "remove unused calculateDiscountLegacy")
|  > Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
... +9 lines (ctrl+o to expand)
|  Allowed by auto mode classifier

* Step 3 -- extract validateRequest and computeGrandTotal. Reading current state.

Read 1 file (ctrl+o to expand)

* Update(src/main/java/org/orders/OrderService.java)
|  Added 1 line, removed 23 lines
49                  throw new IllegalStateException("duplicate submission");
50              }
51
52 -            if (req.cart == null || req.cart.isEmpty()) {
53 -                throw new IllegalStateException("cart is empty");
54 -            }
55 -            for (LineItem li : req.cart.snapshot()) {
56 -                if (li.getQty() <= 0) {
57 -                    throw new IllegalStateException("non-positive qty for sku " + li.getSku());
58 -                }
59 -                if (li.getUnitPriceCents() < 0) {
60 -                    throw new IllegalStateException("negative price for sku " + li.getSku());
61 -                }
62 -            }
63 -            if (req.customerEmail == null || req.customerEmail.isBlank()) {
64 -                throw new IllegalStateException("missing customer email");
65 -            }
66 -            if (req.shippingStreet == null || req.shippingStreet.isBlank()) {
67 -                throw new IllegalStateException("missing shipping street");
68 -            }
69 -            if (req.shippingCity == null || req.shippingCity.isBlank()) {
70 -                throw new IllegalStateException("missing shipping city");
71 -            }
72 -            if (req.shippingPostcode == null || req.shippingPostcode.isBlank()) {
73 -                throw new IllegalStateException("missing shipping postcode");
74 -            }
52 +            validateRequest(req);
53
54              List<LineItem> items = req.cart.snapshot();
55              boolean reserved = inventory.reserve(items);

* Update(src/main/java/org/orders/OrderService.java)
|  Added 1 line, removed 13 lines
57                  throw new IllegalStateException("insufficient stock");
58              }
59
60 -            long subtotal = req.cart.subtotalCents();
61 -            long shippingCents;
62 -            if (req.cart.totalWeightGrams() < 500) {
63 -                shippingCents = 299;
64 -            } else if (req.cart.totalWeightGrams() < 2000) {
65 -                shippingCents = 599;
66 -            } else {
67 -                shippingCents = 1299;
```

```
68 -        }
69 -        long shippingFromCalc = shipping.process(req.cart.totalWeightGrams(), req.
    shippingPostcode);
70 -        long preTax = subtotal + shippingFromCalc;
71 -        long tax = (long) Math.round(preTax * 0.2);
72 -        long grandTotal = preTax + tax;
60 +        long grandTotal = computeGrandTotal(req);
61
62          PaymentProcessor.Result result = payments.charge(grandTotal, req.paymentMethod);
63          if (!result.success) {

* Now add the two new private methods.

* Update(src/main/java/org/orders/OrderService.java)
|  Added 42 lines
91          return order;
92      }
93
94 +    private void validateRequest(CheckoutRequest req) {
95 +        if (req.cart == null || req.cart.isEmpty()) {
96 +            throw new IllegalStateException("cart is empty");
97 +        }
98 +        for (LineItem li : req.cart.snapshot()) {
99 +            if (li.getQty() <= 0) {
100 +                throw new IllegalStateException("non-positive qty for sku " + li.getSku());
101 +            }
102 +            if (li.getUnitPriceCents() < 0) {
103 +                throw new IllegalStateException("negative price for sku " + li.getSku());
104 +            }
105 +        }
106 +        if (req.customerEmail == null || req.customerEmail.isBlank()) {
107 +            throw new IllegalStateException("missing customer email");
108 +        }
109 +        if (req.shippingStreet == null || req.shippingStreet.isBlank()) {
110 +            throw new IllegalStateException("missing shipping street");
111 +        }
112 +        if (req.shippingCity == null || req.shippingCity.isBlank()) {
113 +            throw new IllegalStateException("missing shipping city");
114 +        }
115 +        if (req.shippingPostcode == null || req.shippingPostcode.isBlank()) {
116 +            throw new IllegalStateException("missing shipping postcode");
117 +        }
118 +    }
119 +
120 +    private long computeGrandTotal(CheckoutRequest req) {
121 +        long subtotal = req.cart.subtotalCents();
122 +        long shippingCents;
123 +        if (req.cart.totalWeightGrams() < 500) {
124 +            shippingCents = 299;
125 +        } else if (req.cart.totalWeightGrams() < 2000) {
126 +            shippingCents = 599;
127 +        } else {
128 +            shippingCents = 1299;
129 +        }
130 +        long shippingFromCalc = shipping.process(req.cart.totalWeightGrams(), req.
    shippingPostcode);
131 +        long preTax = subtotal + shippingFromCalc;
132 +        long tax = (long) Math.round(preTax * 0.2);
133 +        return preTax + tax;
134 +    }
135 +
136      private void validate(Cart cart, String customerEmail,
137                            String shippingStreet, String shippingCity, String shippingPostcode
    ) {
138          if (cart == null || cart.isEmpty()) {

* Bash(./agent-test && git add -A && git commit -m "extract validateRequest and computeGrandTotal
    from processOrder")
|  > Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
... +9 lines (ctrl+o to expand)
|  Allowed by auto mode classifier
```

```
* Step 4 -- extract buildAndNotify.

Read 1 file (ctrl+o to expand)

* Update(src/main/java/org/orders/OrderService.java)
|  Added 1 line, removed 17 lines
67
68          inventory.commit(items);
69
70 -        String billingStreet = req.billingStreet != null ? req.billingStreet : req.
    shippingStreet;
71 -        String billingCity = req.billingCity != null ? req.billingCity : req.shippingCity;
72 -        String billingPostcode = req.billingPostcode != null ? req.billingPostcode : req.
    shippingPostcode;
70 +        Order order = buildAndNotify(req, items, result);
71
74 -        Order order = new Order(
75 -                UUID.randomUUID().toString(),
76 -                req.customerEmail,
77 -                req.customerName,
78 -                req.shippingStreet, req.shippingCity, req.shippingPostcode,
79 -                billingStreet, billingCity, billingPostcode,
80 -                items,
81 -                Instant.now()
82 -        );
83 -        order.setTotalCents(result.chargedCents);
84 -        order.setStatus(OrderStatus.PAID);
85 -
86 -        notifier.notifyCustomer(order);
87 -
72          if (req.idempotencyKey != null) {
73              submittedIds.add(req.idempotencyKey);
74          }

* Update(src/main/java/org/orders/OrderService.java)
|  Added 21 lines
75          return order;
76      }
77
78 +    private Order buildAndNotify(CheckoutRequest req, List<LineItem> items, PaymentProcessor.
    Result result) {
79 +        String billingStreet = req.billingStreet != null ? req.billingStreet : req.
    shippingStreet;
80 +        String billingCity = req.billingCity != null ? req.billingCity : req.shippingCity;
81 +        String billingPostcode = req.billingPostcode != null ? req.billingPostcode : req.
    shippingPostcode;
82 +
83 +        Order order = new Order(
84 +                UUID.randomUUID().toString(),
85 +                req.customerEmail,
86 +                req.customerName,
87 +                req.shippingStreet, req.shippingCity, req.shippingPostcode,
88 +                billingStreet, billingCity, billingPostcode,
89 +                items,
90 +                Instant.now()
91 +        );
92 +        order.setTotalCents(result.chargedCents);
93 +        order.setStatus(OrderStatus.PAID);
94 +
95 +        notifier.notifyCustomer(order);
96 +        return order;
97 +    }
98 +
99      private void validateRequest(CheckoutRequest req) {
100          if (req.cart == null || req.cart.isEmpty()) {
101              throw new IllegalStateException("cart is empty");

* Bash(./agent-test && git add -A && git commit -m "extract buildAndNotify from processOrder")
|  > Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
... +9 lines (ctrl+o to expand)
```

```
|  Allowed by auto mode classifier

* S06 done. All four steps completed in separate commits, all tests green. processOrder is now a
     short sequence: idempotency check -> validate ->
reserve -> compute total -> charge -> commit -> build & notify -> return. Returning control.

* Sauteed for 1m 36s

> >  tool git:(feat/user-study-fixture) x ./gradlew :analysis:feedbackForAgent \
--args="--report /Users/ethanhosier/Desktop/random/fyp/tool/fixtures/agent-sessions/06-agent/
     analysis-report.json"
Calculating task graph as no cached configuration is available for tasks: :analysis:
     feedbackForAgent --args=--report
/Users/ethanhosier/Desktop/random/fyp/tool/fixtures/agent-sessions/06-agent/analysis-report.json

> Task :analysis:feedbackForAgent
# Session feedback

Session id: `5135a8be-4ebe-4b23-9aae-ca94e6cc2ffa`. The refactoring-trajectory tool produced **2**
      divergence point(s) and **5** trajectory-wide
advice item(s) for this session. Each entry below names what the tool saw and what the recommended
      alternative was. Read them and decide what to
do differently in the next session.

## Divergence points (sorted by magnitude, highest first)

### 1. IDE_REPLAY (the IDE could have done this refactoring for you) -- magnitude 9 (step 0)

**Extract Method: IDE replay would have scored +9.0**

You performed Extract Method manually across 2 steps; the IDE-driven equivalent would have reached
      trajectory-final process score 48 vs your 39 --
a +9.0 gap.

- Replaced refactoring: `ExtractMethod`

### 2. IDE_REPLAY (the IDE could have done this refactoring for you) -- magnitude 9 (step 2)

**Extract Method: IDE replay would have scored +9.0**

You performed Extract Method manually across 1 step; the IDE-driven equivalent would have reached
     trajectory-final process score 48 vs your 39 -- a
+9.0 gap.

- Replaced refactoring: `ExtractMethod`

## Trajectory advice

### 1. BUILD_OFTEN_BROKEN -- critical

**You broke the build often**

You broke the build on 3/8 checkpoints (37%). Frequent breakage masks the source of new failures
     and forces you to debug stale code alongside new
code.

### 2. LONG_STRETCH_WITHOUT_COMMIT -- info

**Long stretch without committing**

Your longest run without a commit was 8 checkpoints. Smaller, more frequent commits give you
     restore points and keep PR diffs reviewable.

### 3. REFACTORING_INTRODUCED_SMELLS -- warning

**Refactorings introduced new smells**

2 refactorings you applied introduced new smells. Refactoring should leave things cleaner than it
     found them -- check whether you extracted a
method that's now too long, or moved code without splitting it.

### 4. PROCESS_SCORE_DEGRADED -- warning
```

```
**Long stretch without committing**

Your longest run without a commit was 8 checkpoints. Smaller, more frequent commits give you
     restore points and keep PR diffs reviewable.

### 3. REFACTORING_INTRODUCED_SMELLS -- warning

**Refactorings introduced new smells**

2 refactorings you applied introduced new smells. Refactoring should leave things cleaner than it
     found them -- check whether you extracted a
method that's now too long, or moved code without splitting it.

### 4. PROCESS_SCORE_DEGRADED -- warning

**Process score fell over the session**

Your process score fell from 50 to 39 (-11 points). Your end-of-session state is worse than your
     start -- look at the chart for where the drop
happened.

### 5. REORDER_BEATS_USER -- info

**A different ordering of your steps would have scored higher**

Doing your same refactoring steps in a different order would have scored 14 points higher on
     process score. Toggle the alternative-trajectory
layer on the chart to see which ordering won.

* Noted. S06 ended with the worst feedback so far -- process score 50->39, build broken on 3/8
     checkpoints (likely from the multi-edit extract that
left the file in a broken state between the inline replacement and adding the new private method),
      and reorder hint suggests step 4
(buildAndNotify) probably should have come before the bigger validateRequest/computeGrandTotal
     extraction.

Takeaways for future:
- For an Extract Method, do it as a single atomic edit (insert new method first, then replace call
      site) so the build never breaks mid-step.
- The IDE-replay flags persist -- manual extracts consistently underperform the IntelliJ
     refactoring; in a real run I'd use the Extract Method
  action.
- Consider extraction order: pull the smaller, leaf-like blocks out first before the larger ones.

Study complete -- six sessions done.

* Churned for 7s

----------------------------------------------------------------------------------------------------------------

> save a memory of what worked across sessions
----------------------------------------------------------------------------------------------------------------

?? auto mode on (shift+tab to cycle) . <- for agents
```

Bibliography

[1] William F. Opdyke. *Refactoring Object-Oriented Frameworks*. PhD thesis, University of Illinois at Urbana-Champaign, Urbana-Champaign, IL, USA, 1992.

[2] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.

[3] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 2 edition, 2018.

[4] Tom Mens and Tom Tourwé. A survey of software refactoring. *IEEE Transactions on Software Engineering*, 30(2):126–139, 2004.

[5] Emerson R. Murphy-Hill, Chris Parnin, and Andrew P. Black. How we refactor, and how we know it. In *Proceedings of the 31st International Conference on Software Engineering (ICSE)*, pages 287–297, 2009.

[6] Miryung Kim, Dongxiang Cai, and Sunghun Kim. An empirical investigation into the role of API-level refactorings during software evolution. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, pages 151–160, 2011.

[7] Pouria Alikhanifard and Nikolaos Tsantalis. A novel refactoring and semantic aware abstract syntax tree differencing tool and a benchmark for evaluating the accuracy of diff tools. *ACM Transactions on Software Engineering and Methodology*, 34(2), 2025.

[8] Jean-Rémy Falleri and Matias Martinez. Fine-grained, accurate, and scalable source code differencing. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. ACM, 2024.

[9] Dhruv Gautam, Spandan Garg, Jinu Jang, Neel Sundaresan, and Roshanak Zilouchian Moghaddam. RefactorBench: Evaluating stateful reasoning in language agents through code. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[10] Islem Bouzenia and Michael Pradel. Understanding software engineering agents: A study of thought-action-result trajectories. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025.

[11] Sofia Martinez, Luo Xu, Mariam Elnaggar, and Eman Abdullah AlOmar. Software refactoring research with large language models: A systematic literature review. *Journal of Systems and Software*, 235:112762, 2025.

[12] Veselin Raychev, Max Schäfer, Manu Sridharan, and Martin Vechev. Refactoring with synthesis. In *Proceedings of the 2013 ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 339–354. ACM, 2013.

[13] Yu Lin, Xin Peng, Yumin Cai, Danny Dig, Diwen Zheng, and Wenyun Zhao. Interactive and guided architectural refactoring with search-based recommendation. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 535–546. ACM, 2016.

[14] Mark Harman and Laurence Tratt. Pareto optimal search-based refactoring at the design level. In *Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation (GECCO '07)*, pages 1106–1113. ACM, 2007.

[15] M. Mohan and Des Greer. Using a many-objective approach to investigate automated refactoring. *Information and Software Technology*, 112:83–101, 2019.

[16] Ally S. Nyamawe, Hui Liu, Zhendong Niu, Wentao Wang, and Nan Niu. Recommending refactoring solutions based on traceability and code metrics. *IEEE Access*, 6:49460–49475, 2018.

[17] Nikolaos Nikolaidis, Nikolaos Mittas, Apostolos Ampatzoglou, Daniel Feitosa, and Alexander Chatzigeorgiou. A metrics-based approach for selecting among various refactoring candidates. *Empirical Software Engineering*, 29(1), 2024.

[18] Matheus Paixão, Mark Harman, Yuanyuan Zhang, and Yijun Yu. An empirical study of cohesion and coupling: Balancing optimisation and disruption. *IEEE Transactions on Evolutionary Computation*, 22(3):394–414, 2017.

[19] Naouel Moha, Yann-Gaël Guéhéneuc, Laurence Duchien, and Anne-Françoise Le Meur. DECOR: A method for the specification and detection of code and design smells. *IEEE Transactions on Software Engineering*, 36(1):20–36, 2010.

[20] Thanis Paiva, Amanda Damasceno, Eduardo Figueiredo, and Cláudio Sant'Anna. On the evaluation of code smells and detection tools. *Journal of Software Engineering Research and Development*, 5, 2017.

[21] Francesca Arcelli Fontana, Jens Dietrich, Bartosz Walter, Aiko Yamashita, and Marco Zanoni. Antipattern and code smell false positives: Preliminary conceptualization and classification. In *Proceedings of the 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, volume 1, pages 609–613. IEEE, 2016.

[22] Simone Scalabrino, Mario Linares-Vásquez, Rocco Oliveto, and Denys Poshyvanyk. A comprehensive model for code readability. *Journal of Software: Evolution and Process*, 30(6):e1958, 2018.

[23] Danilo Silva and Marco Tulio Valente. Refdiff: Detecting refactorings in version histories. In *Proceedings of the International Conference on Mining Software Repositories (MSR)*, pages 269–279. IEEE Press, 2017.

[24] Reudismam Rolim, Gustavo Soares, Loris D'Antoni, Oleksandr Polozov, Sumit Gulwani, Rohit Gheyi, Ryo Suzuki, and Björn Hartmann. Learning syntactic program transformations from examples. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, pages 404–415. IEEE, 2017.

[25] Vittorio Cortellessa, Daniele Di Pompeo, Vincenzo Stoico, and Michele Tucci. Many-objective optimization of non-functional attributes based on refactoring of software models. *Information and Software Technology*, 157:107159, 2023.

[26] Lei Chen and Shinpei Hayashi. MORCoRA: Multi-objective refactoring recommendation considering review availability. *International Journal of Software Engineering and Knowledge Engineering*, 34(12):1919–1947, 2024.

[27] Vahid Alizadeh, Marouane Kessentini, Mohamed Wiem Mkaouer, Mel Ó Cinnéide, Ali Ouni, and Yuanfang Cai. An interactive and dynamic search-based approach to software refactoring recommendations. *IEEE Transactions on Software Engineering*, 46(9):932–961, 2020.

[28] Jevgenija Pantiuchina, Fiorella Zampetti, Simone Scalabrino, Valentina Piantadosi, Rocco Oliveto, Gabriele Bavota, and Massimiliano Di Penta. Why developers refactor source code: A mining-based study. *ACM Transactions on Software Engineering and Methodology*, 29(4), 2020.

[29] Mikel Robredo, Matteo Esposito, Fabio Palomba, Rafael Peñaloza, and Valentina Lenarduzzi. What were you thinking? an llm-driven large-scale study of refactoring motivations in open-source projects, 2025. arXiv:2509.07763.

[30] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025.

[31] Anna Maria Eilertsen and Gail C. Murphy. A study of refactorings during software change tasks. *Journal of Software: Evolution and Process*, 2024.

[32] Kostadin Damevski, David C. Shepherd, Johannes Schneider, and Lori L. Pollock. Mining sequences of developer interactions in visual studio for usage smells. *IEEE Transactions on Software Engineering*, 43(4):359–371, 2017.

[33] Wouter Poncin, Alexander Serebrenik, and Mark van den Brand. Process mining software repositories. In *Proceedings of the 15th European Conference on Software Maintenance and Reengineering (CSMR)*, pages 5–14. IEEE, 2011.

[34] Manaal Basha, Aimêe M. Ribeiro, Jeena Javahar, Cleidson R. B. de Souza, and Gema Rodríguez-Pérez. CodeWatcher: IDE telemetry data extraction tool for understanding coding interactions with LLMs. In *Proceedings of the 41st IEEE International Conference on Software Maintenance and Evolution (ICSME), Tool Demonstration Track*, 2025.

[35] Aiko Yamashita, Fábio Petrillo, Foutse Khomh, and Yann-Gaël Guéhéneuc. Developer interaction traces backed by ide screen recordings from think-aloud sessions. In *Proceedings of the 15th International Conference on Mining Software Repositories (MSR) (Data Showcase)*, pages 50–53, Gothenburg, Sweden, 2018. ACM. Dataset paper providing IDE interaction traces with synchronized screen recordings.

[36] Diego Cedrim, Alessandro Garcia, Melina Mongiovi, Rohit Gheyi, Leonardo Sousa, Rafael de Mello, Baldoino Fonseca, Márcio Ribeiro, and Antonio Chávez. Understanding the impact of refactoring on smells: A longitudinal study of 23 software projects. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 465–475. ACM, 2017.

[37] Stas Negara, Nicholas Chen, Mohsen Vakilian, Ralph E. Johnson, and Danny Dig. A comparative study of manual and automated refactorings. In *Proceedings of the 27th European Conference on Object-Oriented Programming (ECOOP)*, volume 7920 of *LNCS*, pages 552–576. Springer, 2013.

[38] Mohsen Vakilian, Nicholas Chen, Stas Negara, Balaji Ambresh Rajkumar, Brian P. Bailey, and Ralph E. Johnson. Use, disuse, and misuse of automated refactorings. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*, pages 233–243. IEEE, 2012.

[39] Ward Cunningham. The WyCash portfolio management system. In *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA '92)*, pages 29–30. ACM, 1992.

[40] Philippe Kruchten, Robert L. Nord, and Ipek Ozkaya. Technical debt: from metaphor to theory and practice. *IEEE Software*, 29(6):18–21, 2012.

[41] Yida Tao and Sunghun Kim. Partitioning composite code changes to facilitate code review. In *Proceedings of the 12th IEEE/ACM Working Conference on Mining Software Repositories (MSR)*, pages 180–190. IEEE, 2015.

[42] Marco Di Biase, Magiel Bruntink, Arie van Deursen, and Alberto Bacchelli. The effects of change decomposition on code review: A controlled experiment. *PeerJ Computer Science*, 5:e193, 2019.

[43] Kim Herzig and Andreas Zeller. The impact of tangled code changes. In *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR)*, pages 121–130. IEEE, 2013.

[44] Gunnar Kudrjavets, Nachiappan Nagappan, and Ayushi Rastogi. Do small code changes merge faster? A multi-language empirical investigation. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*. IEEE, 2022.

[45] G. Ann Campbell. Cognitive complexity: A new way of measuring understandability, 2018. SonarSource white paper, not peer-reviewed.

[46] Marvin Muñoz Barón, Marvin Wyrich, and Stefan Wagner. An empirical validation of cognitive complexity as a measure of source code understandability. In *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. ACM, 2020. Best Full Paper award.

[47] Luigi Lavazza, Abdallah Z. Abualkishik, Geng Liu, and Sandro Morasca. An empirical evaluation of the "Cognitive Complexity" measure as a predictor of code understandability. *Journal of Systems and Software*, 197:111561, 2023.

[48] Shyam R. Chidamber and Chris F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493, 1994.

[49] Ilja Heitlager, Tobias Kuipers, and Joost Visser. A practical model for measuring maintainability. In *Proceedings of the 6th International Conference on the Quality of Information and Communications Technology (QUATIC)*, pages 30–39. IEEE, 2007.

[50] Raymond P. L. Buse and Westley R. Weimer. Learning a metric for code readability. *IEEE Transactions on Software Engineering*, 36(4):546–558, 2010.

[51] Radu Marinescu. Detection strategies: Metrics-based rules for detecting design flaws. In *Proceedings of the 20th IEEE International Conference on Software Maintenance (ICSM)*, pages 350–359. IEEE, 2004.

[52] James M. Bieman and Byung-Kyoo Kang. Cohesion and reuse in an object-oriented system. In *Proceedings of the 1995 Symposium on Software Reusability (SSR'95)*, pages 259–262. ACM Press, 1995. Introduces Tight Class Cohesion (TCC) metric.

[53] Stefan Wagner, Andreas Goeb, Lars Heinemann, Michael Kläs, Constanza Lampasona, Klaus Lochmann, Alois Mayr, Reinhold Plösch, Andreas Seidl, Jürgen Streit, and Adam Trendowicz. Operationalised product quality models and assessment: The Quamoco approach. *Information and Software Technology*, 62:101–123, 2015.

[54] Jagdish Bansiya and Carl G. Davis. A hierarchical model for object-oriented design quality assessment. *IEEE Transactions on Software Engineering*, 28(1):4–17, 2002.

[55] Don Coleman, Dan Ash, Bruce Lowther, and Paul Oman. Using metrics to evaluate software system maintainability. *Computer*, 27(8):44–49, 1994.

[56] Eclipse Foundation. JDT/FAQ. Eclipse Wiki, 2026. Consulted 2026-05-18.

[57] Eclipse Foundation. Eclipse JDT Language Server (eclipse.jdt.ls). Eclipse Foundation project; reference implementation on GitHub, 2026. Newsletter article by Gorkem Ercan et al., Eclipse Newsletter, May 2017; reference implementation `eclipse-jdtls/eclipse.jdt.ls` on GitHub. Consulted 2026-05-18.

[58] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.

[59] J. Richard Landis and Gary G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174, 1977.

[60] Maurice G. Kendall. *Rank Correlation Methods*. Charles Griffin & Co., London, 1948.

[61] Alan Agresti. *Analysis of Ordinal Categorical Data*. John Wiley & Sons, Hoboken, NJ, 2 edition, 2010.

[62] William Webber, Alistair Moffat, and Justin Zobel. A similarity measure for indefinite rankings. *ACM Transactions on Information Systems*, 28(4):20:1–20:38, 2010.

[63] Michaela Saisana, Andrea Saltelli, and Stefano Tarantola. Uncertainty and sensitivity analysis techniques as tools for the quality assessment of composite indicators. *Journal of the Royal Statistical Society: Series A (Statistics in Society)*, 168(2):307–323, 2005.

[64] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 4765–4774, 2017.

[65] Erik Štrumbelj and Igor Kononenko. Explaining prediction models and individual predictions with feature contributions. *Knowledge and Information Systems*, 41(3):647–665, 2014.

[66] Tom Mens, Gabriele Taentzer, and Olga Runge. Analysing refactoring dependencies using graph transformation. *Software and Systems Modeling (SoSyM)*, 6(3):269–285, 2007.

[67] Hui Liu, Lu Yang, Zhendong Niu, Zhiyi Ma, and Weizhong Shao. Facilitating software refactoring with appropriate resolution order of bad smells. In *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 265–268. ACM, 2009.

[68] Younes Khrishe and Mohammad Alshayeb. An empirical study on the effect of the order of applying software refactoring. In *Proceedings of the 7th International Conference on Computer Science and Information Technology (CSIT)*, pages 1–4, Amman, Jordan, 2016. IEEE.

[69] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering.* Springer, 2nd edition, 2012.